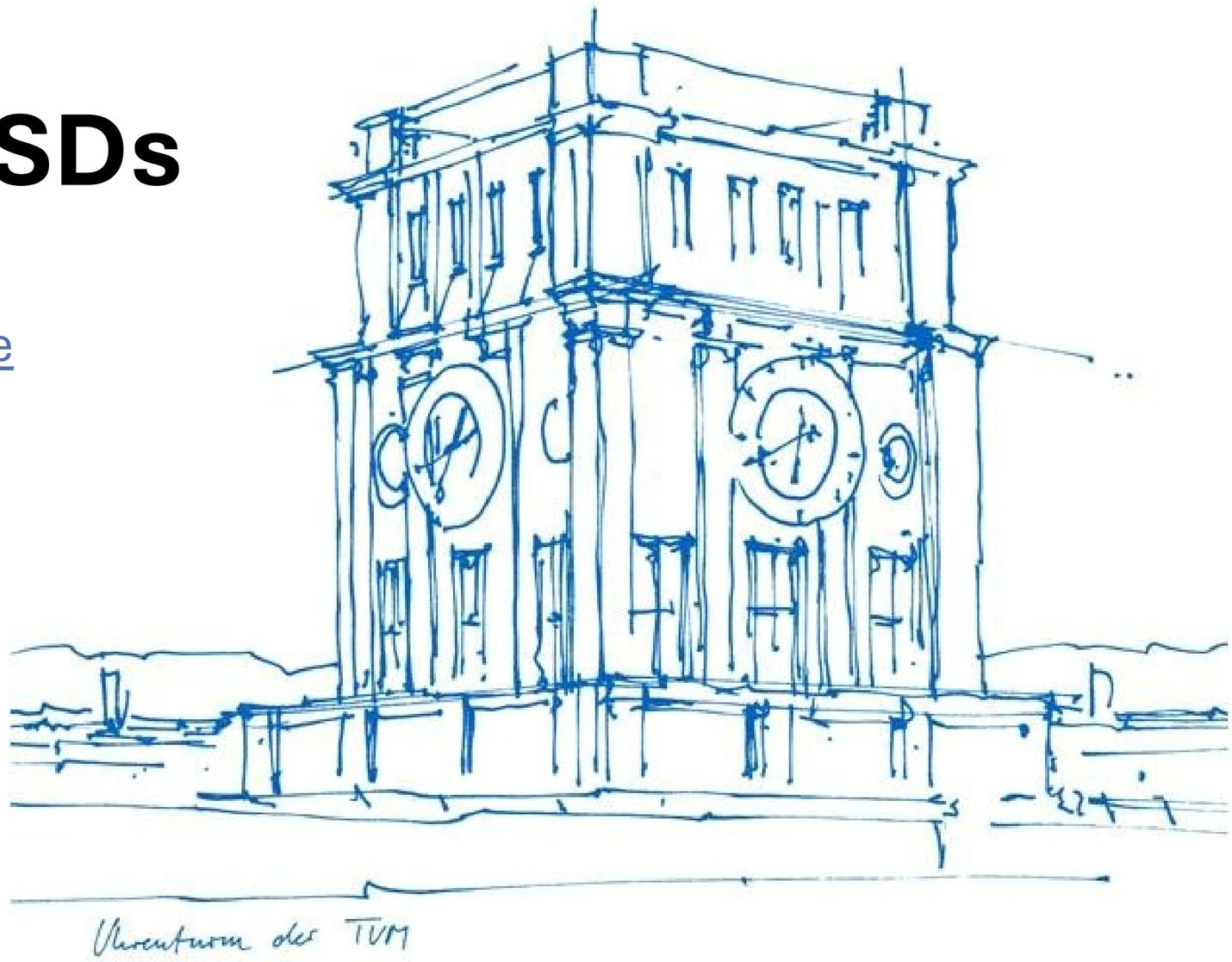
leanstore

How to Write to SSDs

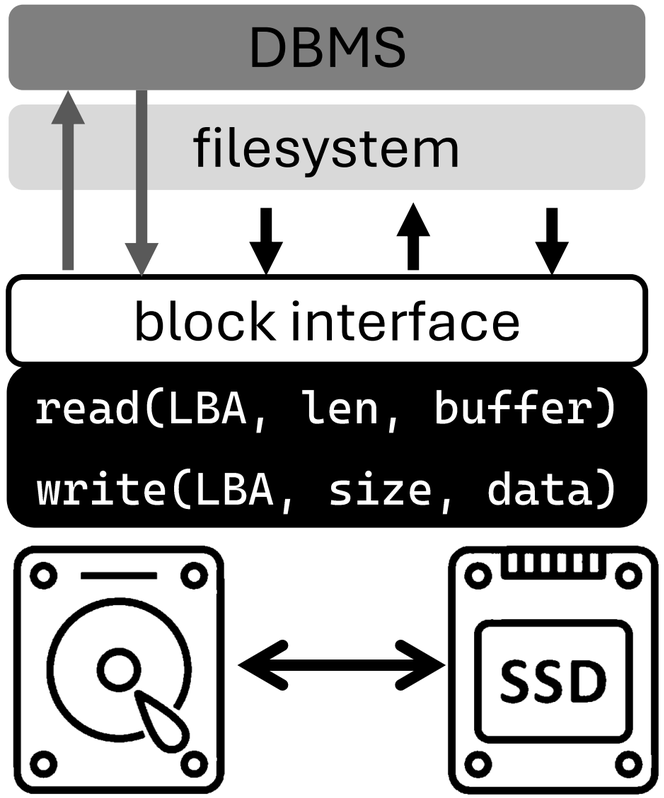
Bohyun (Lia) Lee bohyun.lee@tum.de

with **Tobias Ziegler** and **Viktor Leis**



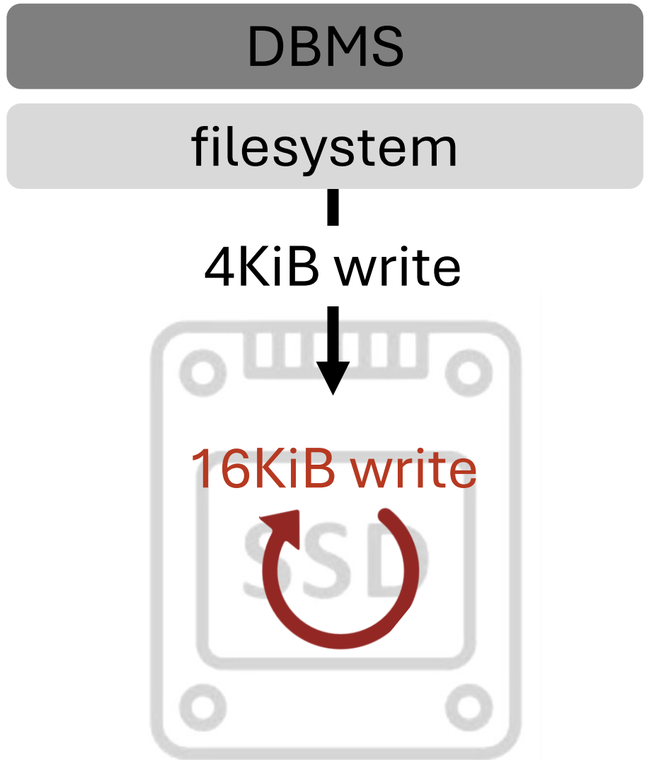
Two different types of block devices

- HDDs and SSDs both present ***block interface*** to the host
- Logical Block Address (LBA) → byte offset



SSD writes are different from disk writes

- SSD writes are special due to internal *write amplification*
  e.g., 4KiB write request can be amplified to 16KiB



Implications of SSD WAF

- High SSD WAF leads to
  - lower SSD throughput → lower DBMS throughput

Implications of SSD WAF

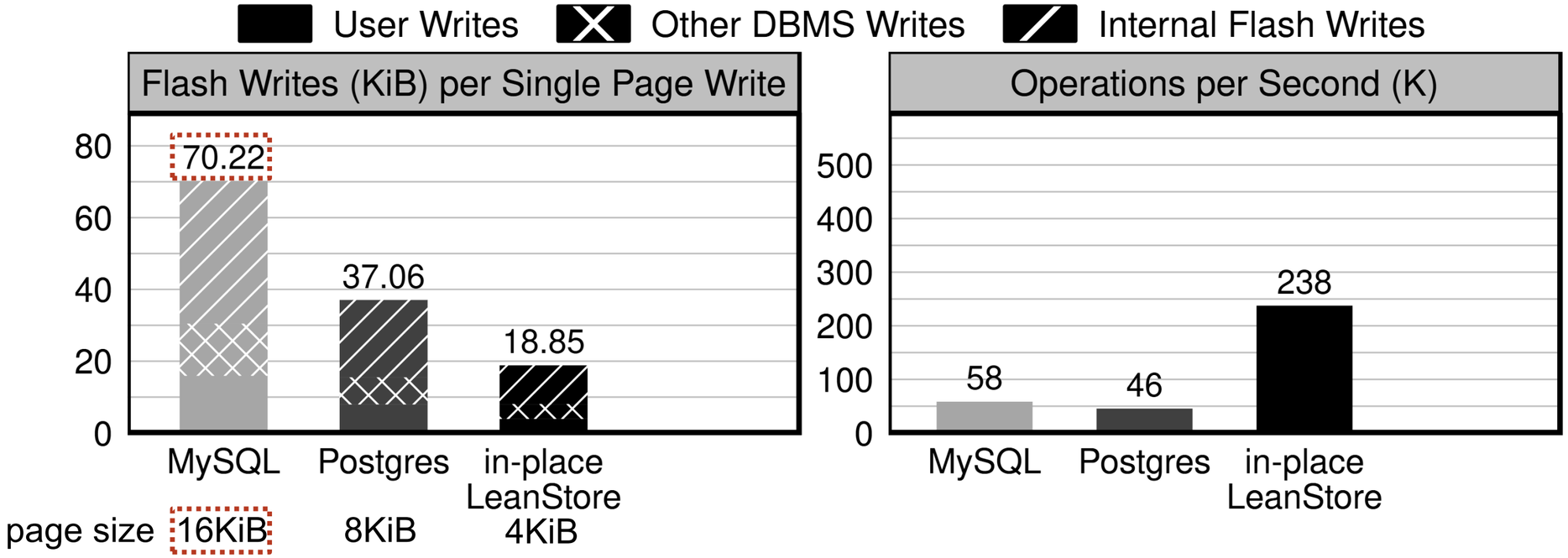
- High SSD WAF leads to
  - lower SSD throughput → lower DBMS throughput
  - reduced SSD lifespan
    E.g., 400MB/s writes break an SSD (5 DWPD) in 2 months

Implications of SSD WAF

- High SSD WAF leads to
  - lower SSD throughput → lower DBMS throughput
  - reduced SSD lifespan
    E.g., 400MB/s writes break an SSD (5 DWPD) in 2 months
- ***In general, fewer writes, the better!***

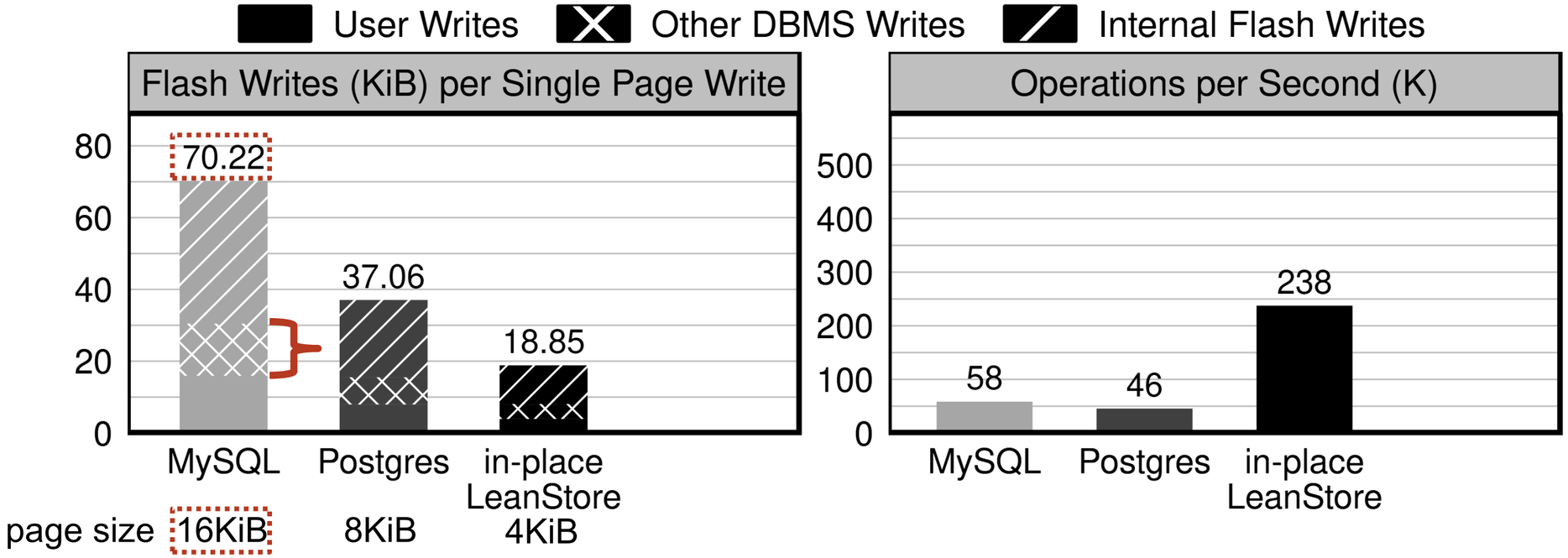
DBMSs suffer from write amplification

- User writes are amplified by 4.3-4.7x in three DBMSes
- Culprit 1:
- Culprit 2:



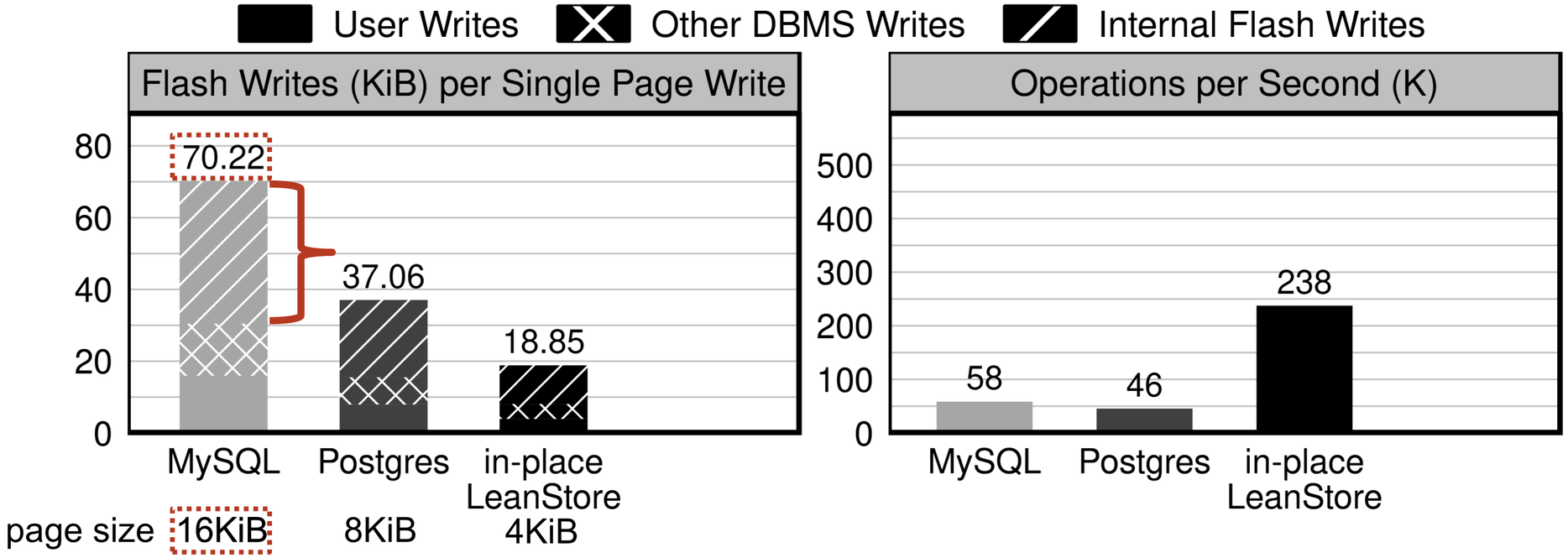
Two culprits behind write amplification

- User writes are amplified by 4.3-4.7x in three DBMSes
- Culprit 1: DBMSs themselves write more (i.e., torn page write protection *doubles* writes)
- Culprit 2:

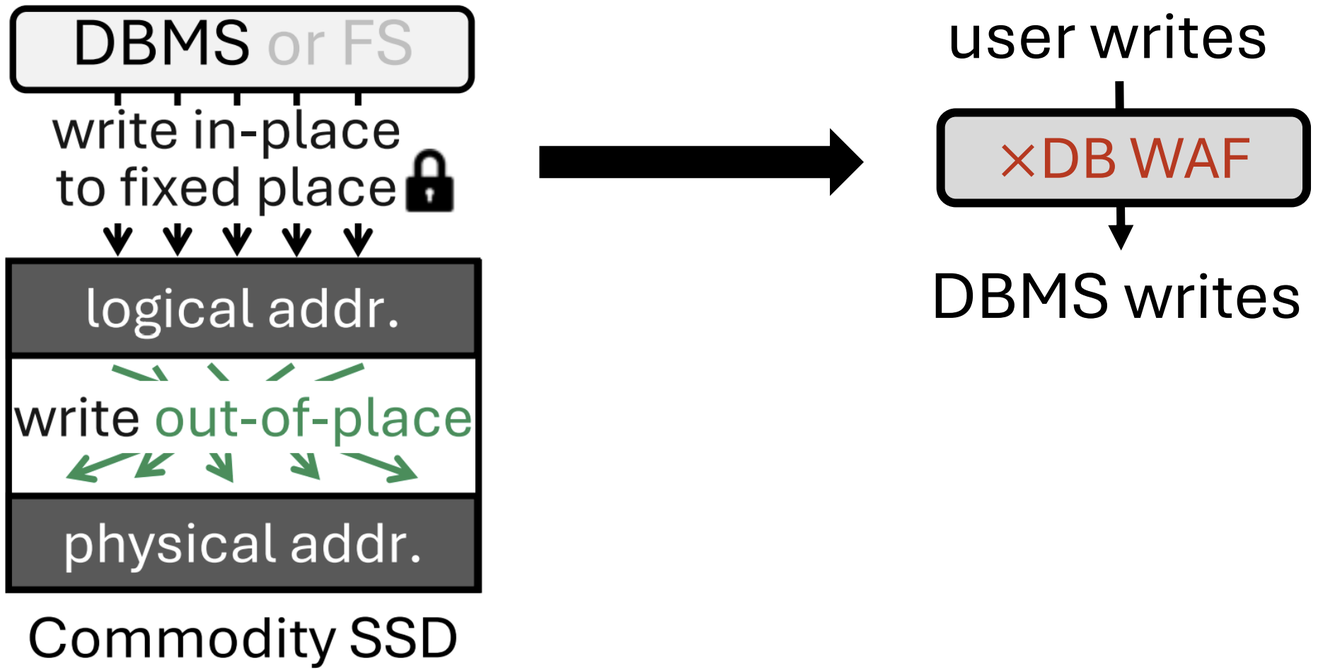


Two culprits behind write amplification

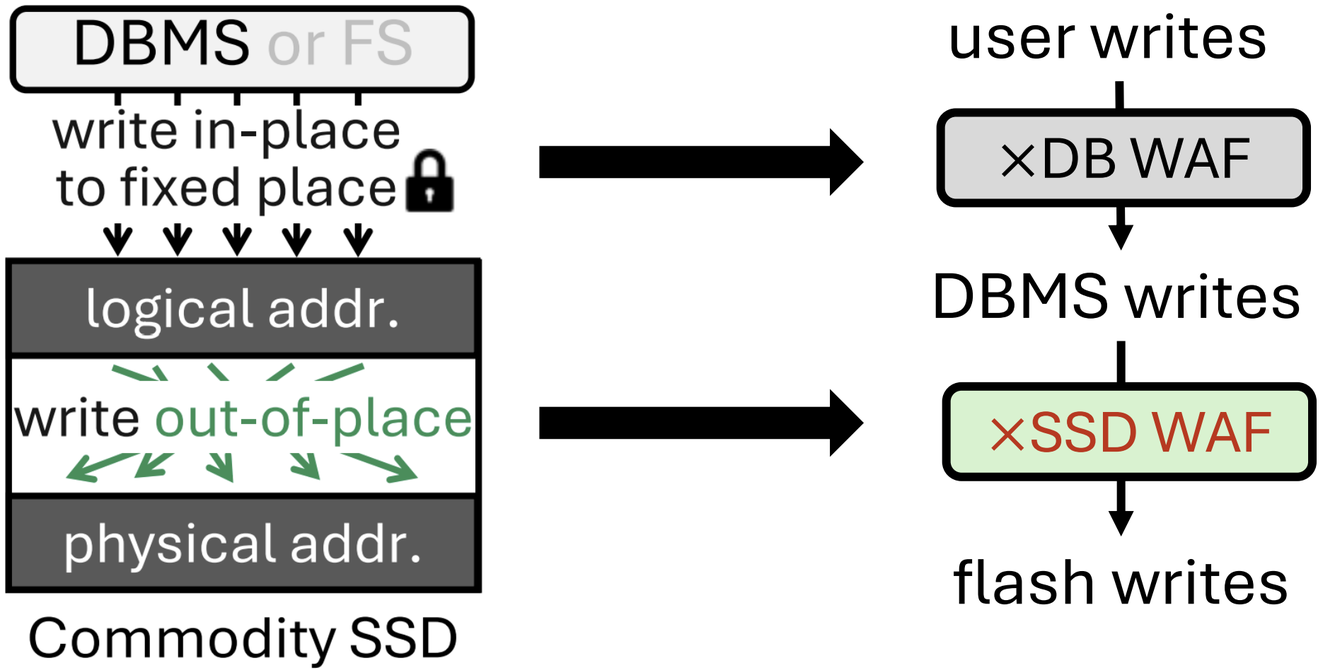
- User writes are amplified by 4.3-4.7x in three DBMSes
- Culprit 1: DBMSs themselves write more (i.e., torn page write protection *doubles* writes)
- Culprit 2: SSDs internally amplify DBMS writes further



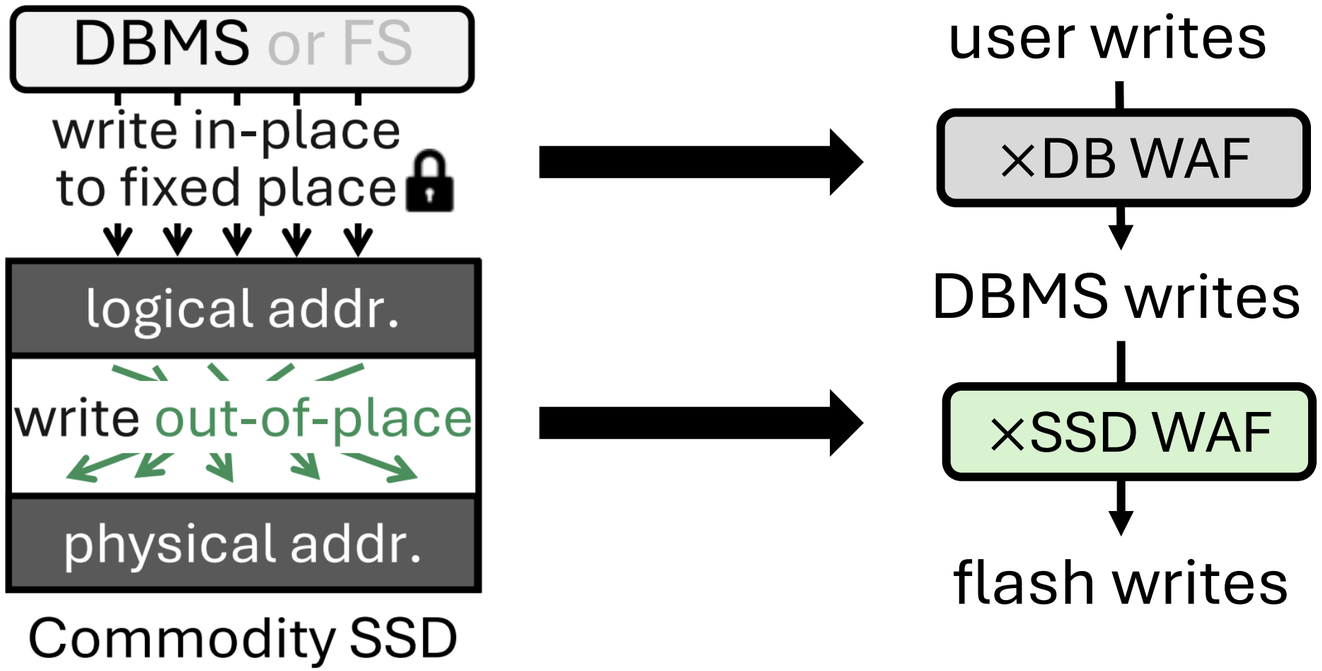
Total WAF


DBMS or FS
write in-place
to fixed place
logical addr.
write out-of-place
physical addr.
Commodity SSD
user writes
×DB WAF
DBMS writes

Total WAF

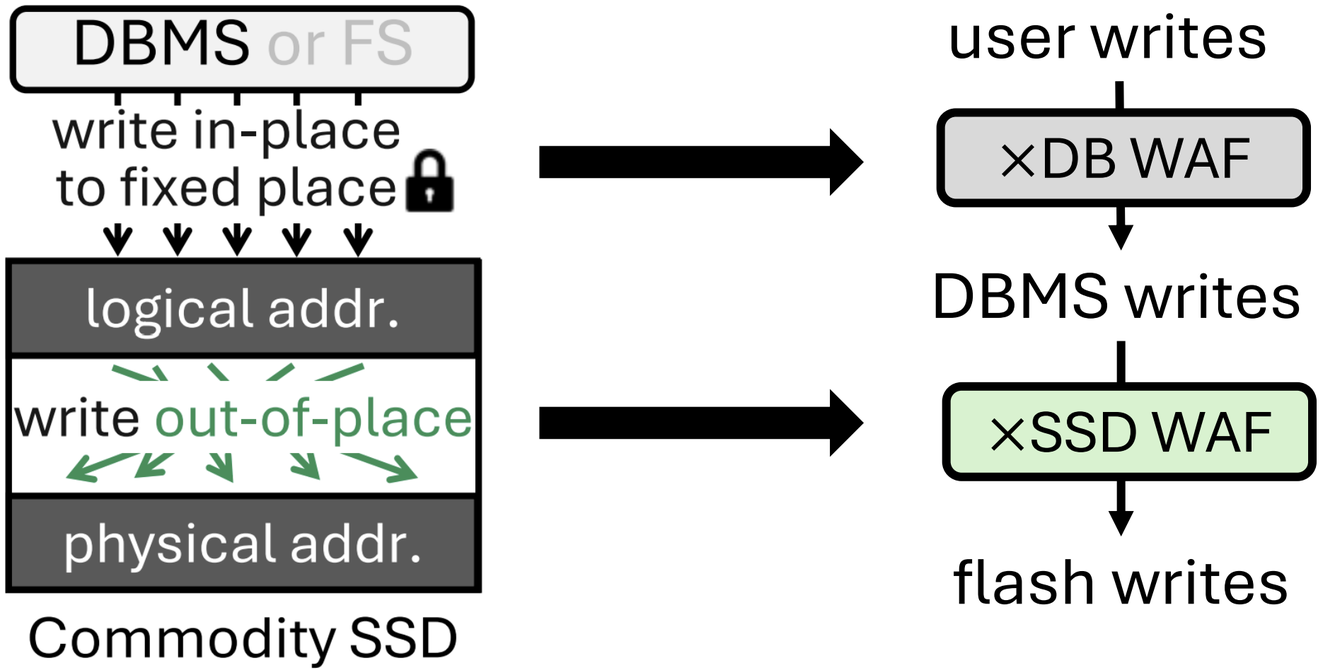

DBMS or FS
write in-place
to fixed place
logical addr.
write out-of-place
physical addr.
Commodity SSD
user writes
×DB WAF
DBMS writes
×SSD WAF
flash writes

Total WAF



total WAF* = *DB WAF* × *SSD WAF

Total WAF

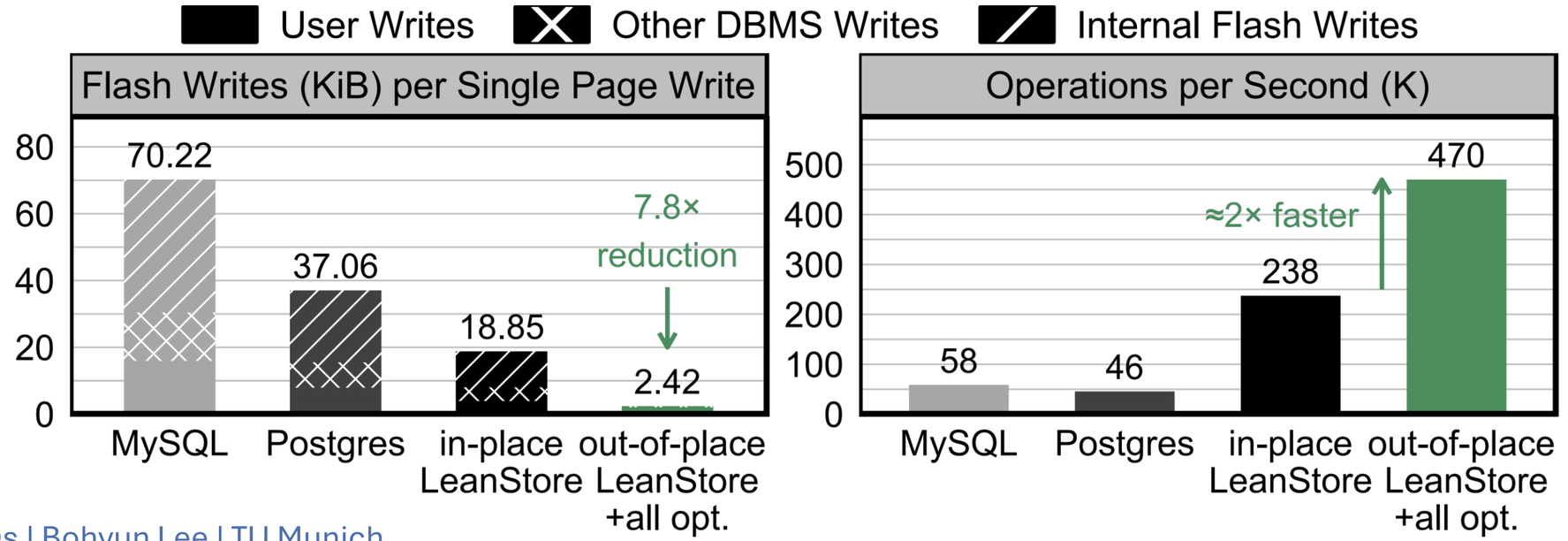


total WAF* = *DB WAF* × *SSD WAF

Database system should address it

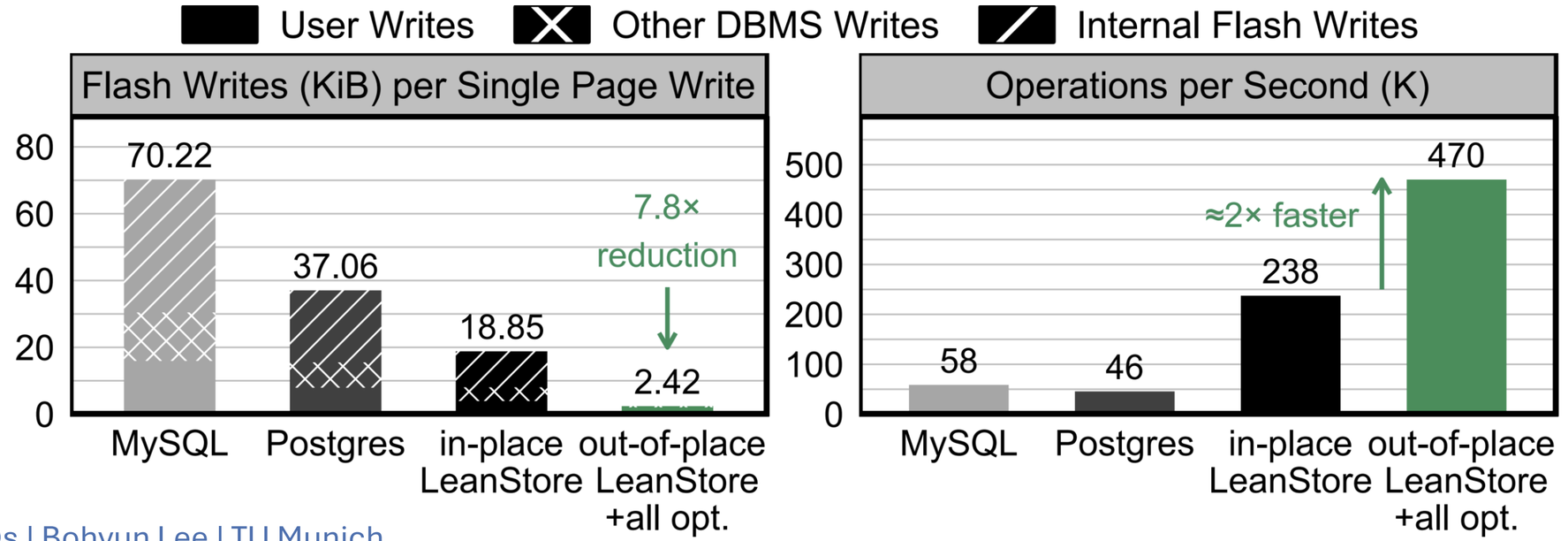
Addressing total WAF

- Can achieve 7.8x flash write reduction and 2x throughput



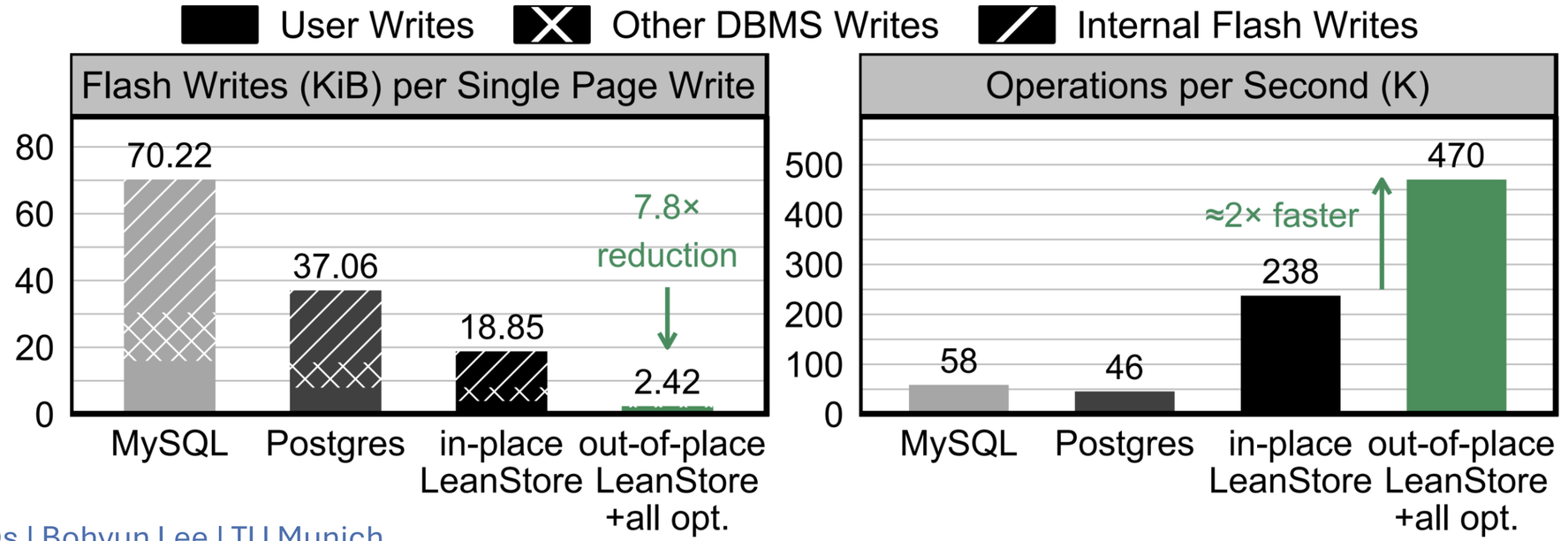
Addressing total WAF

- Can achieve 7.8x flash write reduction and 2x throughput
- First, switch to **out-of-place writes**



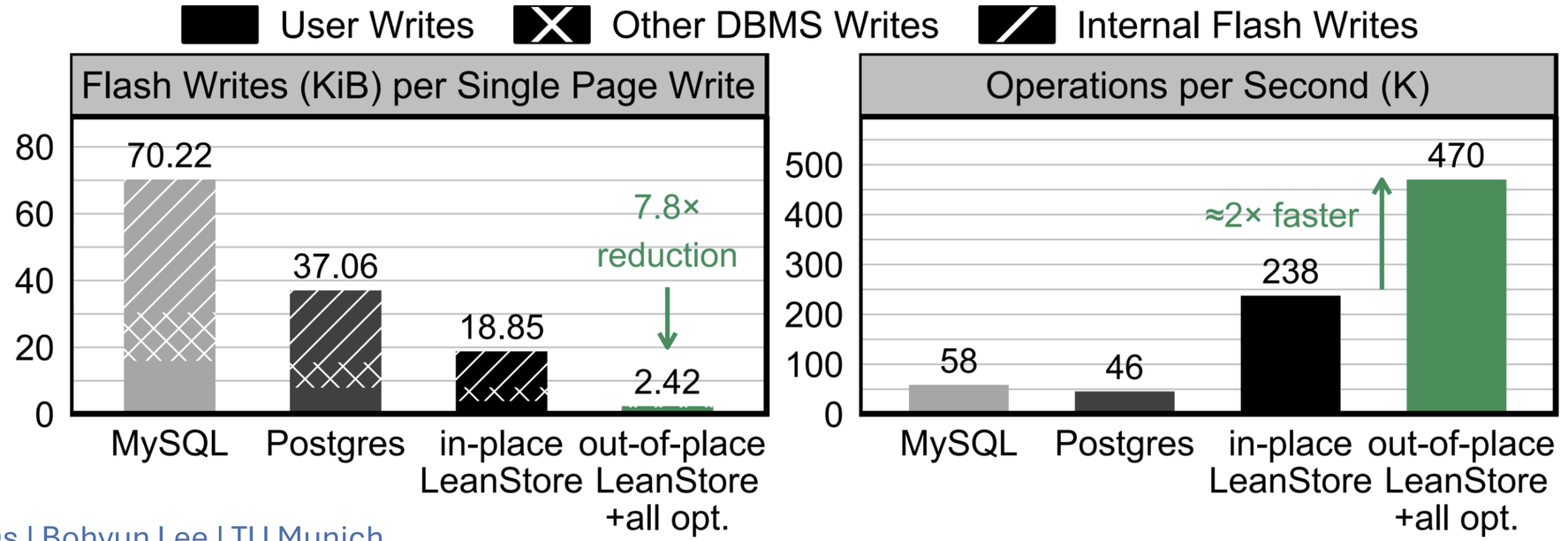
Addressing total WAF

- Can achieve 7.8x flash write reduction and 2x throughput
- First, switch to out-of-place writes
- **DB WAF optimizations:** compression, page packing, GDT



Addressing total WAF

- Can achieve 7.8x flash write reduction and 2x throughput
- First, switch to out-of-place writes
- DB WAF optimizations: compression, page packing, GDT
- **SSD WAF optimizations:** Align DB SSD GC unit size, ZNS, FDP, and NoWA pattern



Out-of-place write-based optimizations

switch from in-place to *out-of-place write*

↓

\+ compressing 4KiB pages

↓

\+ page packing compressed pages

↓

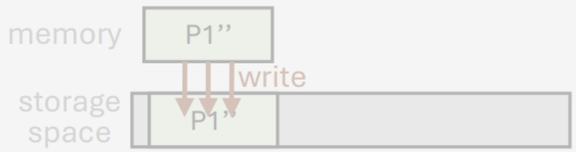
\+ deathtime-based placement & GC

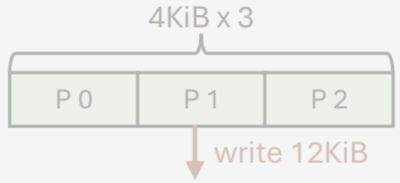
↓

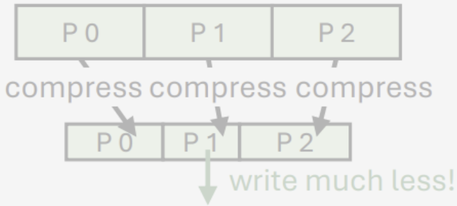
\+ aligning DB SSD GC unit size

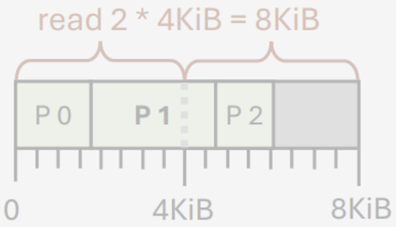
↓ *SSD WAF = 1 !*

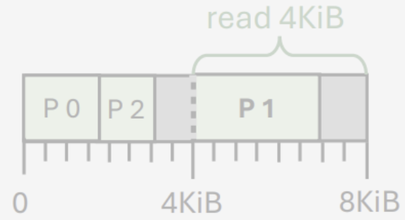
which SSD? → standard SSD | ZNS SSD | FDP SSD

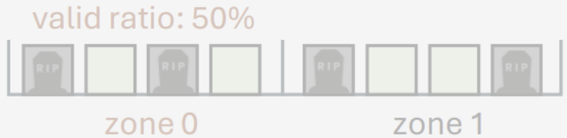


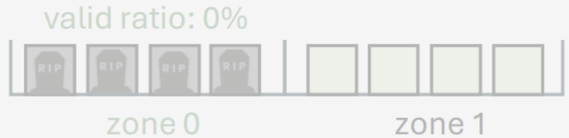


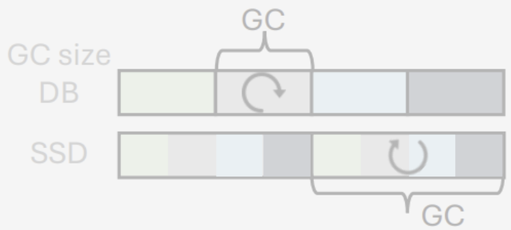


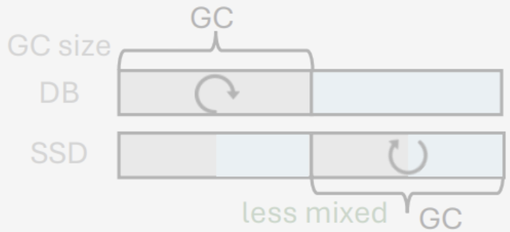












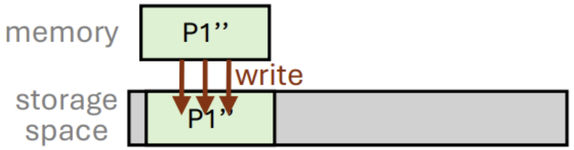
→ \+ NoWA pattern + zone cmds + use placement hints

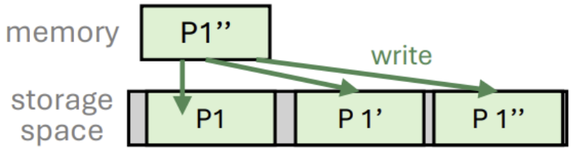
To be on the same page

- DBMS is running an out-of-memory OLTP workload, I/O bound
- Writes happen directly to an SSD (no filesystem)
- Eviction or checkpoint writes have been triggered (WAL is located elsewhere)
- Dirty page writes are issued in a batch (e.g., 64)

Out-of-place write-based optimizations

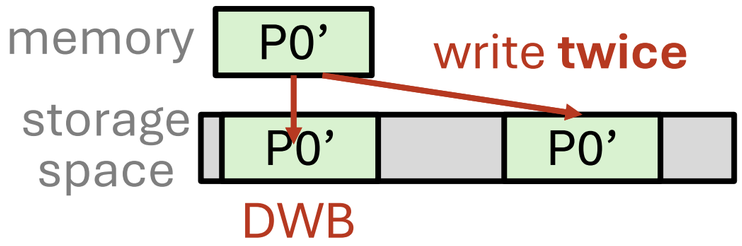
switch from in-place to *out-of-place write*



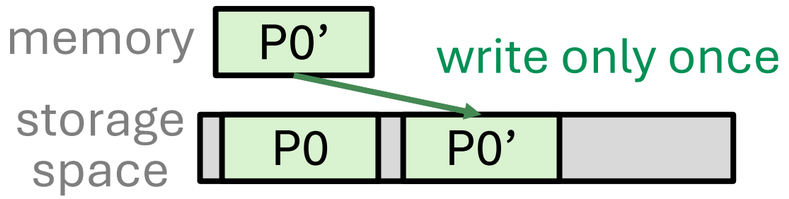


Out-of-place write is necessary because

- Out-of-place writes minimize **doublewrite buffering writes (DWB)** to protect from torn page writes
  - out-of-place writes can nearly **halve** the logical write



in-place write



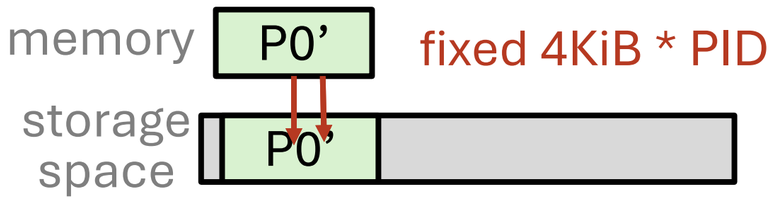
out-of-place write

Out-of-place write is necessary because

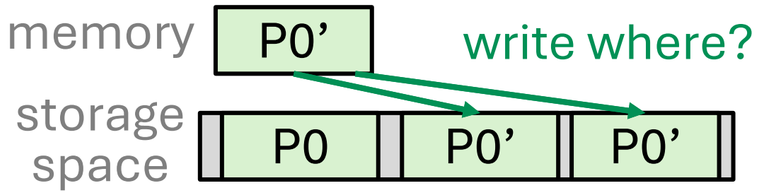
- Out-of-place write offers ***flexibility*** to tailor the write pattern according to system's needs in terms of

Out-of-place write is necessary because

- Out-of-place write offers ***flexibility*** to tailor the write pattern according to system's needs in terms of
  - **placement** (place P0 LBA [8-15] or…?)



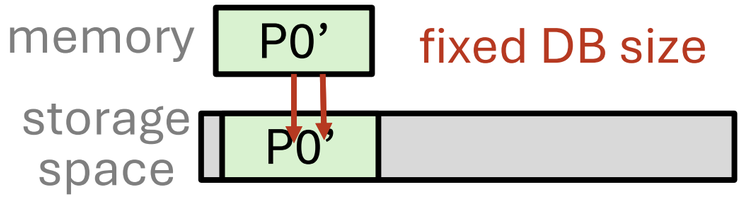
in-place write



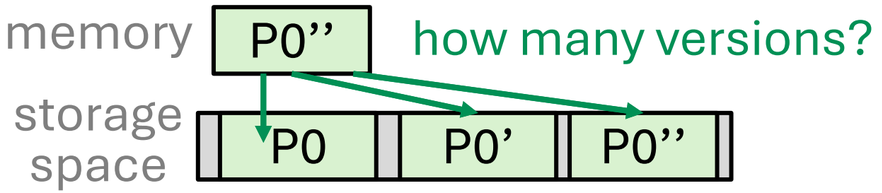
out-of-place write

Out-of-place write is necessary because

- Out-of-place write offers ***flexibility*** to tailor the write pattern according to system's needs in terms of
  - placement (place P0 LBA [8-15] or…?)
  - **storage size** (how big can the database grow?)

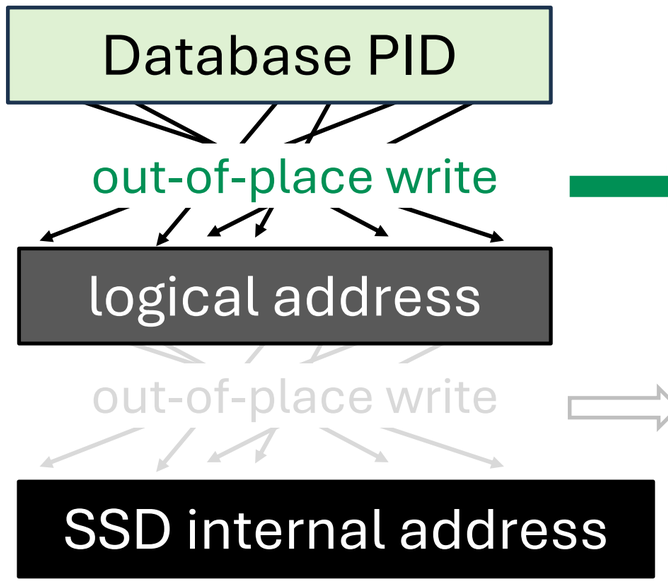


in-place write



out-of-place write

Reducing logical writes (DB WAF)



$$\text{DB WAF} = \frac{\text{DB-issued writes}}{\text{user (eviction + cp) writes}}$$

$$\text{SSD WAF} = \frac{\text{SSD physical writes}}{\text{DB-issued writes}}$$

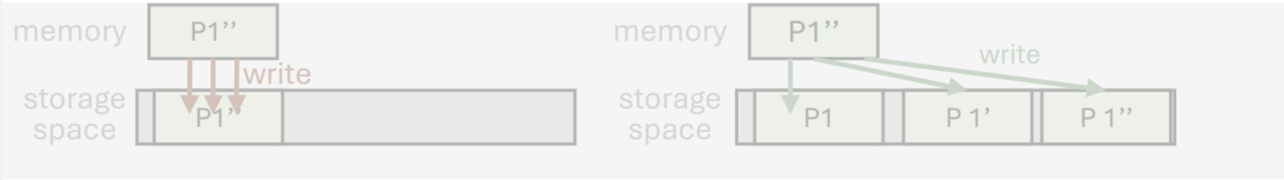
Out-of-place write-based optimizations

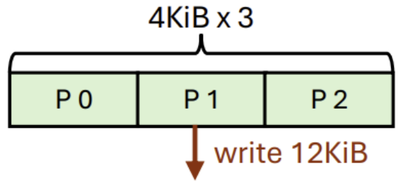
switch from in-place
to *out-of-place write*

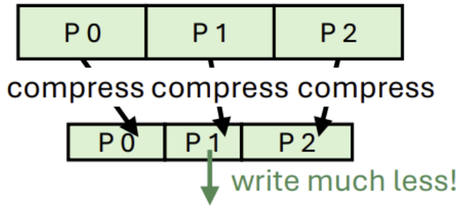
↓

\+ compressing
4KiB pages

↓

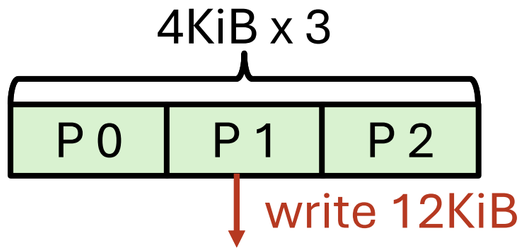


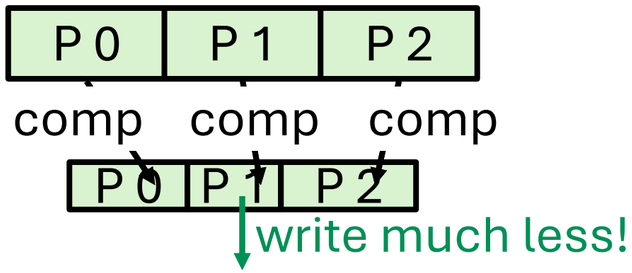




Optimization: page-level compression

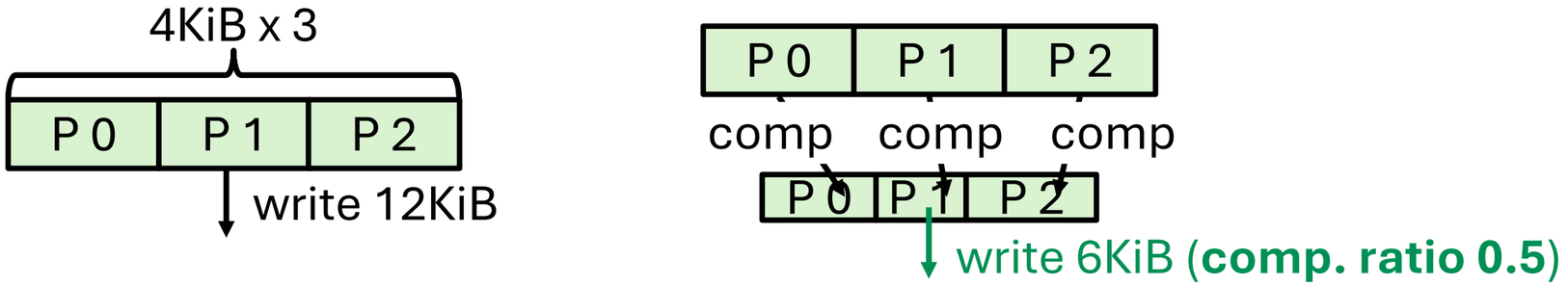
- Compression **reduces write and space**
- Out-of-place writes make compression so much **easier**





Optimization: page-level compression

- Compression reduces write and space
- Out-of-place writes make compression so much easier



- This reduction depends on the **compression ratio**
- But real-world datasets are oftentimes compressible (e.g., 0.15-0.5)

Compression ratio of various datasets

Comp. Ratio (%)	TPC-C	YCSB	Email	URL	Wikipedia
LZ4 [23]	49.0	41.2	40.1	25.0	31.3
ZSTD [24]	36.5	34.4	27.4	14.5	17.8
zlib [40]	38.1	34.3	25.5	15.5	19.0

Out-of-place write-based optimizations

switch from in-place to *out-of-place write*

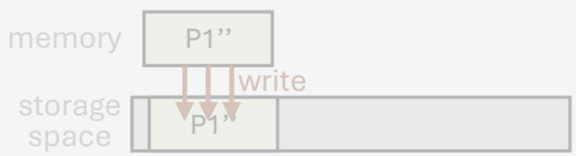
↓

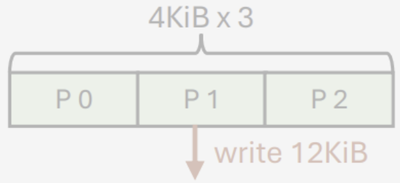
\+ compressing 4KiB pages

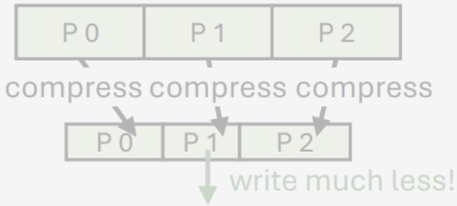
↓

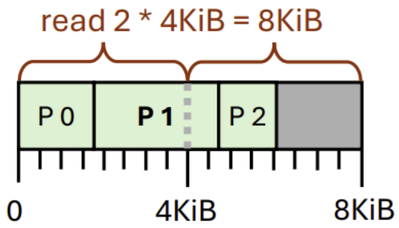
\+ page packing compressed pages

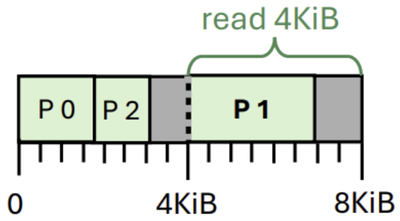
↓





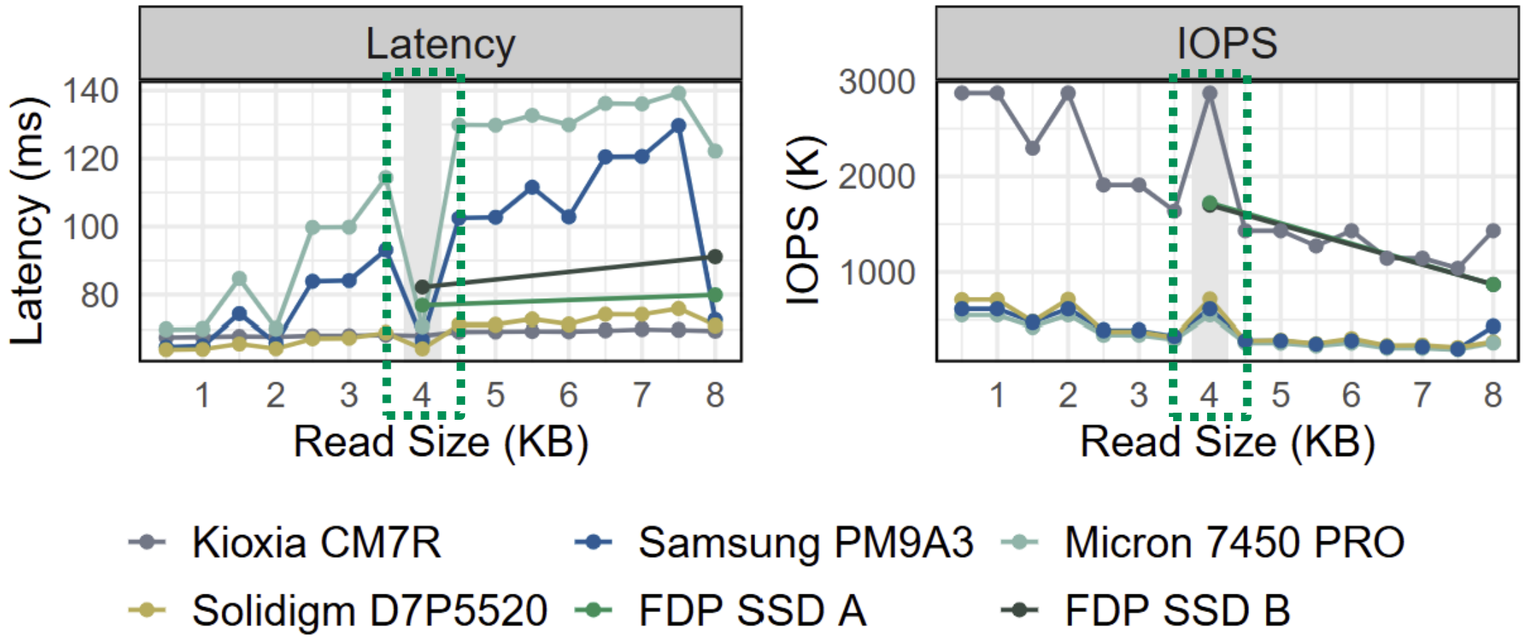






Choosing 4KiB as a page size

4KiB gives the best **read latency** and highest **read throughput**



Hard to place a single <4KiB compressed page

- Now we have compressed pages < 4KiB

Hard to place a single <4KiB compressed page

- Now we have compressed pages < 4KiB

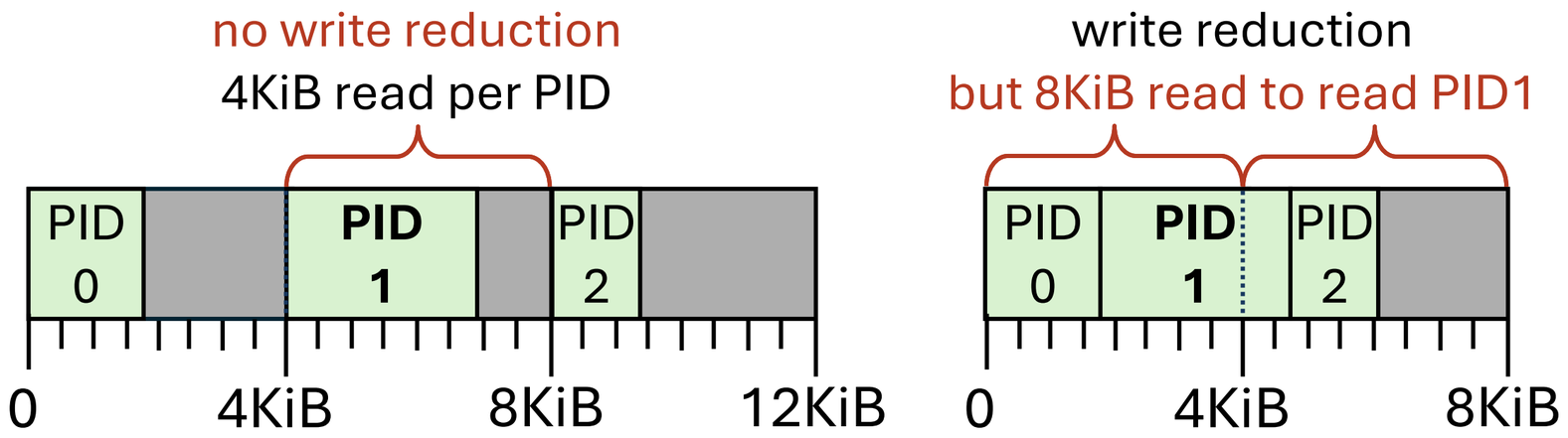
- Block devices only allow block (512b, 4KiB) granular/aligned I/O
- Filesystems: 4KiB same story

Hard to place a single <4KiB compressed page

- Now we have compressed pages < 4KiB

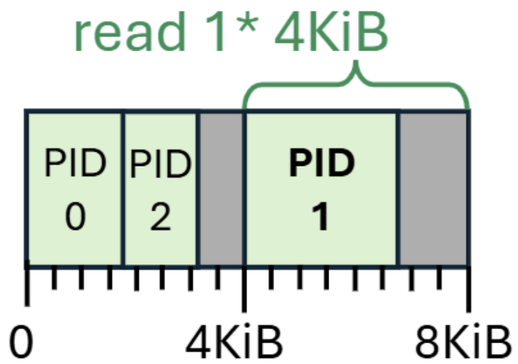
- Block devices only allow block (512b, 4KiB) granular/aligned I/O
- Filesystems: 4KiB same story

- Placing the compressed pages < 4KiB becomes tricky



Optimization: 4KiB-granular page packing

- Pack PIDs so that it always resides in aligned 4KiB offset
- Best-fit binpacking algorithm is used
- Guarantees **4KiB read** when reading a compressed page



Out-of-place write-based optimizations

switch from in-place to *out-of-place write*

↓

\+ compressing 4KiB pages

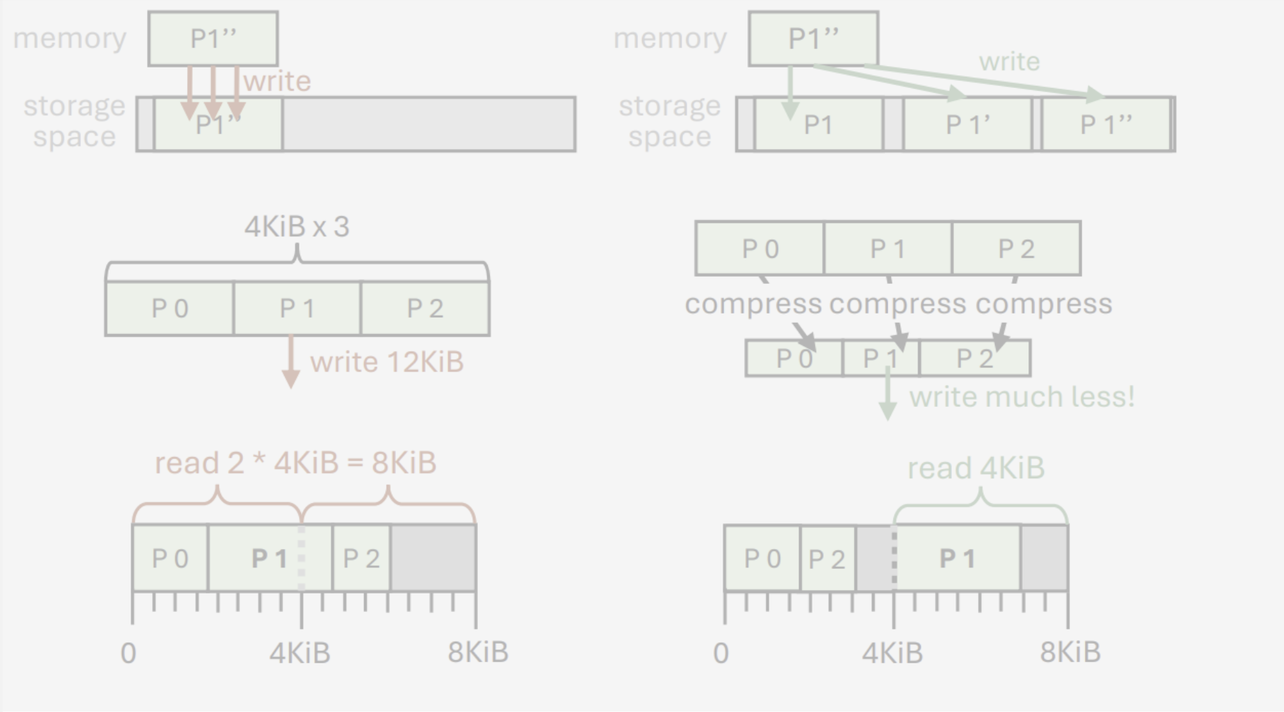
↓

\+ page packing compressed pages

↓

\+ deathtime-based placement & GC

↓



Out-of-place write & space management

- Space management granularity: **zone**
- A single zone contains multiple pages (8 in this example)

Out-of-place write & space management

- Suppose we write out PID 0 0

PID2OffsetTable

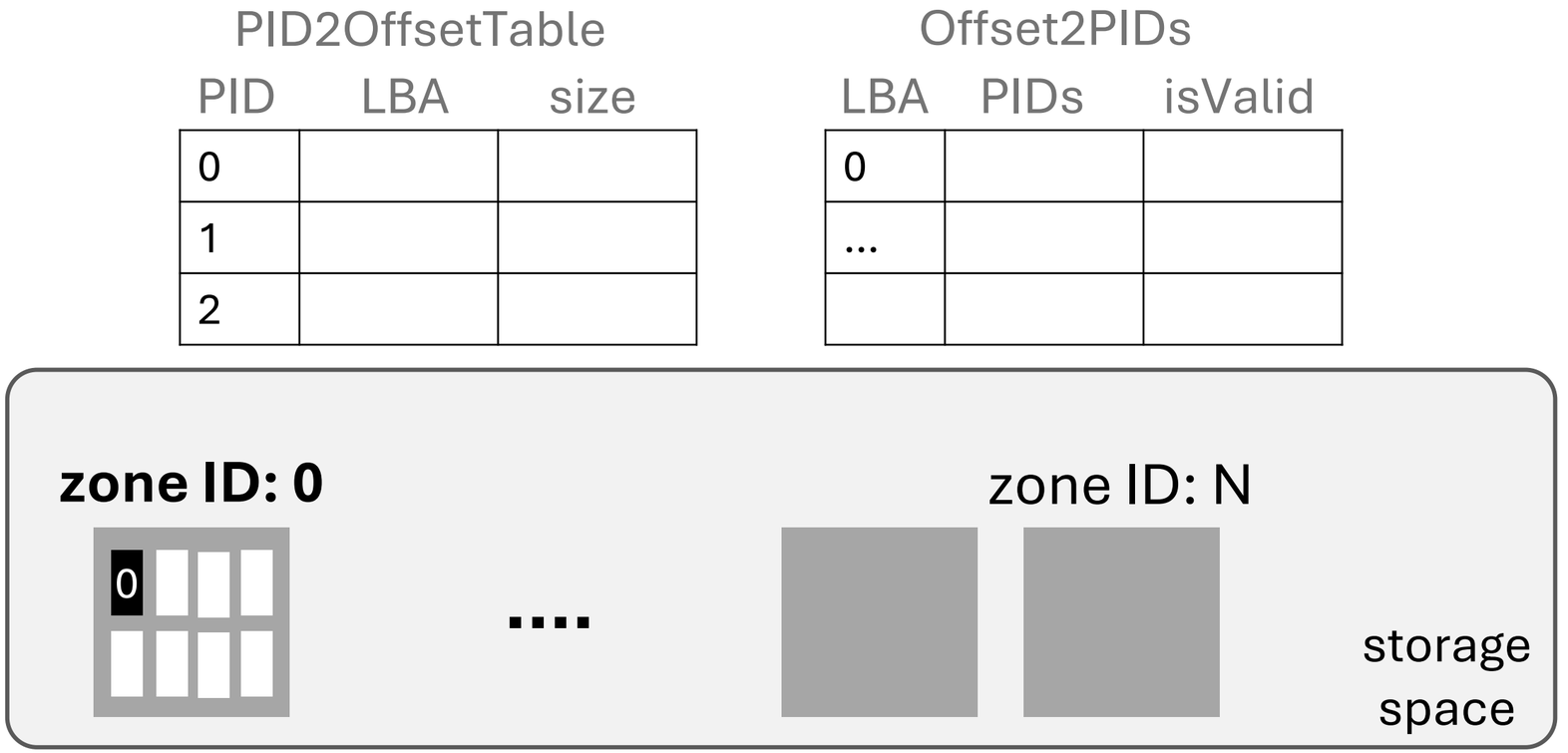
PID	LBA	size
0		
1		
2		

Offset2PIDs

LBA	PIDs	isValid
0		
…		

Out-of-place write & space management

- Place PID 0 in the first slot of zone 0



Out-of-place write & space management

- Update space metadata accordingly

PID2OffsetTable

PID	LBA	size
0	**0**	**1800**
1		
2		

Offset2PIDs

LBA	PIDs	isValid
0	**0**	**TRUE**
…		

Out-of-place accumulates stale pages

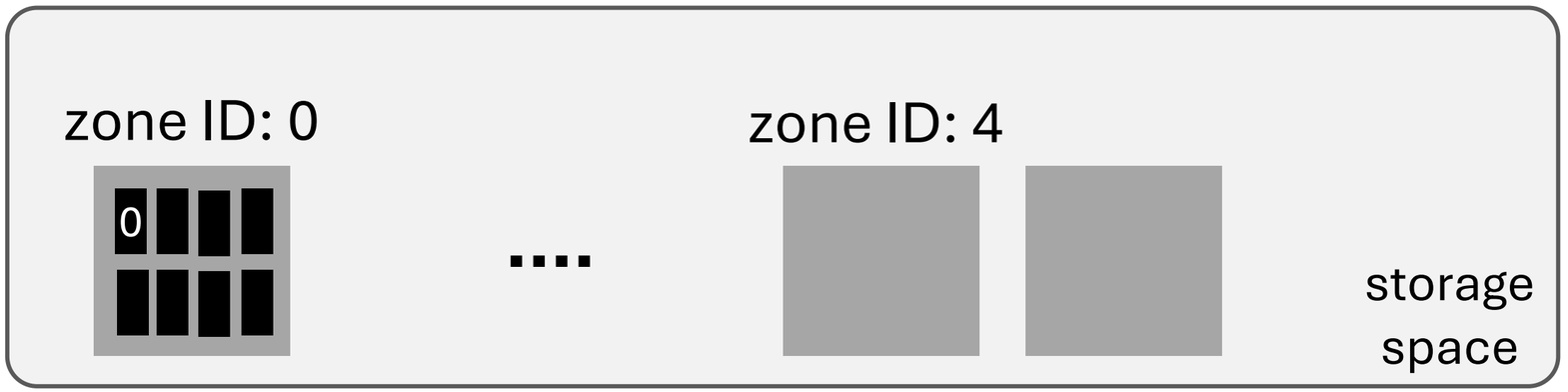
- Later on, PID 0 0 is flushed to storage again

PID2OffsetTable

PID	LBA	size
0	0	1800
1		
2		

Offset2PIDs

LBA	PIDs	isValid
0	0	TRUE
...		



Out-of-place accumulates stale pages

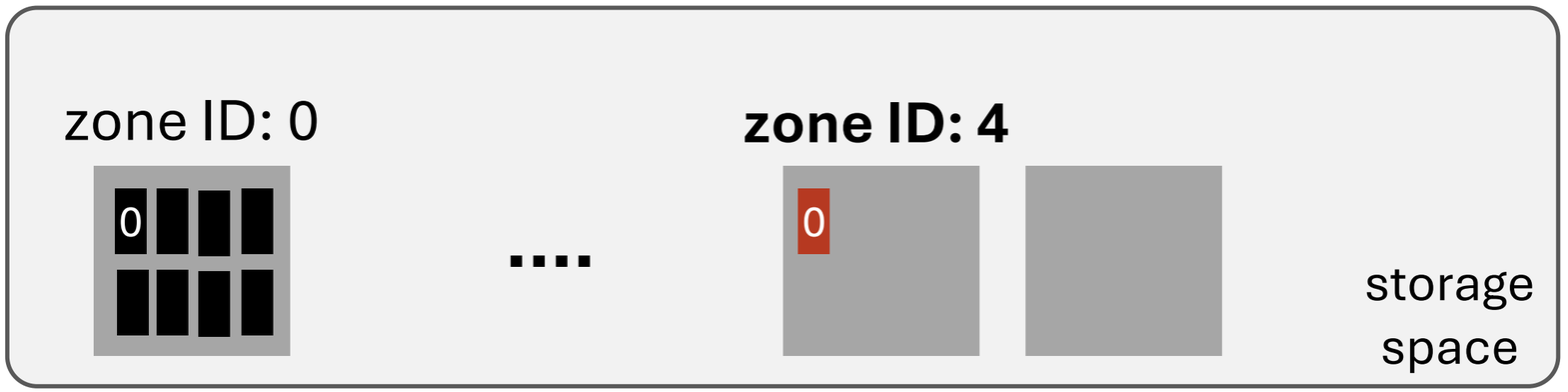
- Place updated PID 0 inside the first slot of zone 4

PID2OffsetTable

PID	LBA	size
0	0	1800
1		
2		

Offset2PIDs

LBA	PIDs	isValid
0	0	TRUE
...		



Out-of-place accumulates stale pages

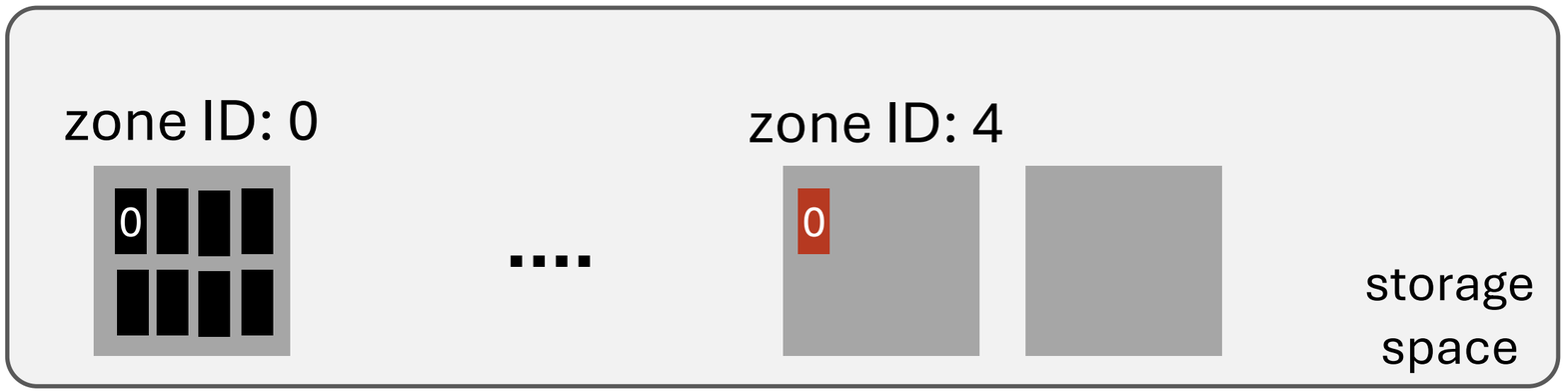
- Update space metadata

PID2OffsetTable

PID	LBA	size
0	1GB	1680
1		
2		

Offset2PIDs

LBA	PIDs	isValid
0	0	FALSE
...		
1GB	0	TRUE



Out-of-place accumulates stale pages

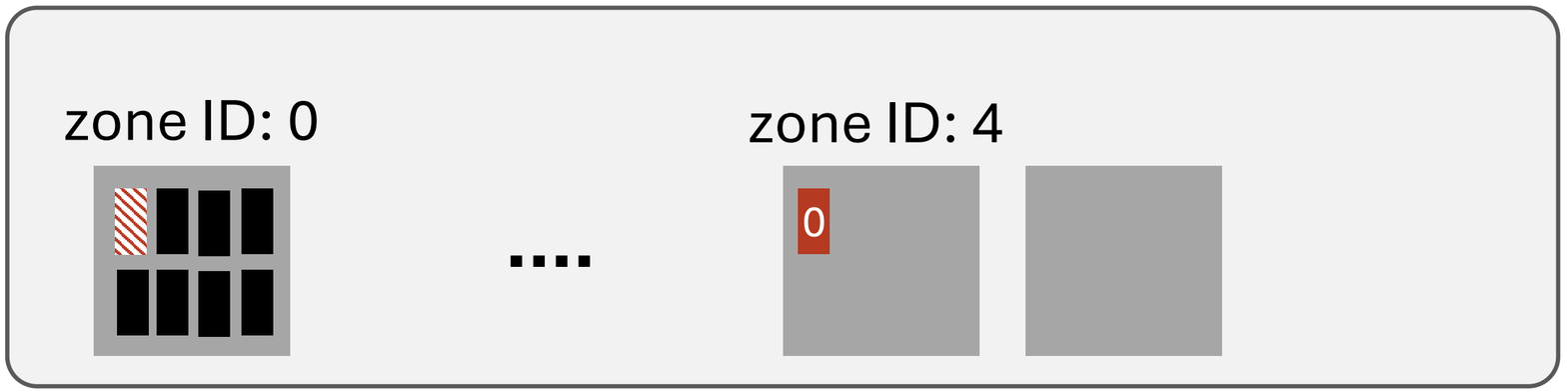
- Previous version becomes stale

PID2OffsetTable

PID	LBA	size
0	1GB	1680
1		
2		

Offset2PIDs

LBA	PIDs	isValid
0	0	FALSE
...		
1GB	0	TRUE

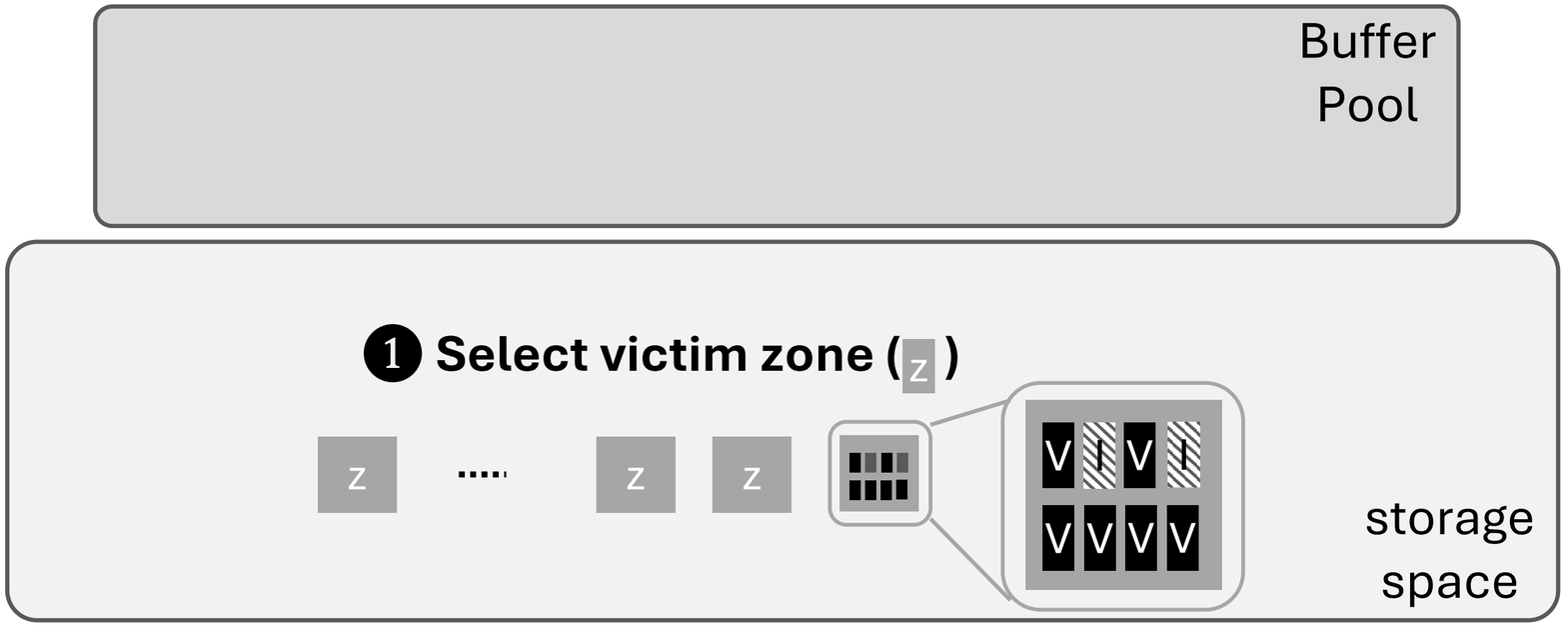


Relation between valid ratio and OP

- Invalidated pages take up storage space
- As used storage space grows, pages have more chance to get invalidated
- **overprovisioned space ↑, average valid ratio per zone ↓**

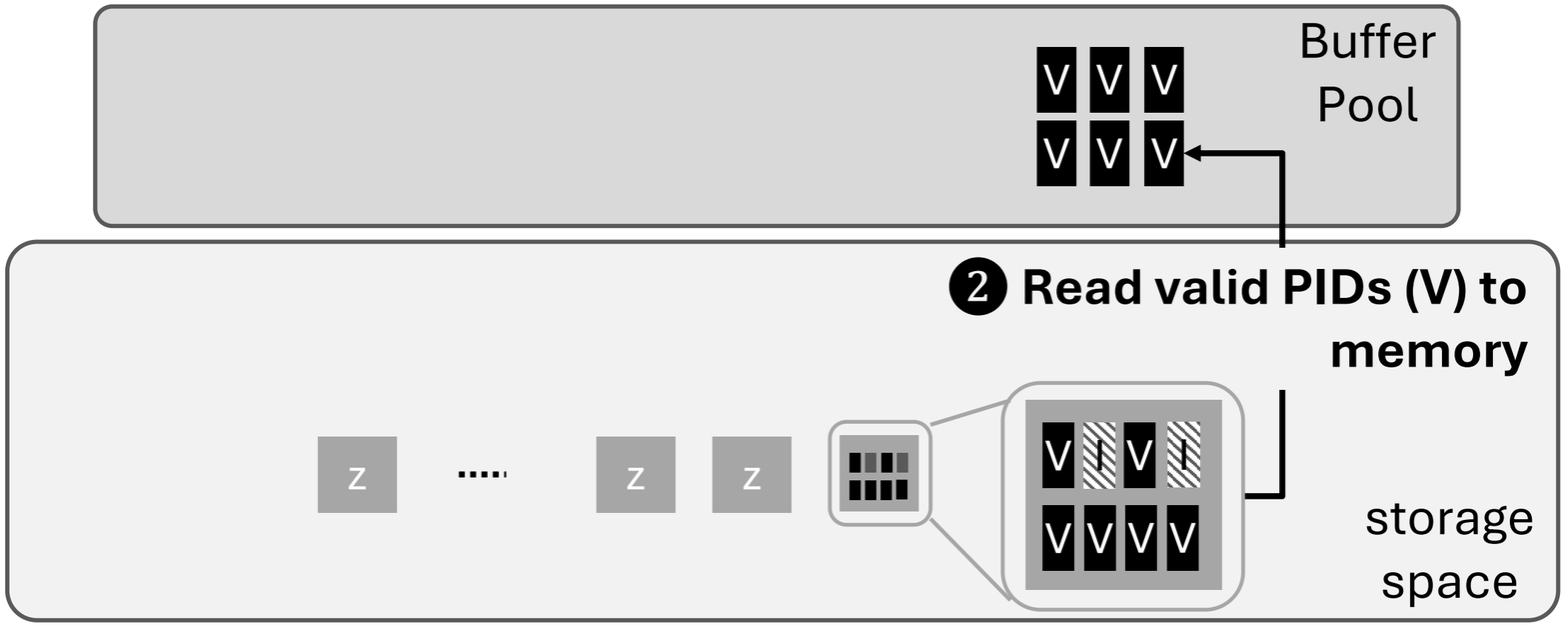
Garbage Collection

- GC reclaims space occupied by invalidated pages
- GC granularity = zone
- Use **greedy** algorithm for victim selection policy



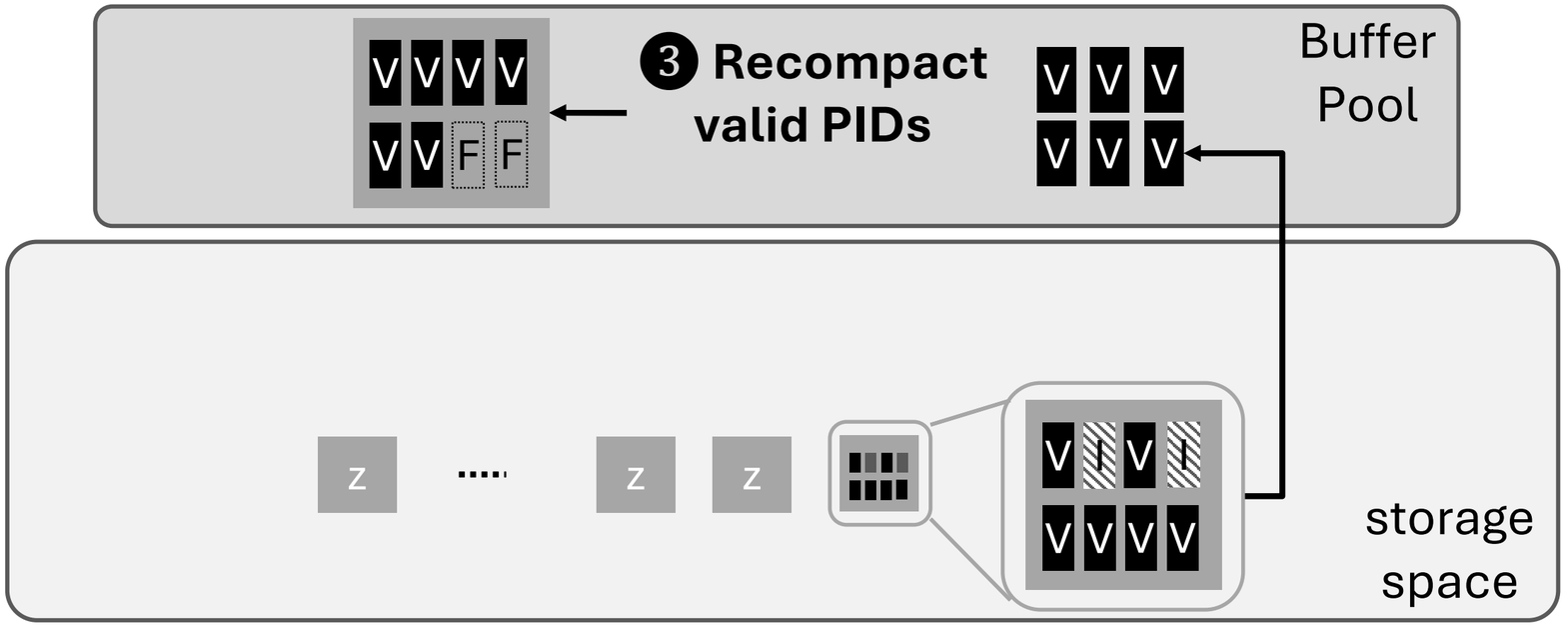
Garbage Collection

- GC reclaims space occupied by invalidated pages



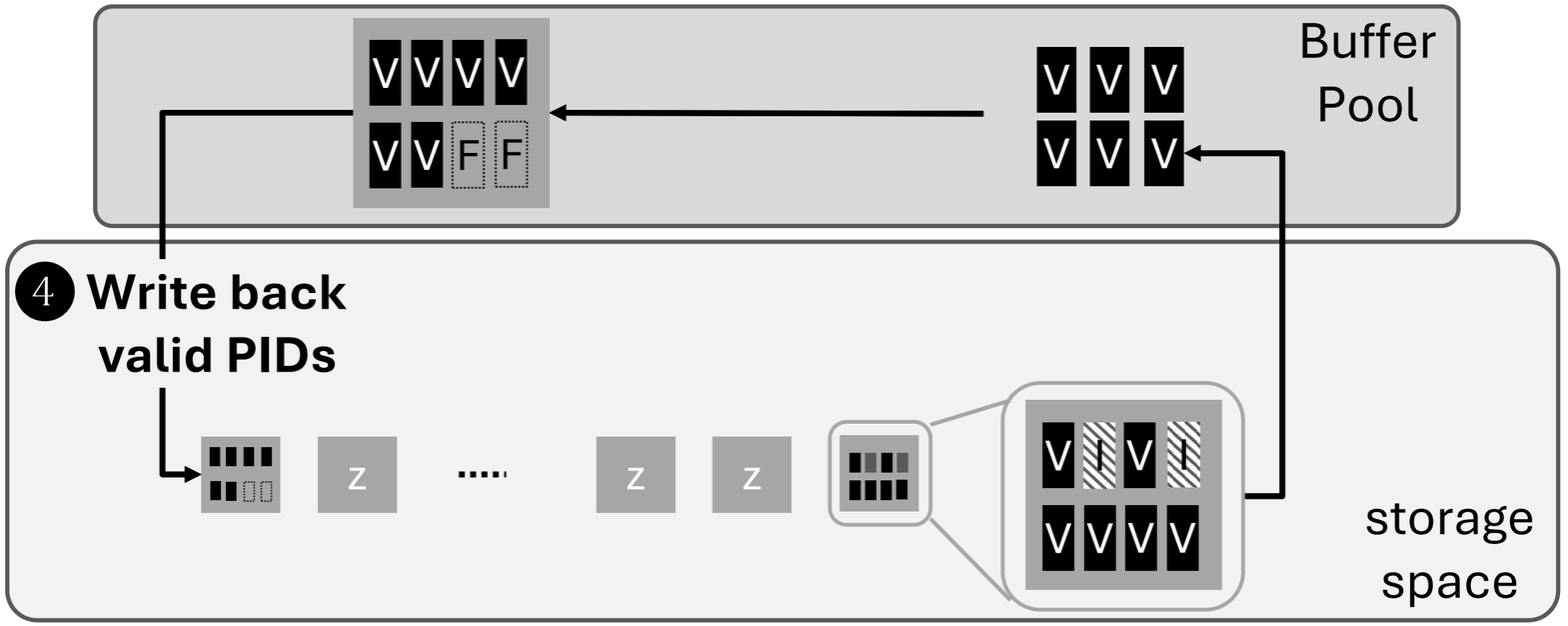
Garbage Collection

- GC reclaims space occupied by invalidated pages



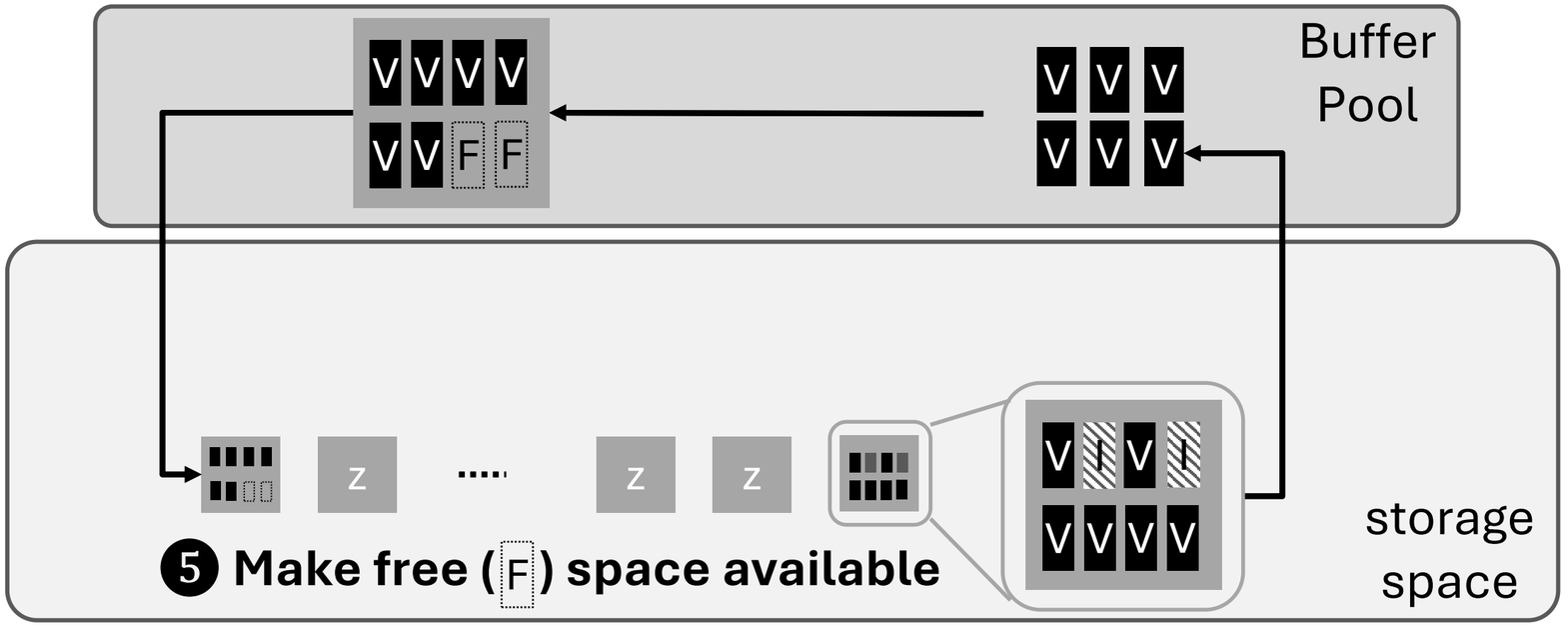
Garbage Collection

- GC reclaims space occupied by invalidated pages



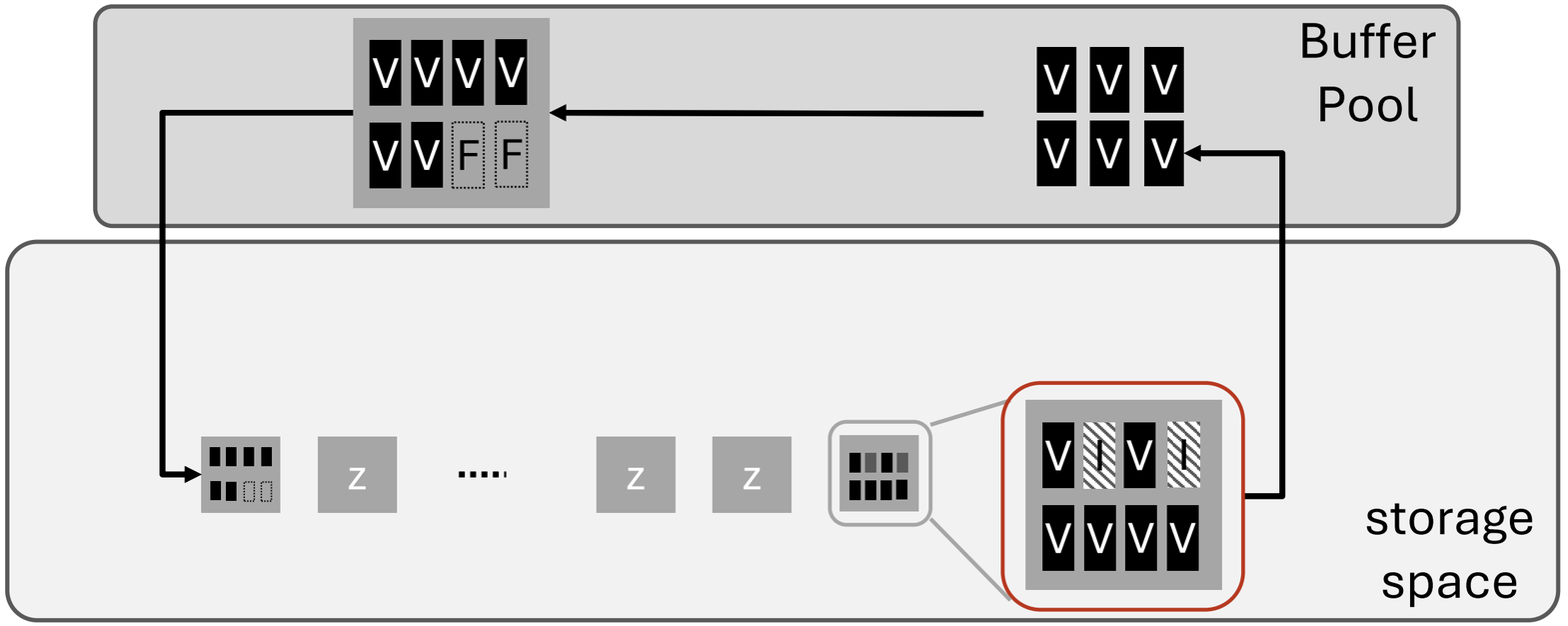
Garbage Collection

- GC reclaims space occupied by invalidated pages



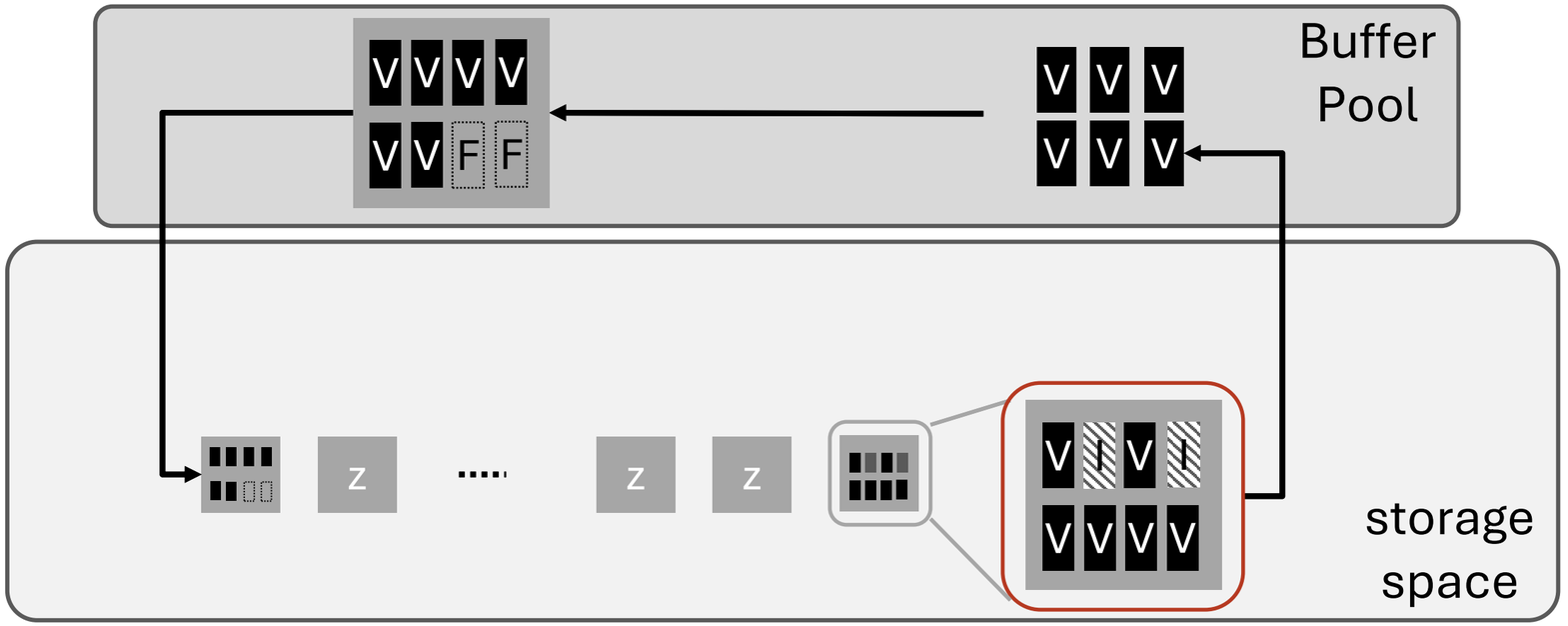
WAF and valid ratio

- Valid ratio: 0.75
- GC writes 6 more pages to reclaim 2 space



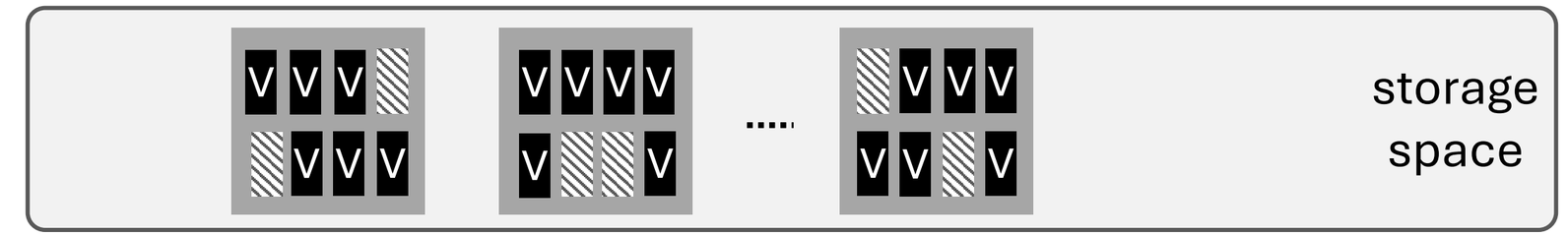
WAF and valid ratio

- Valid Ratio (VR): 0.75
- GC writes 6 more pages to reclaim 2 space
- **WAF = 1/(1-VR) = 4**



WAF varies depending on skewness

- Invalidation occurs **uniformly** under **uniform random access pattern**
- So **greedy** works well

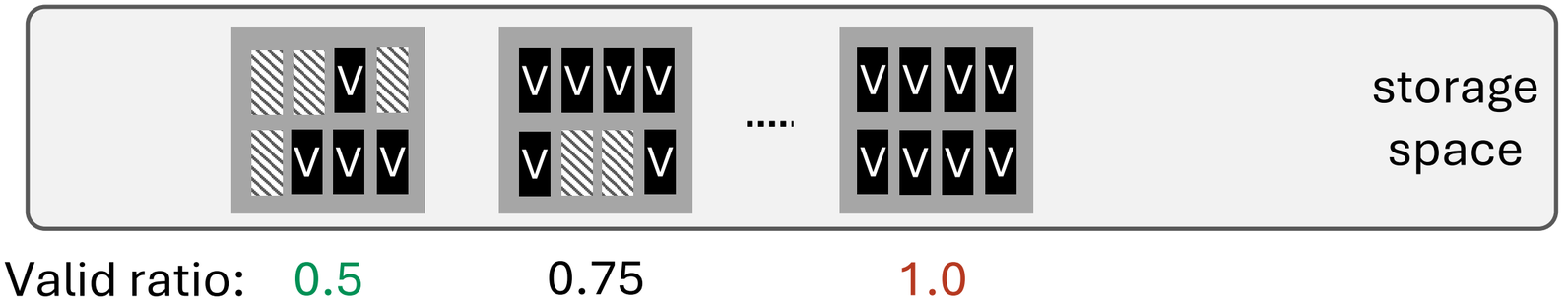


Valid ratio: 0.75 0.75 0.75

WAF =4

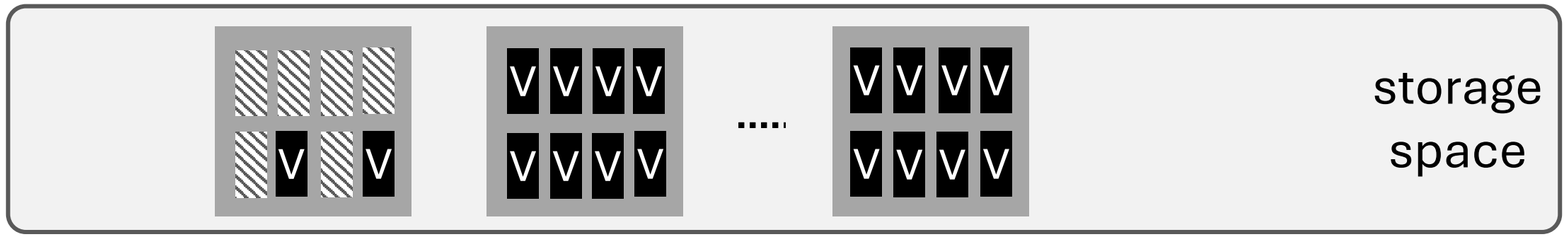
WAF varies depending on skewness

- But under **skewed** access, some PIDs are hot, or temporal/spatial locality
- Greedy can make WAF worse in the long run under skew



How to use skewness to mitigate WAF?

- What if we can intentionally place pages that will be **invalidated during a similar timeframe**?

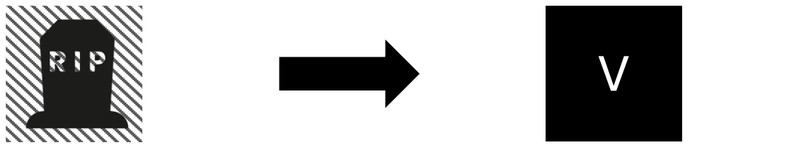


Valid ratio: 0.25 1.0 1.0

WAF = 1.25

Optimization: Grouping By Deathtime (GDT)

- Previous page versions **“die”** once it is written to a new location
- **Deathtime** = LSN when the next page version is flushed to disk

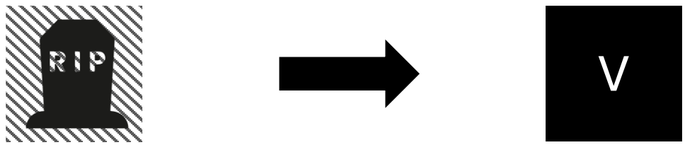


written at LSN 5
died at LSN 10

written at LSN 10

Optimization: Grouping By Deathtime (GDT)

- Previous page versions “die” once it is written to a new location
- Deathtime = LSN when the next page version is flushed to disk



written at LSN 5
died at LSN 10

written at LSN 10

- **How can we use deathtime?**

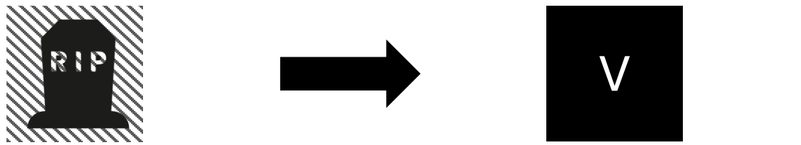
(1) Predict each PID’s next deathtime

(2) Place PIDs that have similar deathtimes into the same zone

(3) Also maintain deathtime-based placement upon GC

Optimization: Grouping By Deathtime (GDT)

- Previous page versions “die” once it is written to a new location
- Deathtime = LSN when the next page version is flushed to disk



written at LSN 5
died at LSN 10

written at LSN 10

- How can we use deathtime?

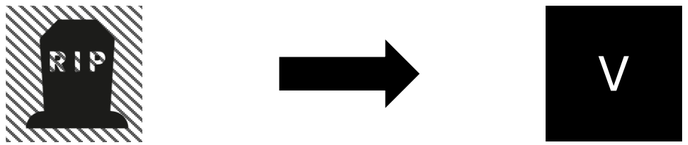
(1) **Predict** each PID’s next deathtime

(2) Place PIDs that have similar deathtimes into the same zone

(3) Also maintain deathtime-based placement upon GC

Optimization: Grouping By Deathtime (GDT)

- Previous page versions “die” once it is written to a new location
- Deathtime = LSN when the next page version is flushed to disk



written at LSN 5
died at LSN 10

written at LSN 10

- How can we use deathtime?

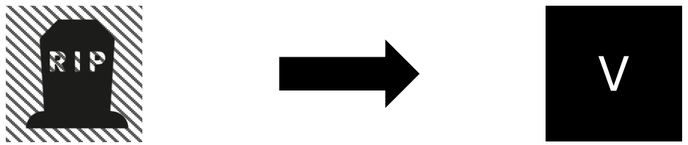
(1) Predict each PID’s next deathtime

(2) **Place** PIDs that have similar deathtimes into the same zone

(3) Also maintain deathtime-based placement upon GC

Optimization: Grouping By Deathtime (GDT)

- Previous page versions “die” once it is written to a new location
- Deathtime = LSN when the next page version is flushed to disk



written at LSN 5
died at LSN 10

written at LSN 10

- How can we use deathtime?

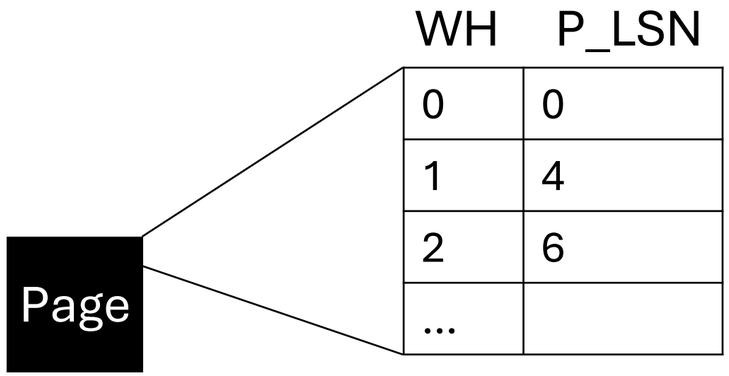
(1) Predict each PID’s next deathtime

(2) Place PIDs that have similar deathtimes into the same zone

(3) Also maintain deathtime-based placement upon **GC**

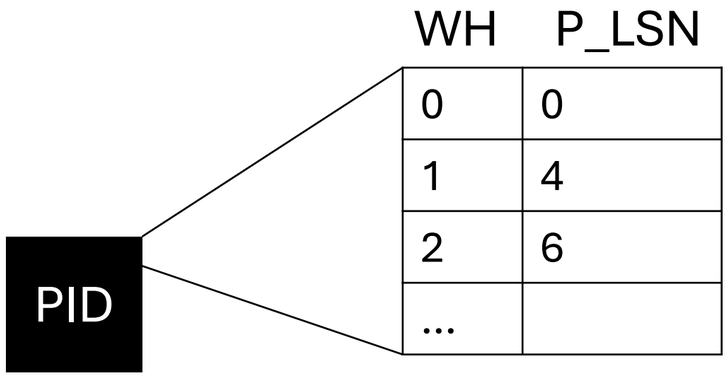
Grouping By Deathtime (1) Deathtime estimation

- How to predict deathtime:
  - Track **write history** (4-8) in the page header (WATT: Vöhringer et.al., VLDB23)



Grouping By Deathtime (1) Deathtime estimation

- How to predict deathtime:
- Track write history (WH) (4-8) in the page header (WATT: Vöhringer et.al., VLDB23)

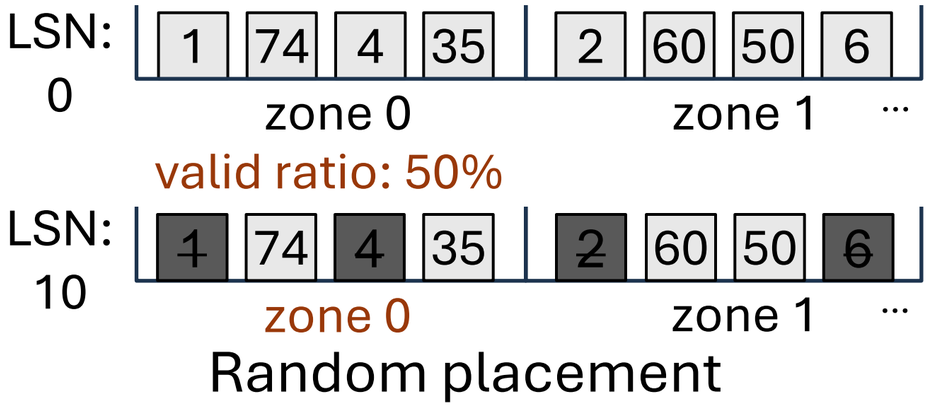


- **Estimate** next invalidation time (deathtime) per page with LSNs

$$\textbf{Estimated Death } \boldsymbol{Time}(\boldsymbol{EDT}) = currentLSN + \frac{WH_n - WH_1}{n-1}$$

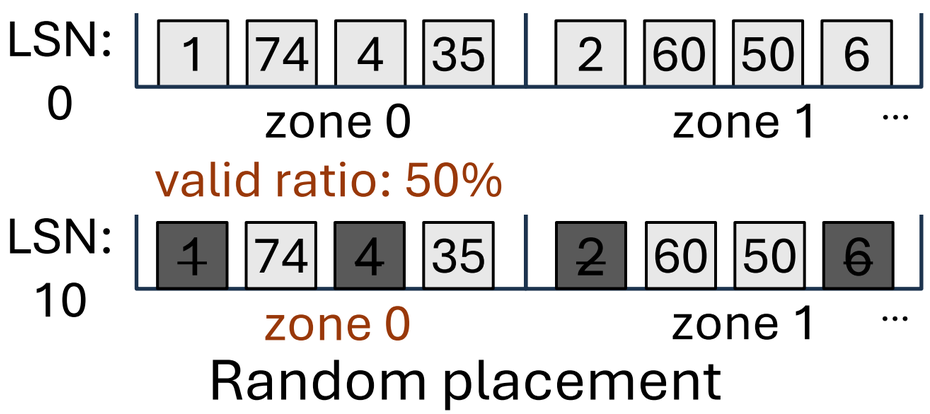
Grouping By Deathtime (2) Placement

- **Place** PIDs with similar EDT to the same zone

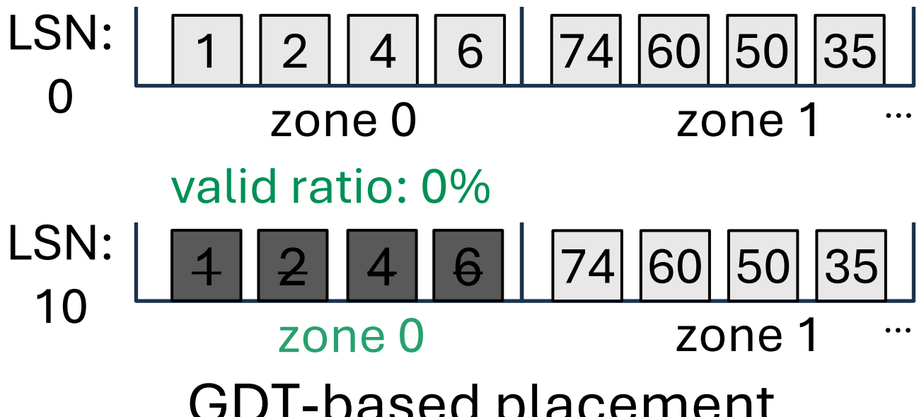


Grouping By Deathtime (2) Placement

- **Place** PIDs with similar EDT to the same zone



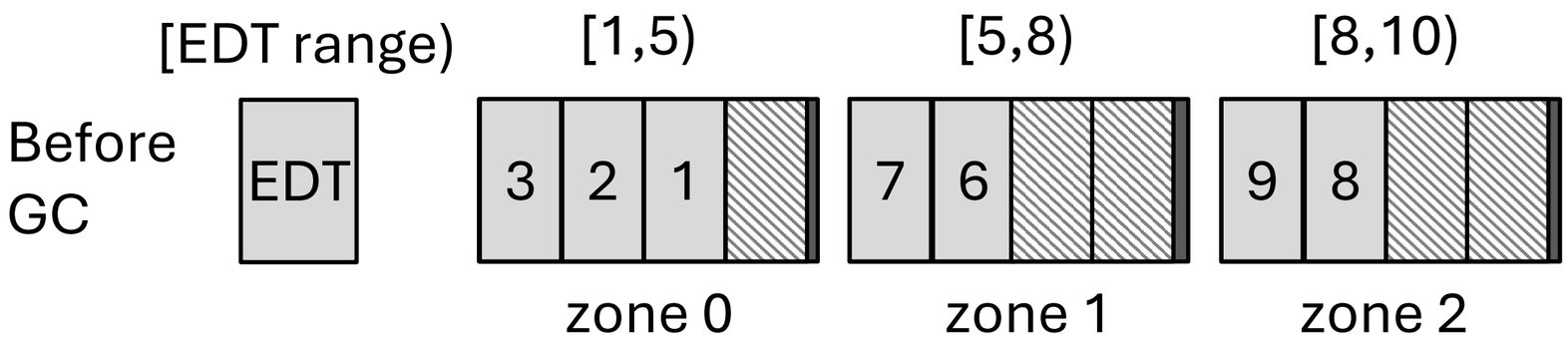
Random placement



GDT-based placement

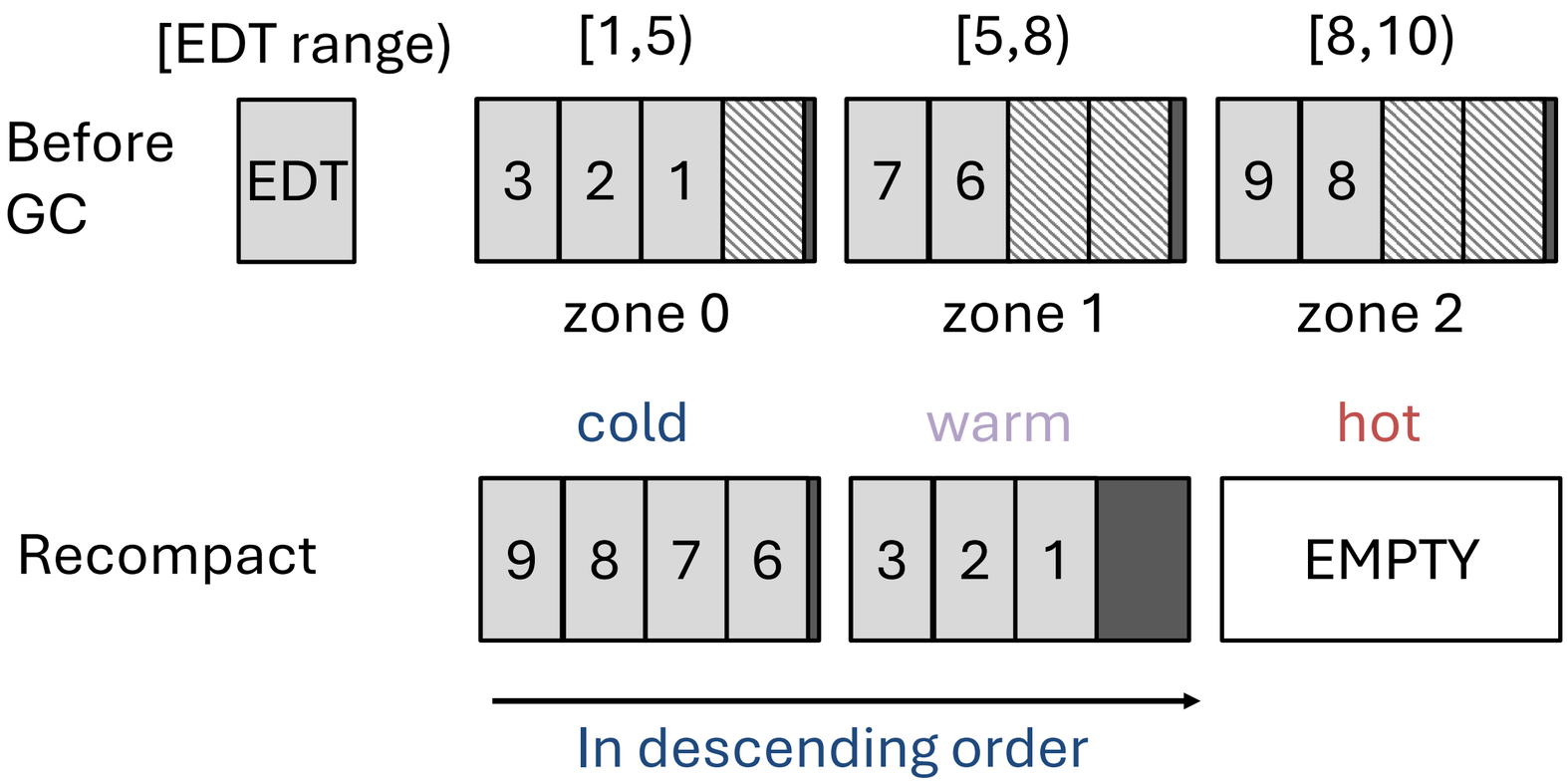
Grouping By Deathtime (3) GC

- Upon GC (GC target: zone 0, 1, 2 to produce an empty zone)



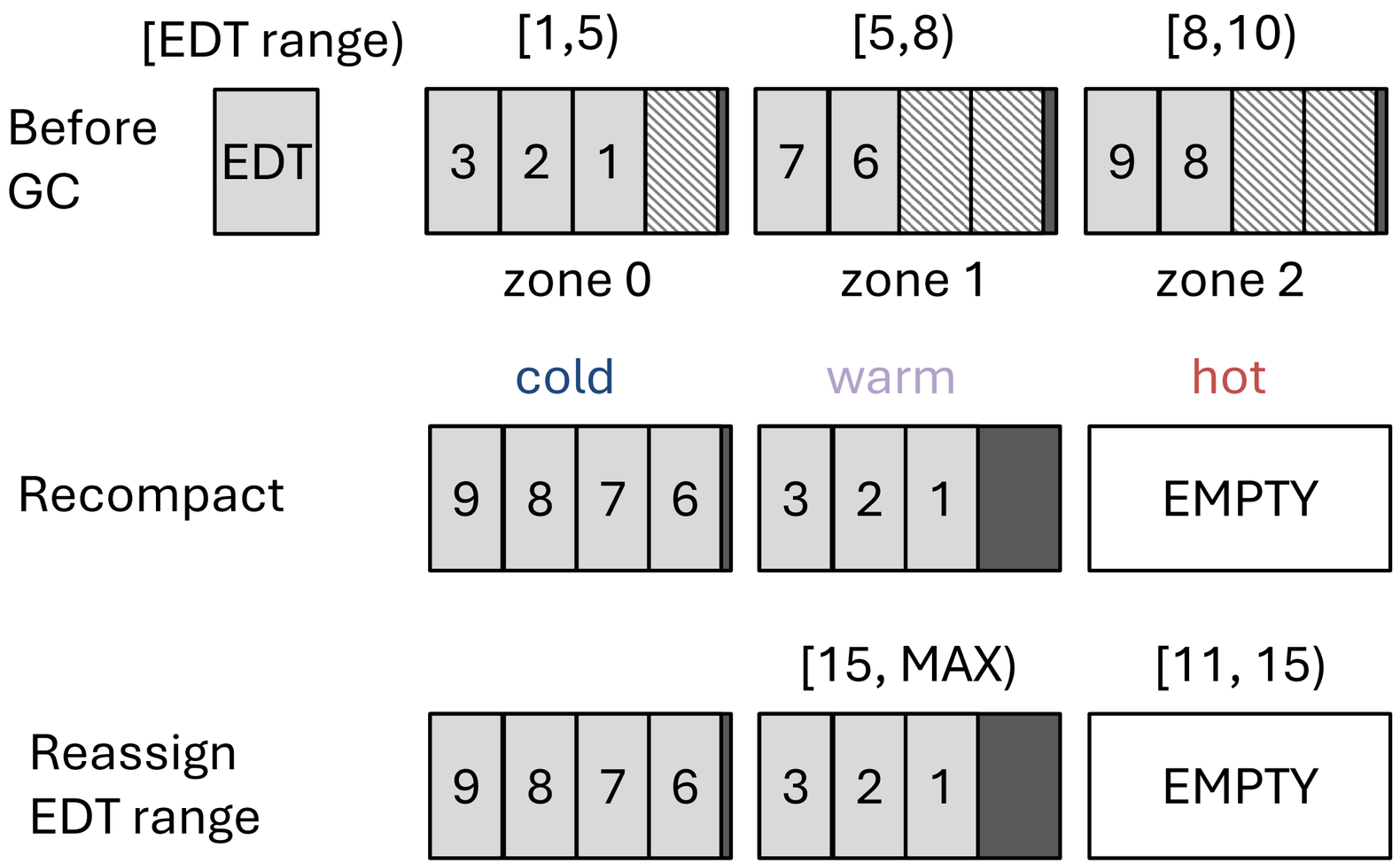
Grouping By Deathtime (3) GC

- Upon GC (GC target: zone 0, 1, 2 to produce an empty zone)



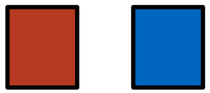
Grouping By Deathtime (3) GC

- Upon GC (GC target: zone 0, 1, 2 to produce an empty zone)



Workload-aware GC is not new of course

- Hundreds of related papers exist in filesystem/SSD community



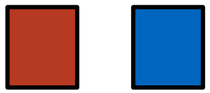
Hot/cold separation

Lifetime-based GC

Deathtime-based GC

But, DBMS can do better

- Hundreds of related papers exist in filesystem/SSD community



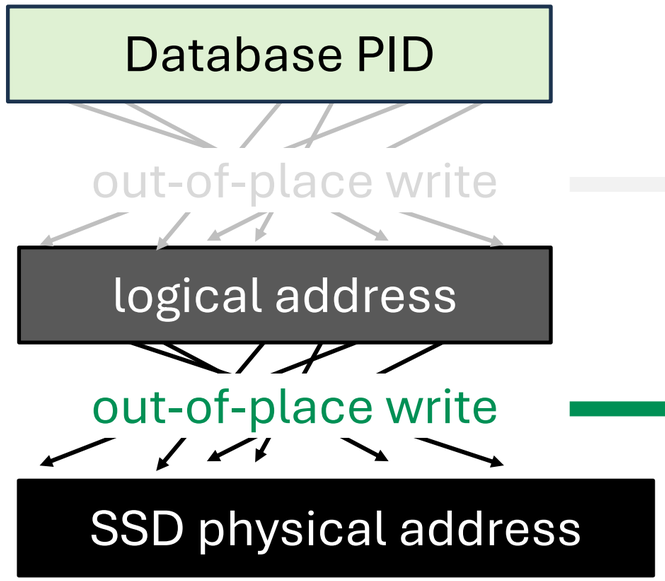
Hot/cold separation

Lifetime-based GC

Deathtime-based GC

- But this is the **first work** to use **DB semantics**
- Database inherently has more workload knowledge
- Therefore, DBMS can do better!

Part 2. Reducing physical writes (SSD WAF)

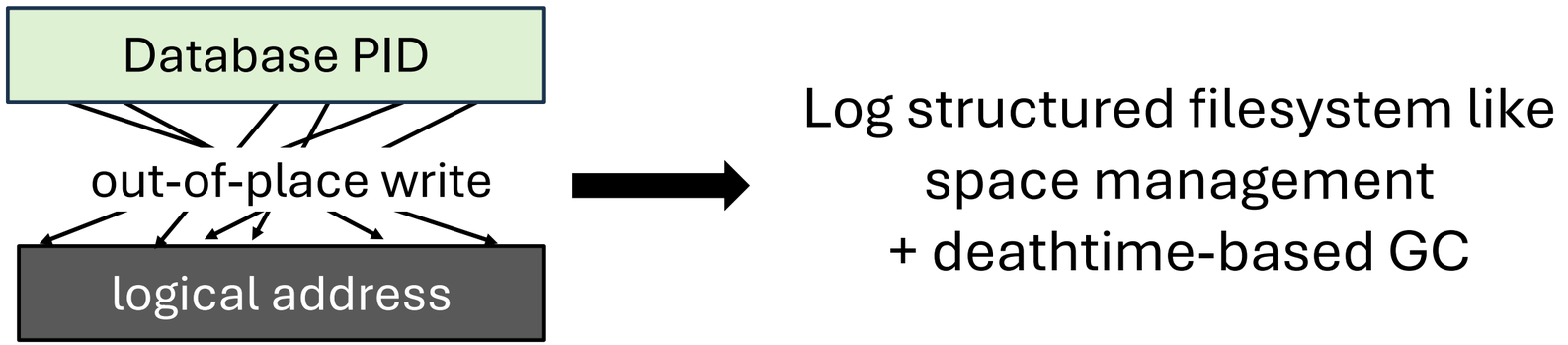


$$\text{DB WAF} = \frac{\text{DB-issued writes}}{\text{user writes}}$$

$$\text{SSD WAF} = \frac{\text{SSD physical writes}}{\text{DB-issued writes}}$$

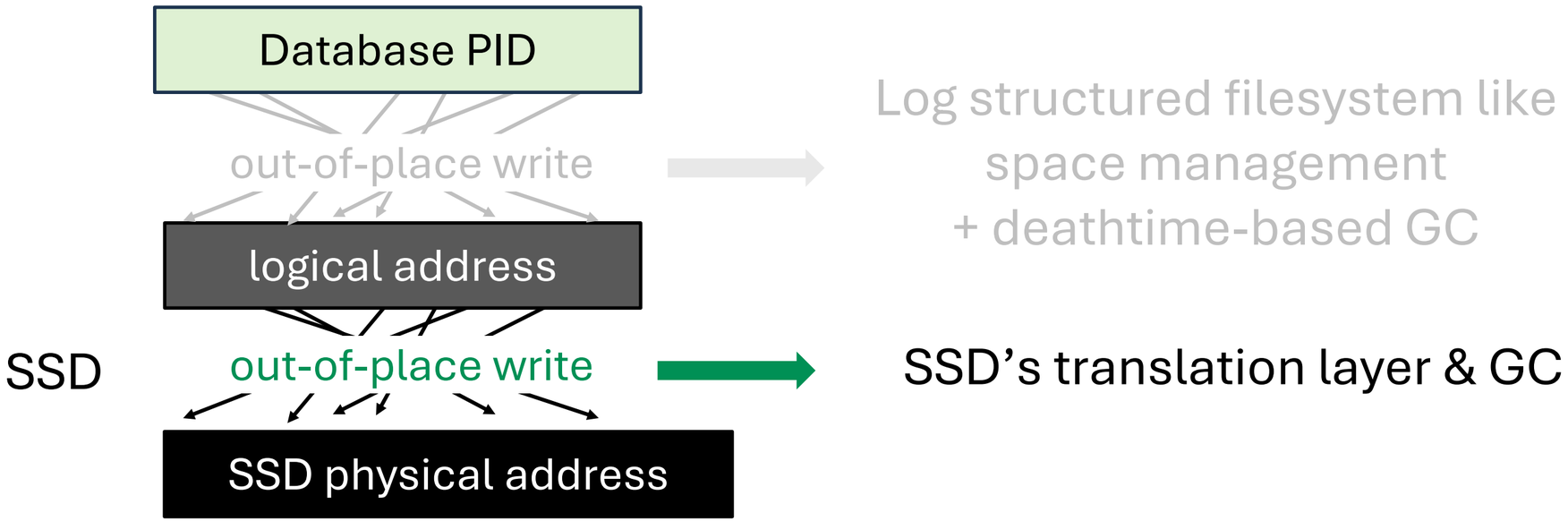
So far...

- We have DBMS doing out-of-place write and GC



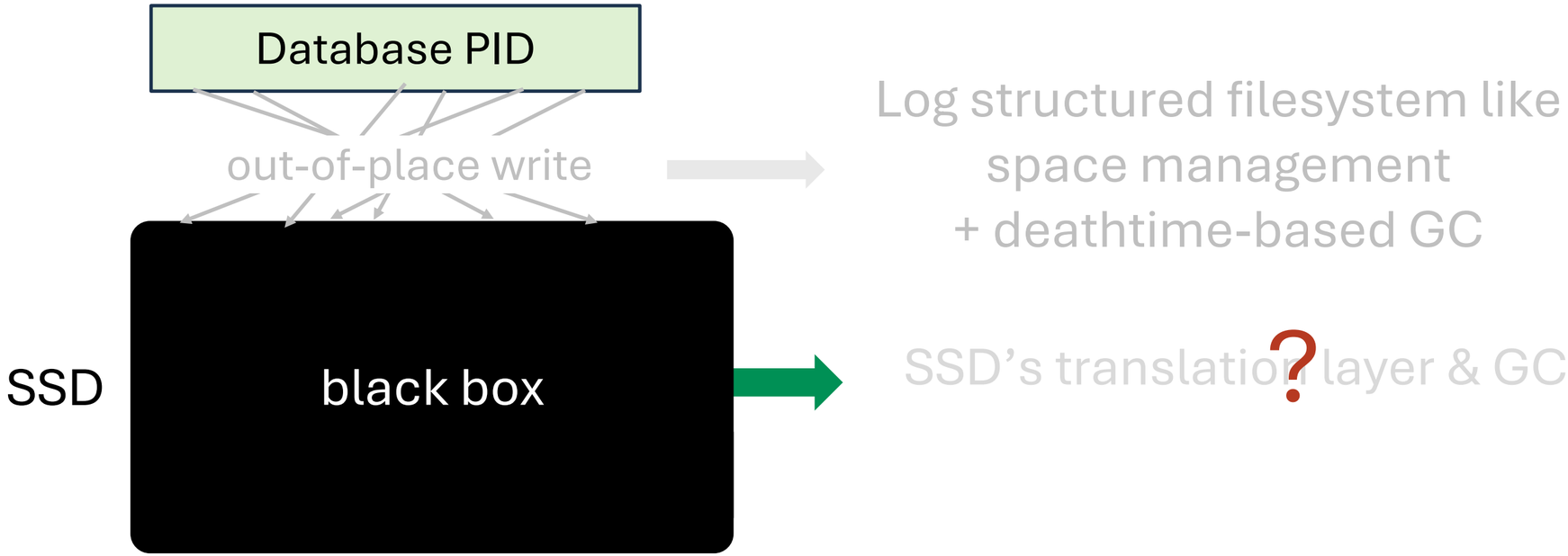
SSDs also do out-of-place write

- Turns out SSDs also do out-of-place write and GC



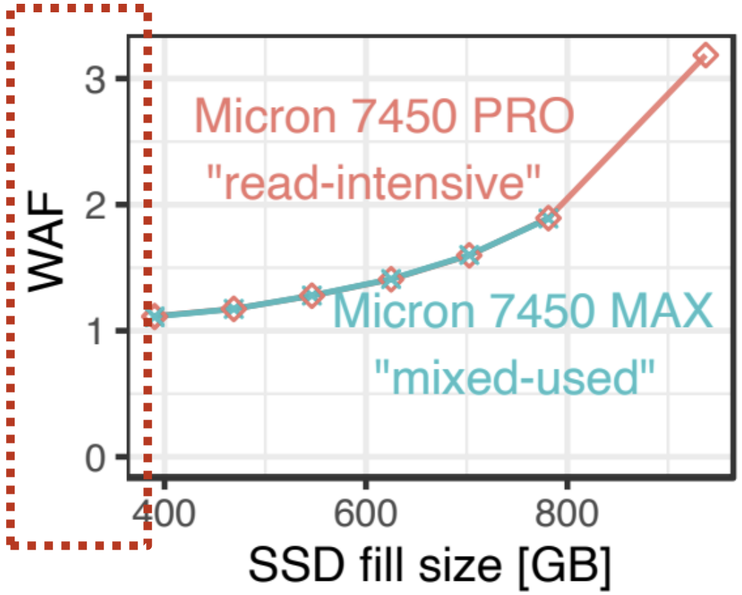
SSDs are blackboxes

- But, we don’t know what’s happening inside the **blackbox**



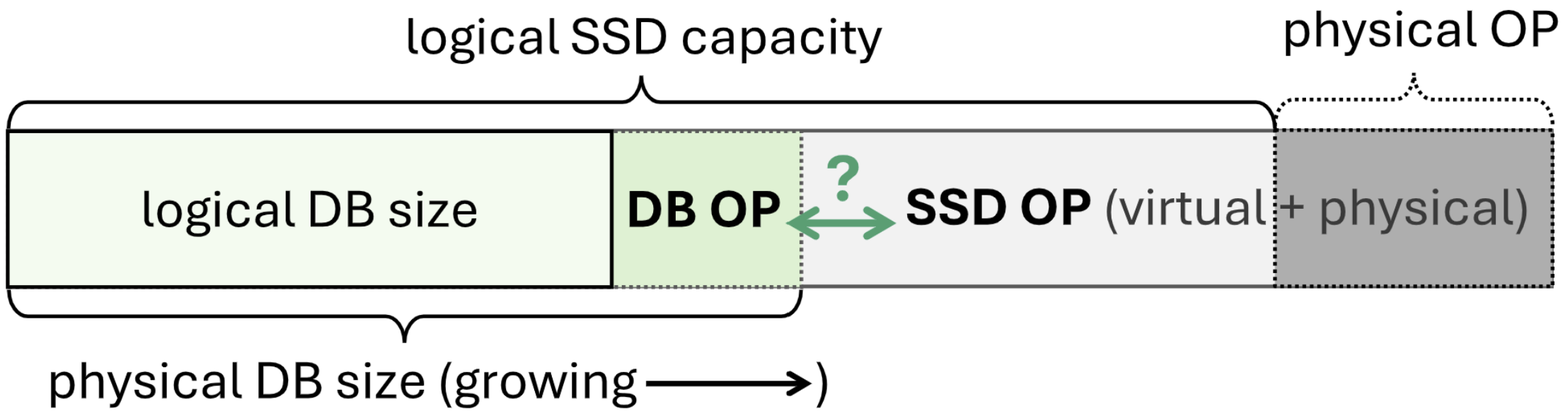
Modern SSDs can exhibit very high WAF

- SSD WAF can be very high, depending on various factors like **fill factor**



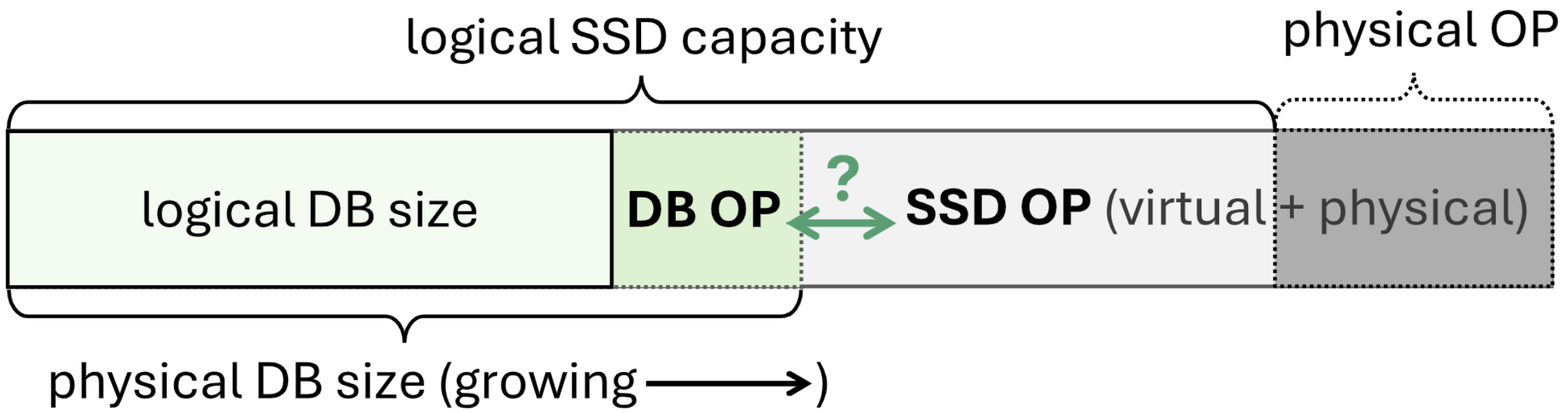
SSDiq: Haas et.al. VLDB25

Two layers of indirection and GC



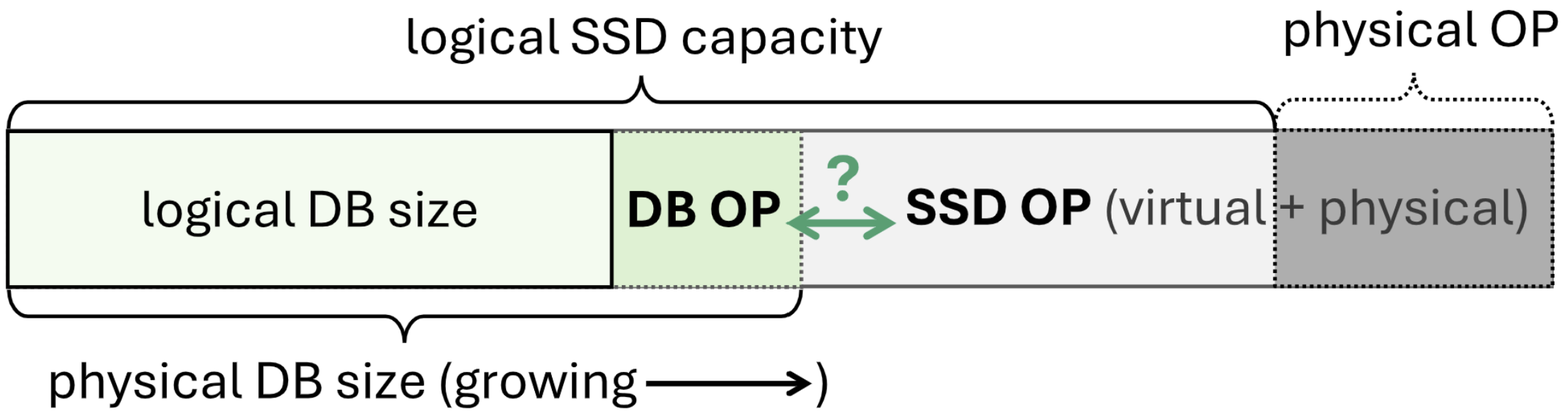
- DB GC is fighting against SSD GC for limited **OP space**

Two layers of indirection and GC



- DB GC is fighting against SSD GC for limited **OP space**
- **DB GC triggering point** splits the DB OP and SSD OP space

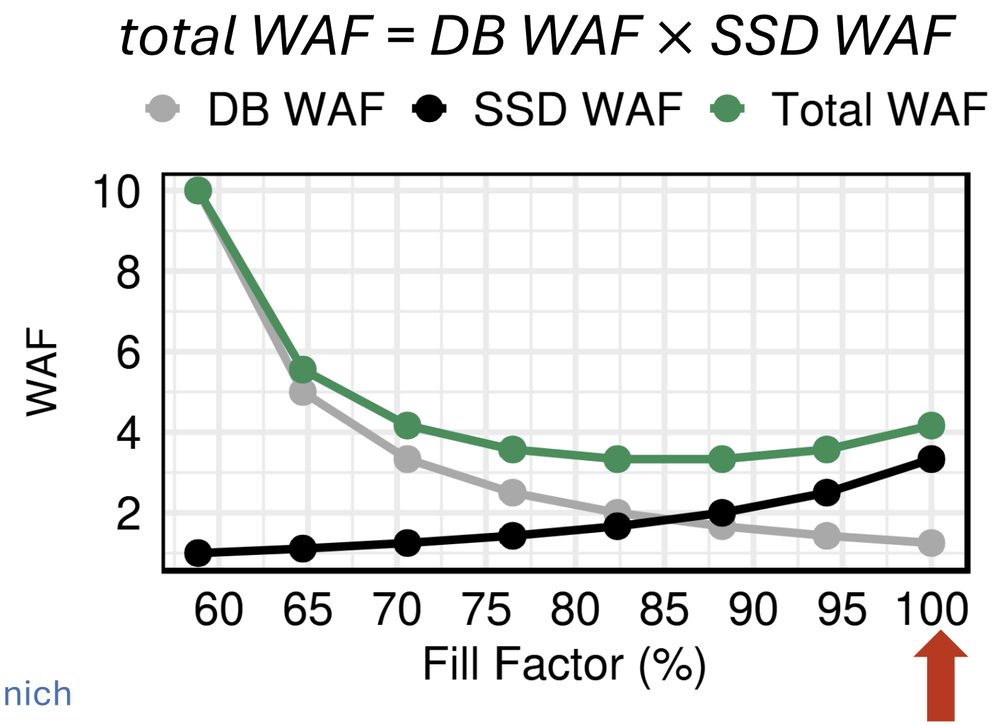
Two layers of indirection and GC



- DB GC is fighting against SSD GC for limited OP space
- DB GC triggering point splits the DB OP and SSD OP space
- Thus, DB GC behavior can shape **both** DB WAF and SSD WAF

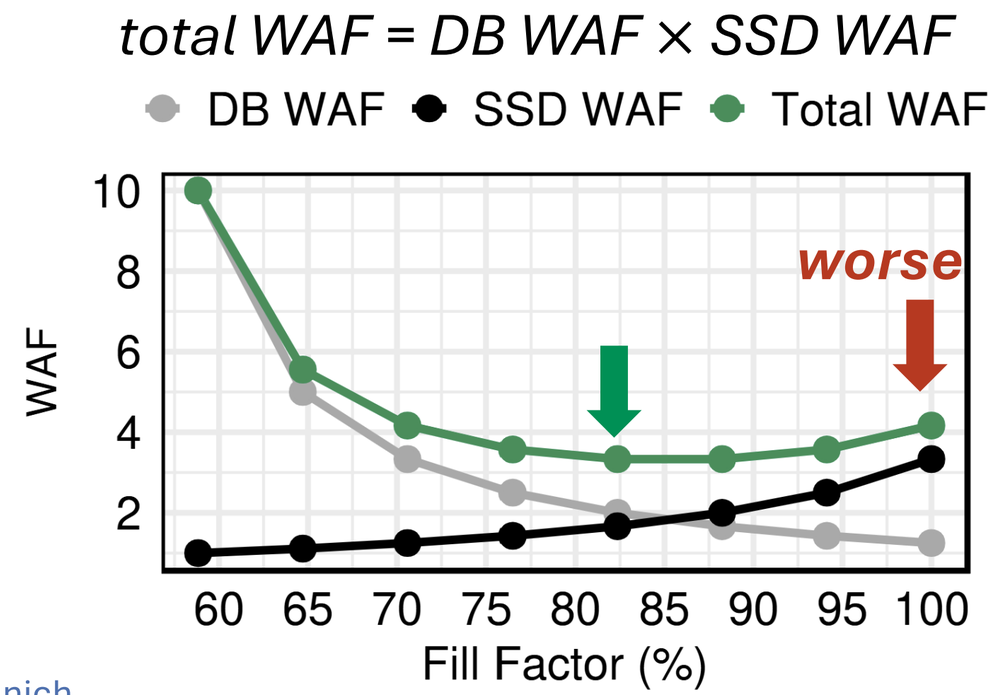
DB GC decides both DB and SSD WAF

- Suppose we use **full SSD capacity** to lower DB WAF



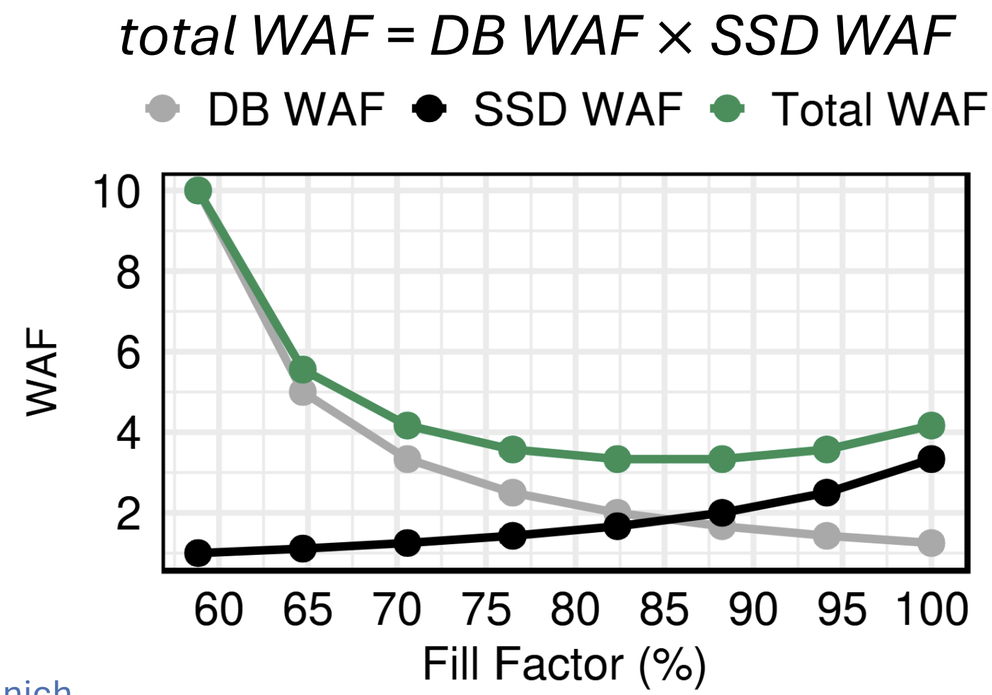
DB GC decides both DB and SSD WAF

- Suppose we use full SSD capacity to lower DB WAF
- But in return, it can paradoxically result in more flash writes

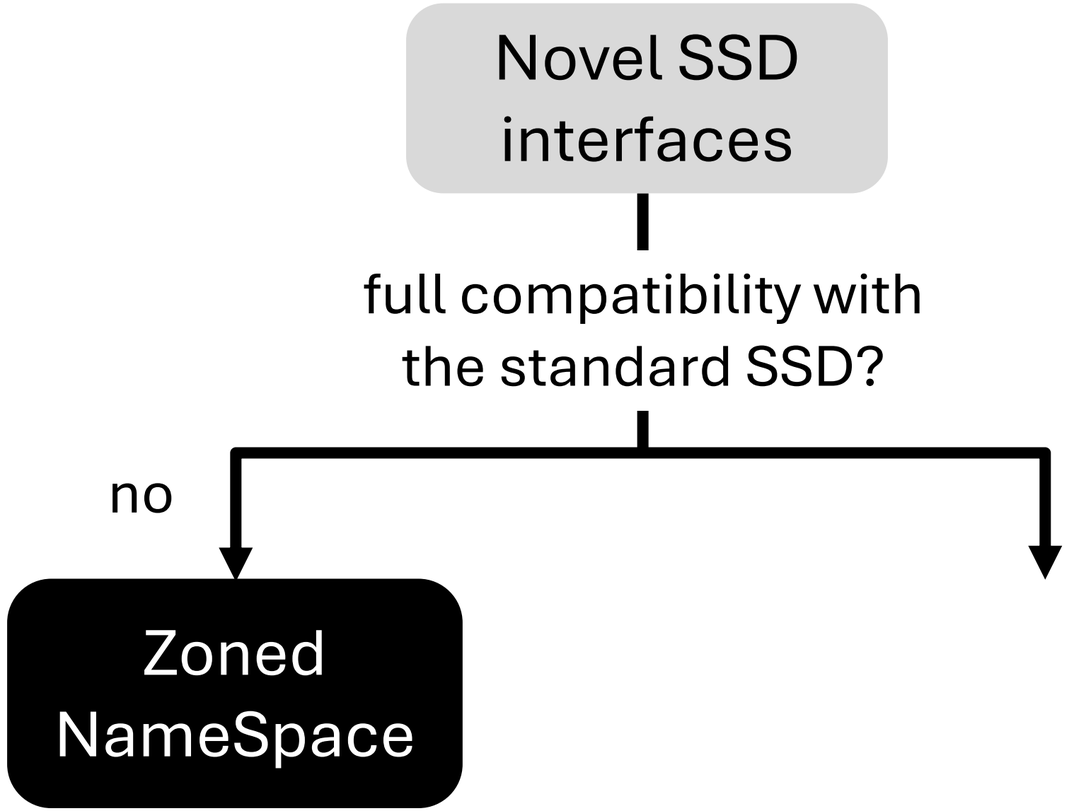


DB GC decides both DB and SSD WAF

- Suppose we use full SSD capacity to lower DB WAF
- But in return, it can paradoxically result in more flash writes
- Thus, DB GC should operate considering **total WAF**

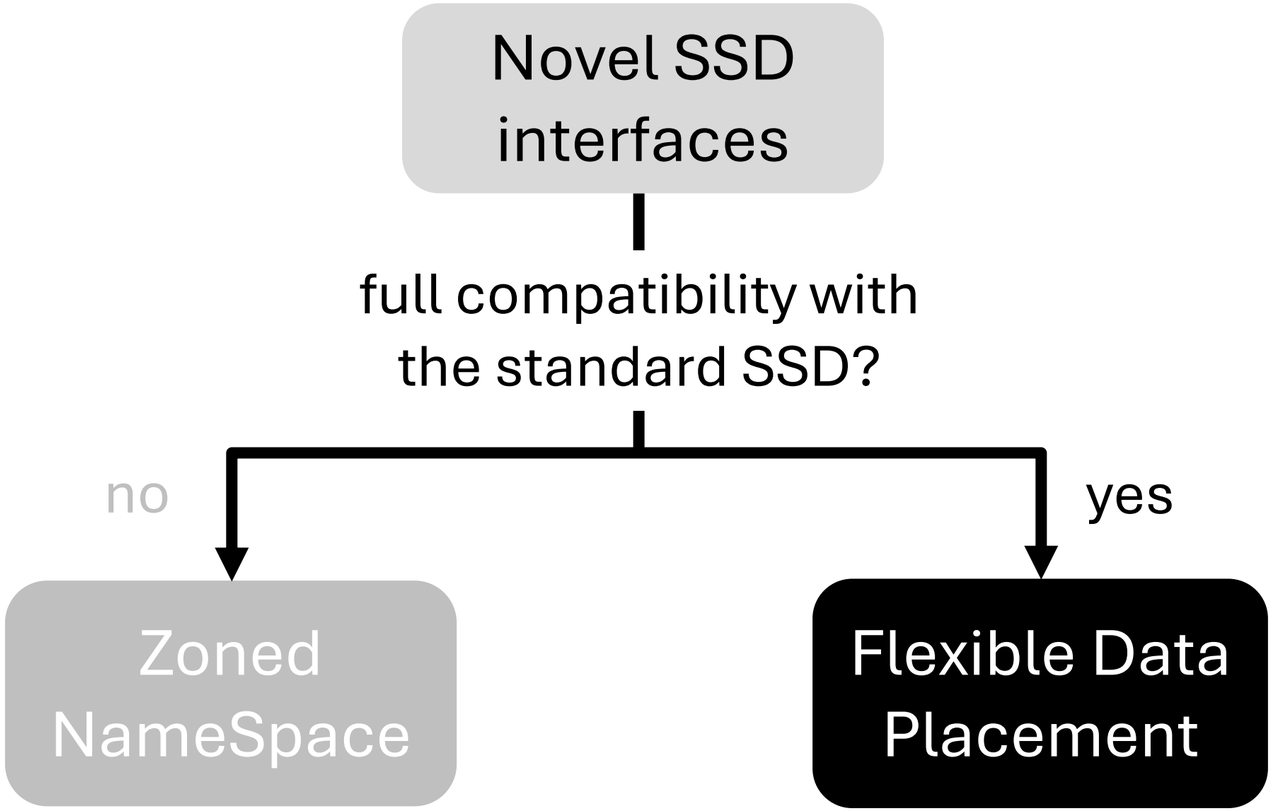


Different SSD interfaces to address WAF



- zone interface (append only)
- no SSD GC, no SSD WAF
- but GC offloaded to host

Different SSD interfaces to address WAF

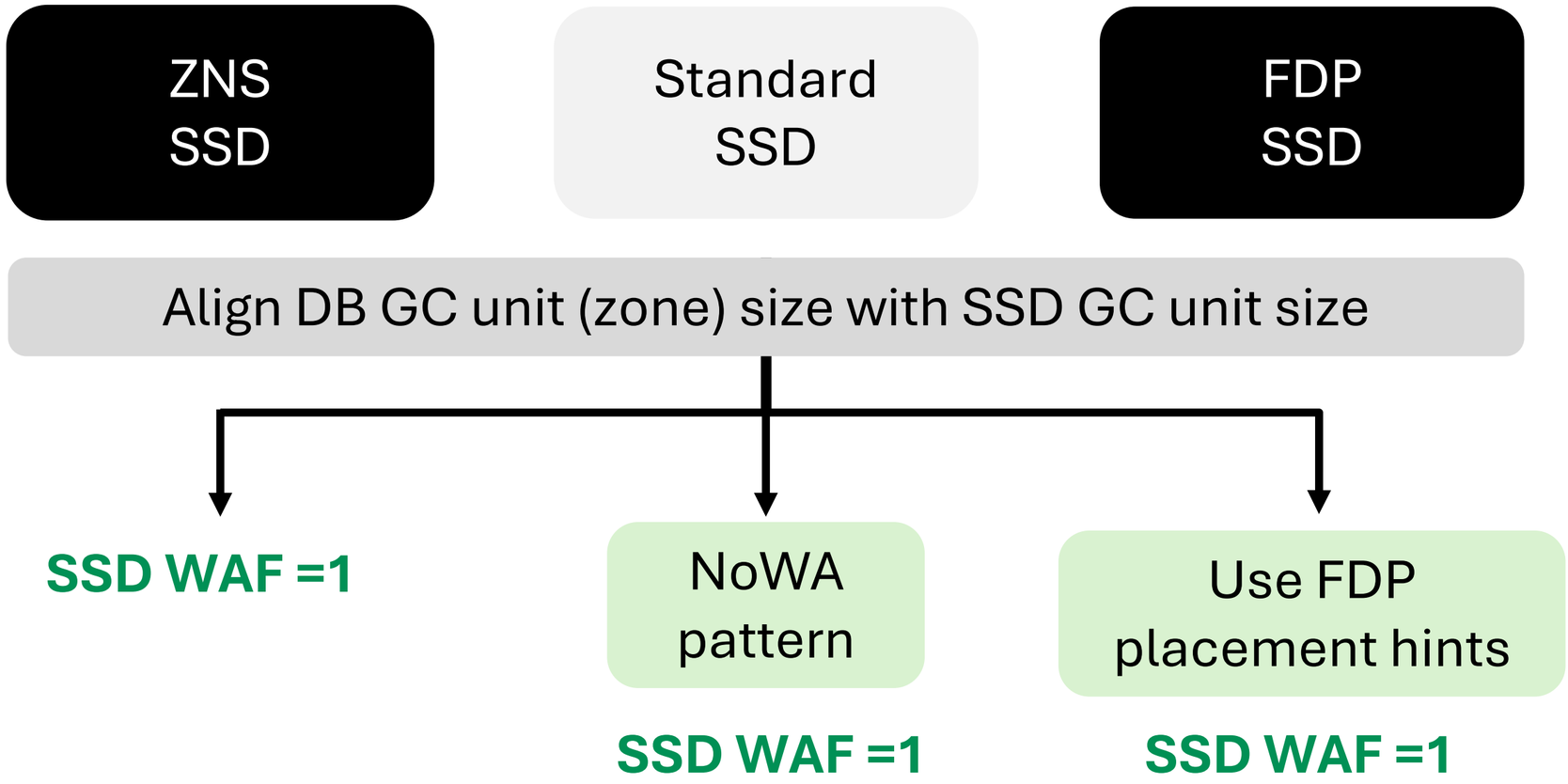


- zone interface (append only)
- no SSD GC, no SSD WAF
- but GC offloaded to host

- block interface
- SSD WAF can persist
- but can be mitigated by placement hints from host

SSD WAF=1 with three different SSD types

- Achieve **SSD WAF =1** even with full capacity usage



Out-of-place write-based optimizations

switch from in-place to *out-of-place write*

↓

+ compressing 4KiB pages

↓

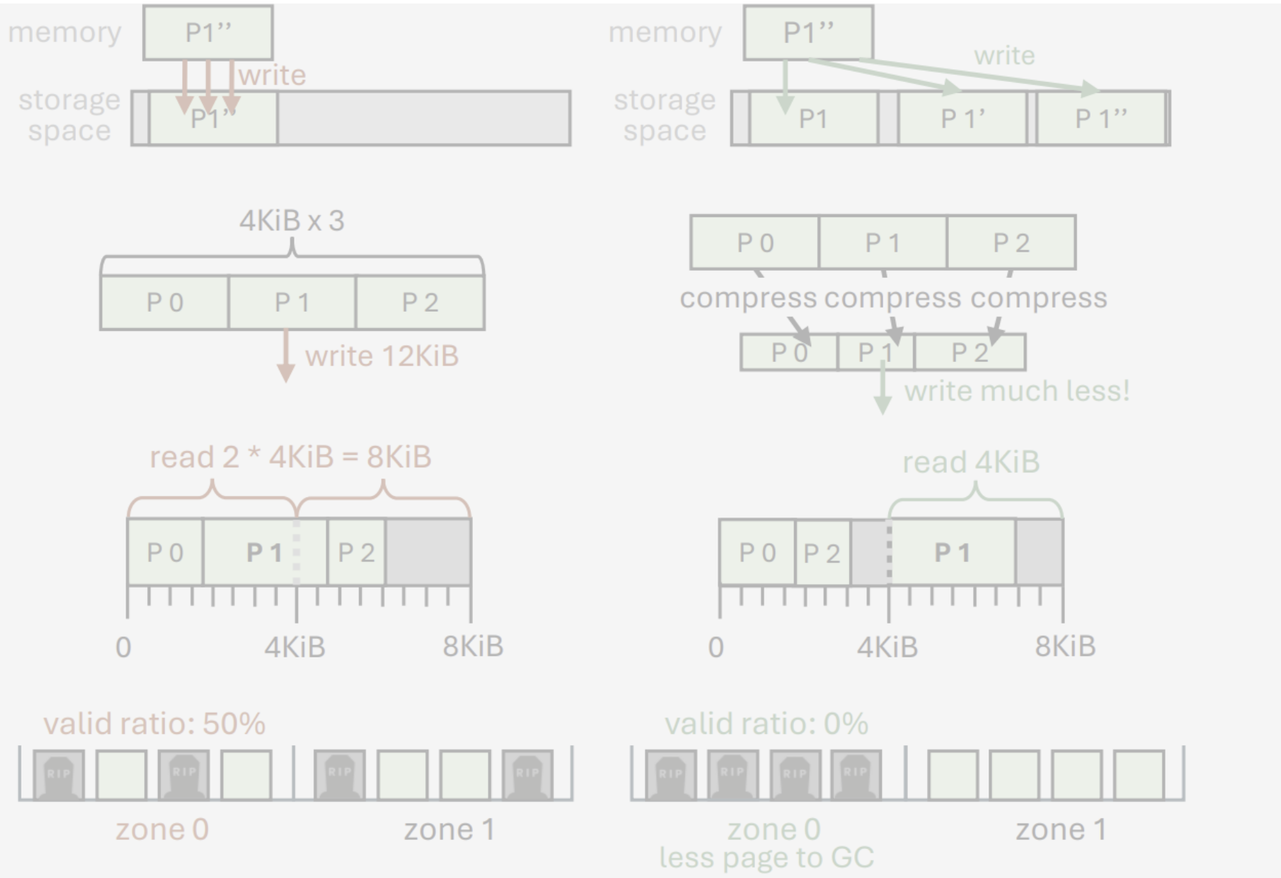
+ page packing compressed pages

↓

+ deathtime-based placement & GC

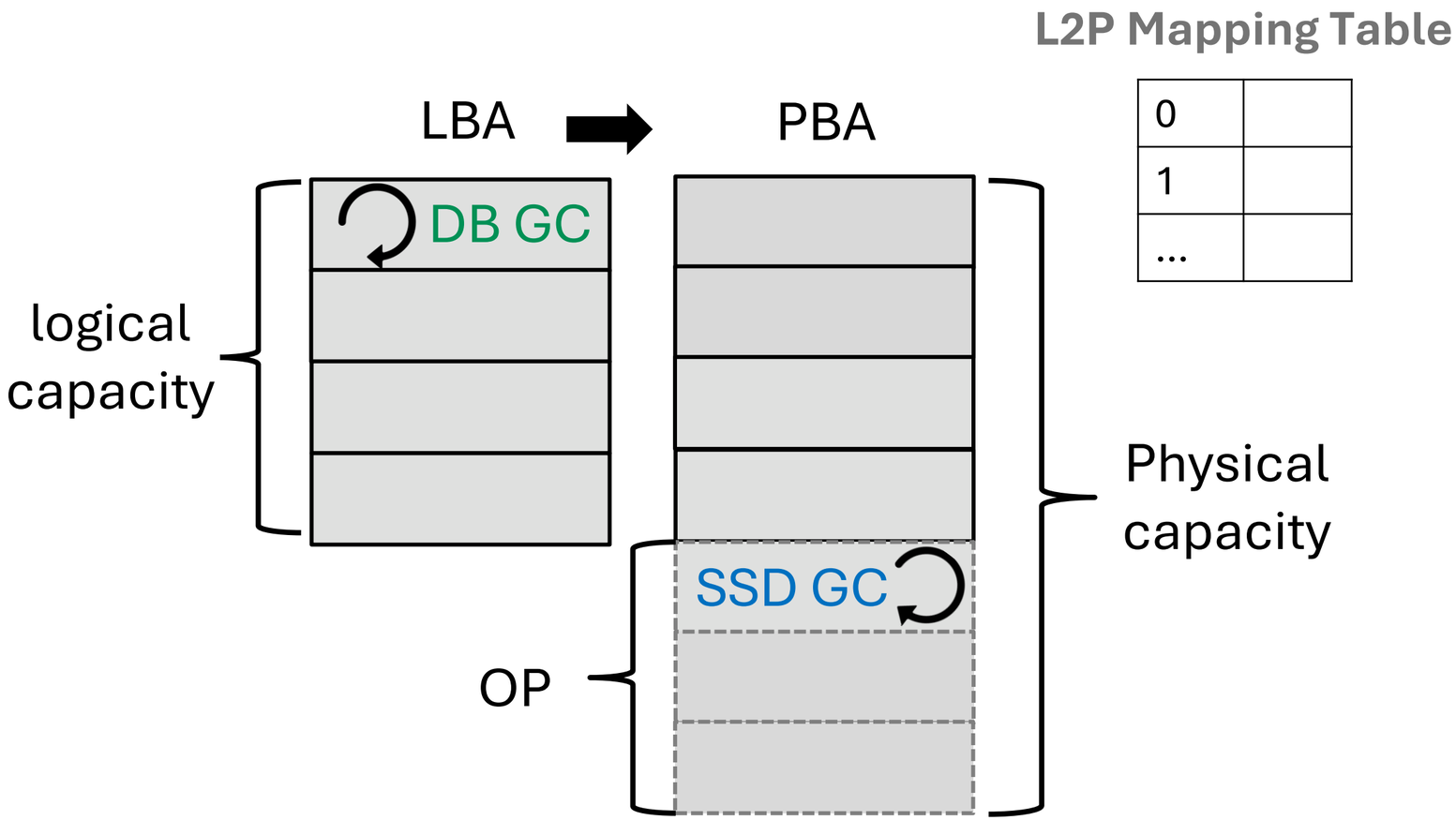
↓

+ aligning DB SSD GC unit size



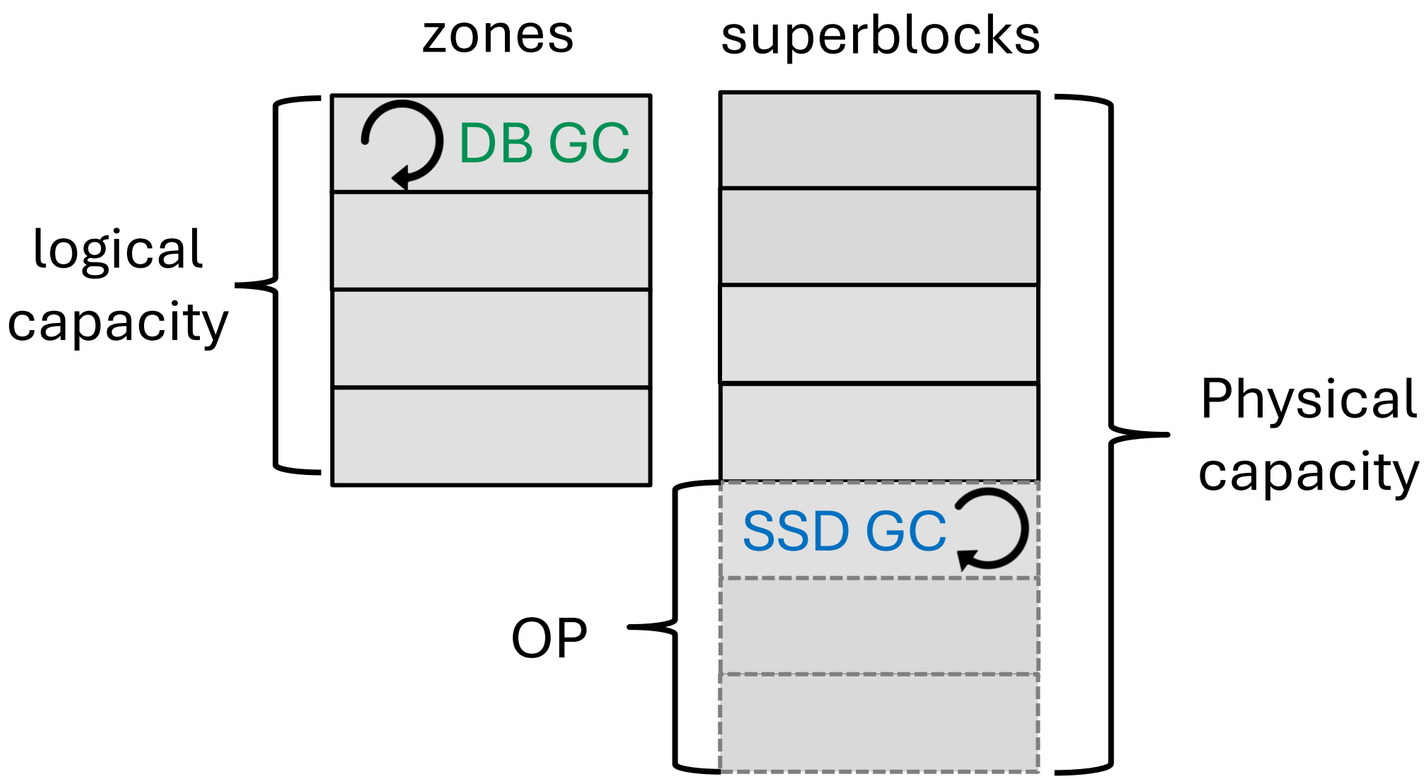
Logical to physical mapping inside SSD

- FTL **maps** the LBAs to physical BAs (PBA) to do out-of-place write



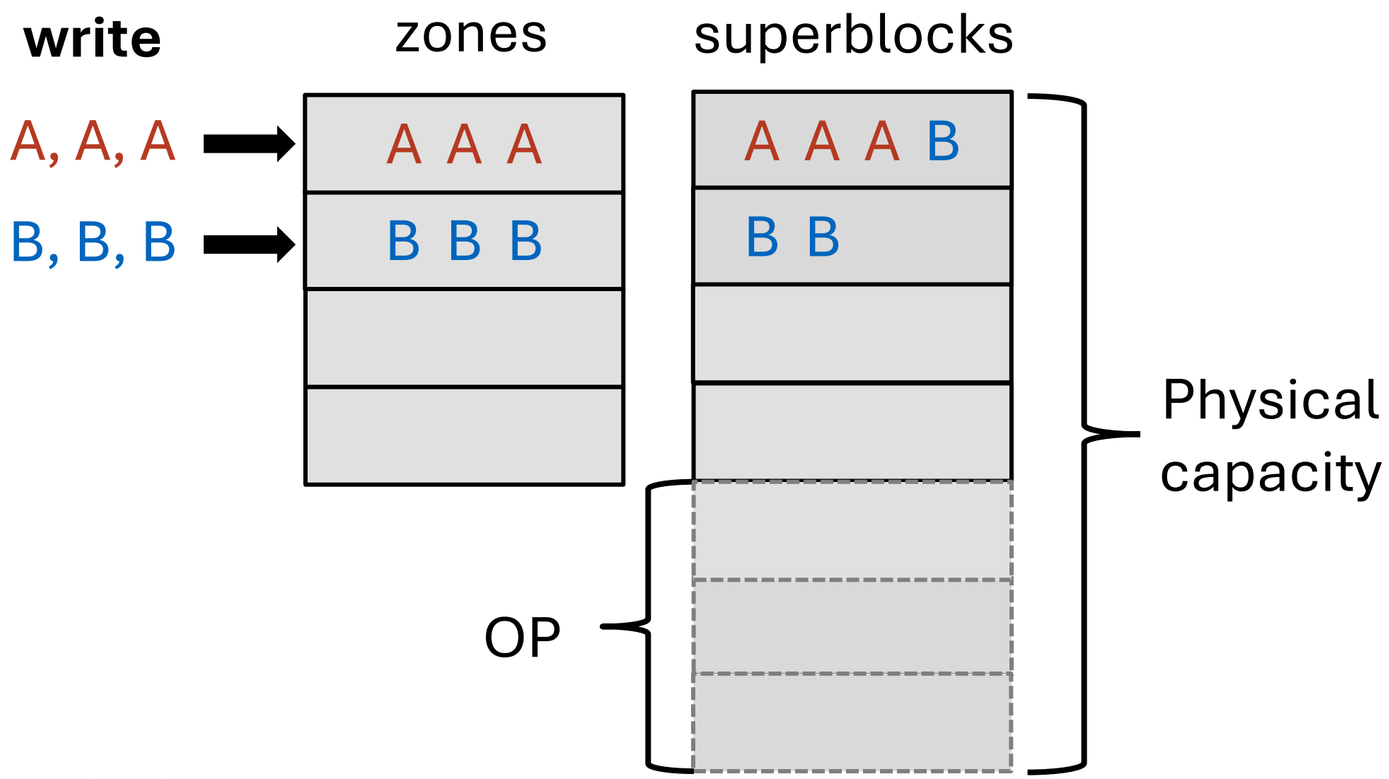
Logical to physical mapping inside SSD

- DB GC granularity: zone
- SSD GC granularity: superblock



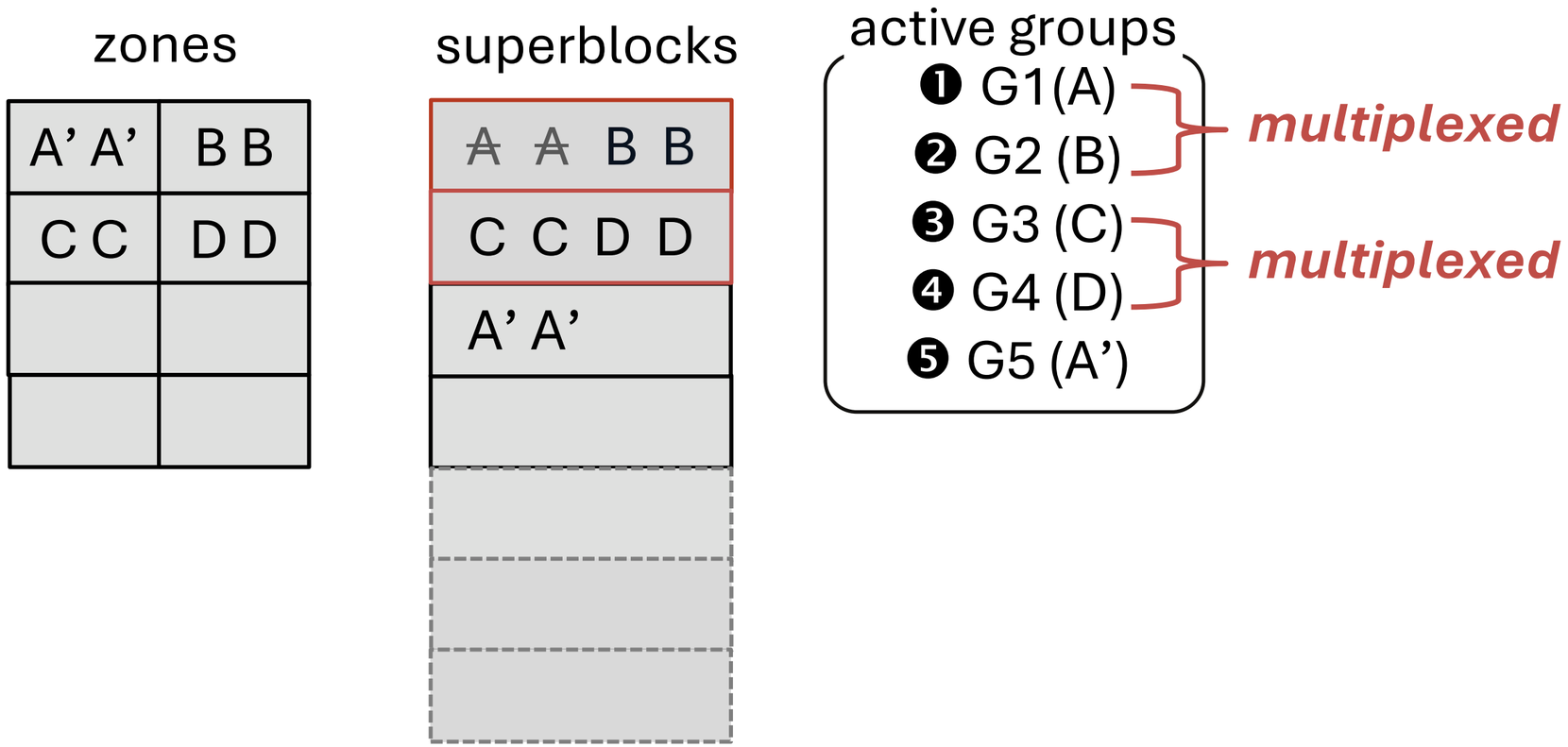
Logical to physical mapping inside SSD

- Stream of writes are appended to superblocks **in arrival order**
- E.g, zone 0 append → zone 1 append



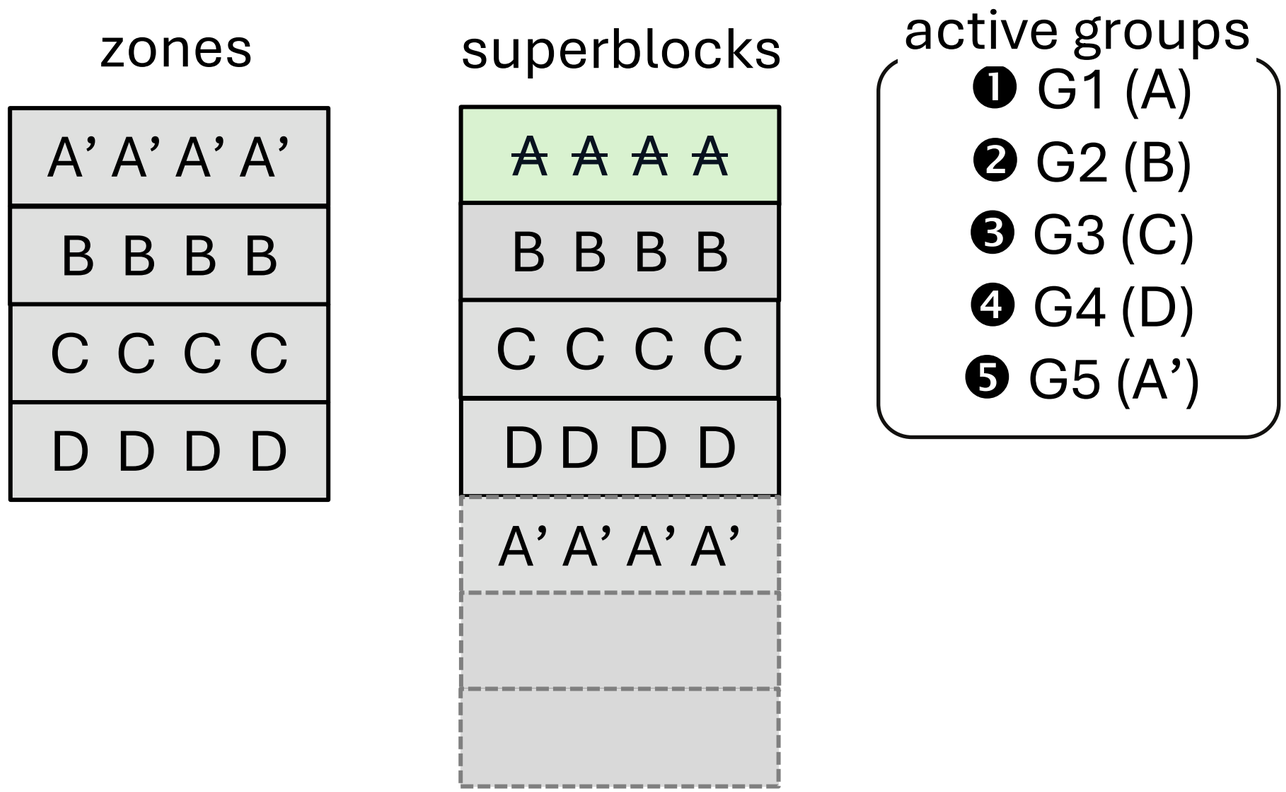
If zone size < SSD GC unit size

- Zone size < SSD GC unit size
- pages from different zones are **multiplexed** (FlashAlloc: Park et.al. VLDB23)



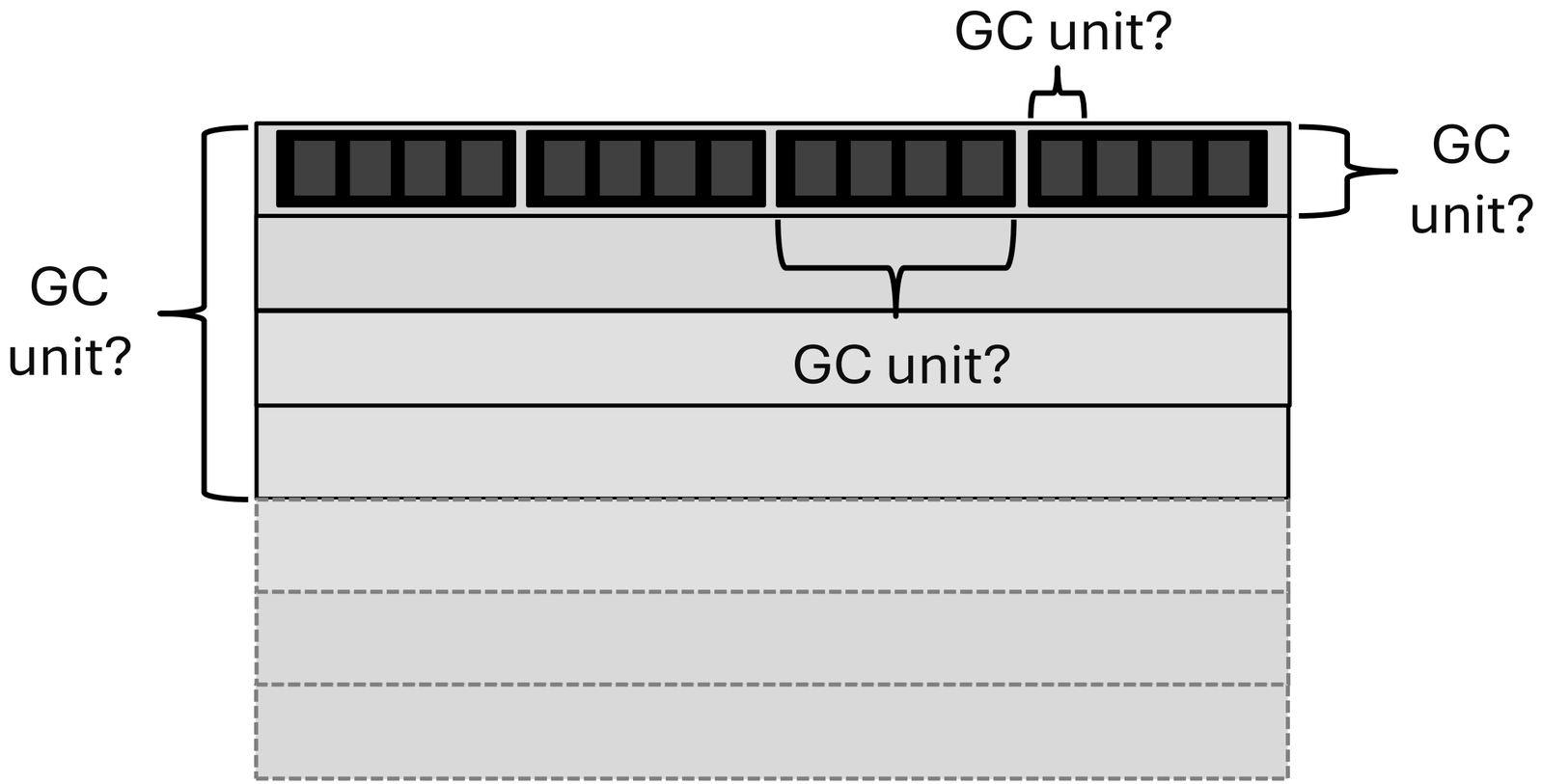
If zone size = SSD GC unit size

- Zone size = SSD GC unit size
- It is important to match the **zone size** with the **SSD GC unit size**



SSD GC granularity differs across SSDs

- SSD GC granularity varies across SSD models, vendors, capacity…
- Different size/number of parallel components



Optimization: Aligning DB SSD GC Unit Size

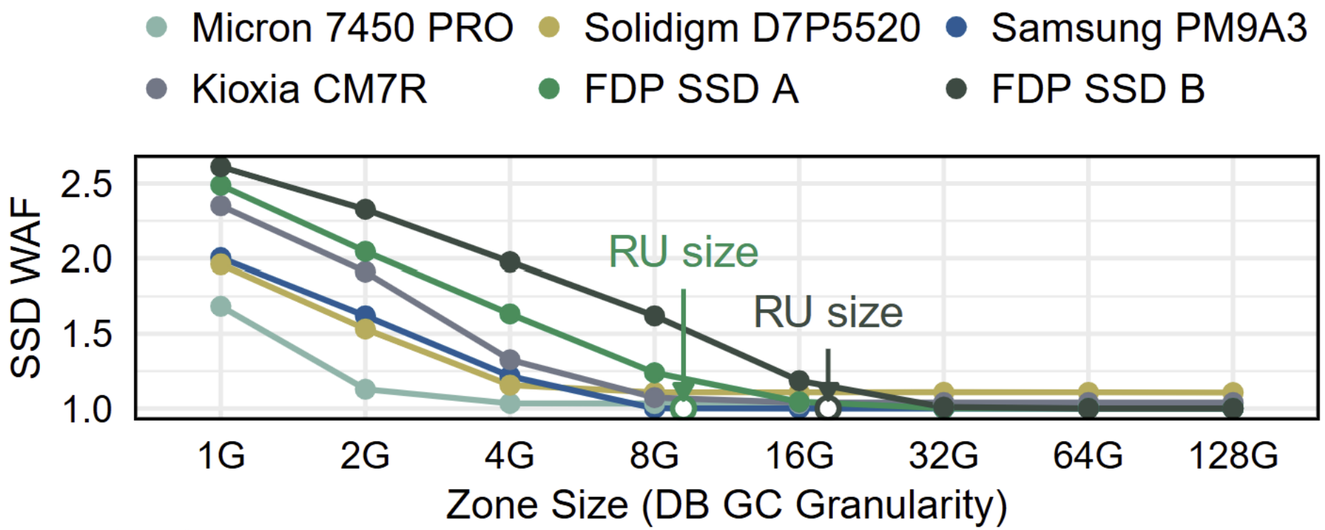
- **ZNS SSDs:** reference the **zone size** (~=1GBish for our ZNS)

Optimization: Aligning DB SSD GC Unit Size

- ZNS SSDs: reference the zone size (~=1GBish for our ZNS)
- **FDP-enabled SSDs:** reference the **RU (Reclaim Unit) size**

Optimization: Aligning DB SSD GC Unit Size

- ZNS SSDs: reference the zone size (~=1GBish for our ZNS)
- FDP-enabled SSDs: reference the RU (Reclaim Unit) size
- **Standard SSDs**: use **ZNS-like write pattern** to find the approx. SSD GC unit size



Out-of-place write-based optimizations

switch from in-place to *out-of-place write*

↓

+ compressing 4KiB pages

↓

+ page packing compressed pages

↓

+ deathtime-based placement & GC

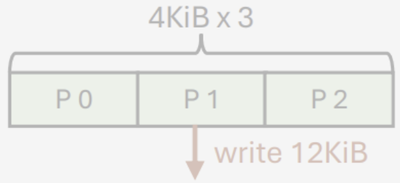
↓

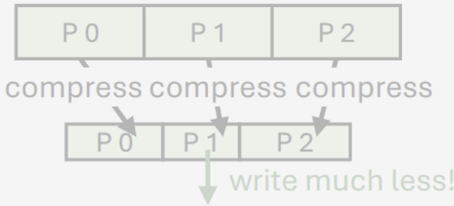
+ aligning DB SSD GC unit size

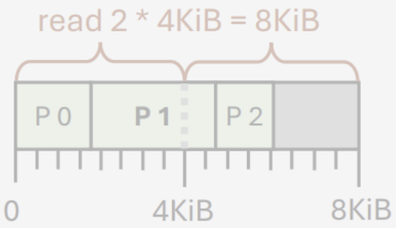
↓ *SSD WAF = 1 !*

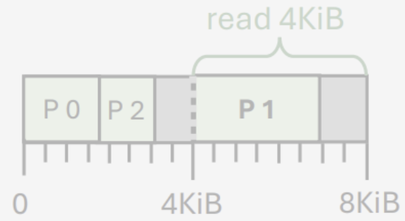
which SSD? → standard SSD | ZNS SSD | FDP SSD

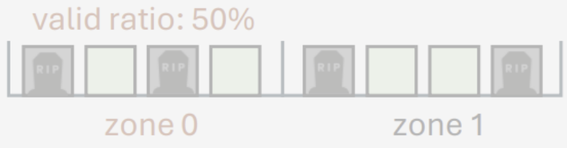
+ NoWA pattern + zone cmds + use placement hints

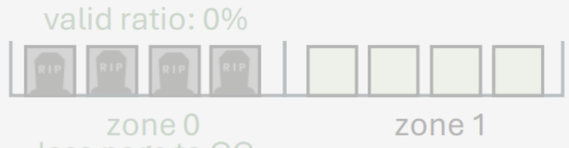


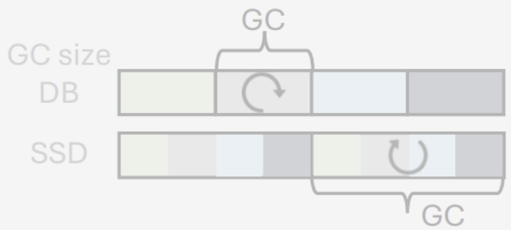


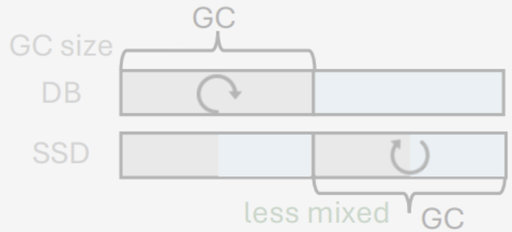






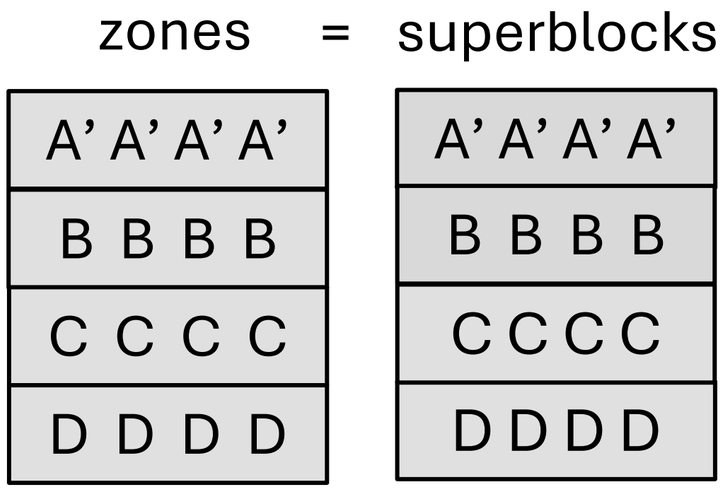






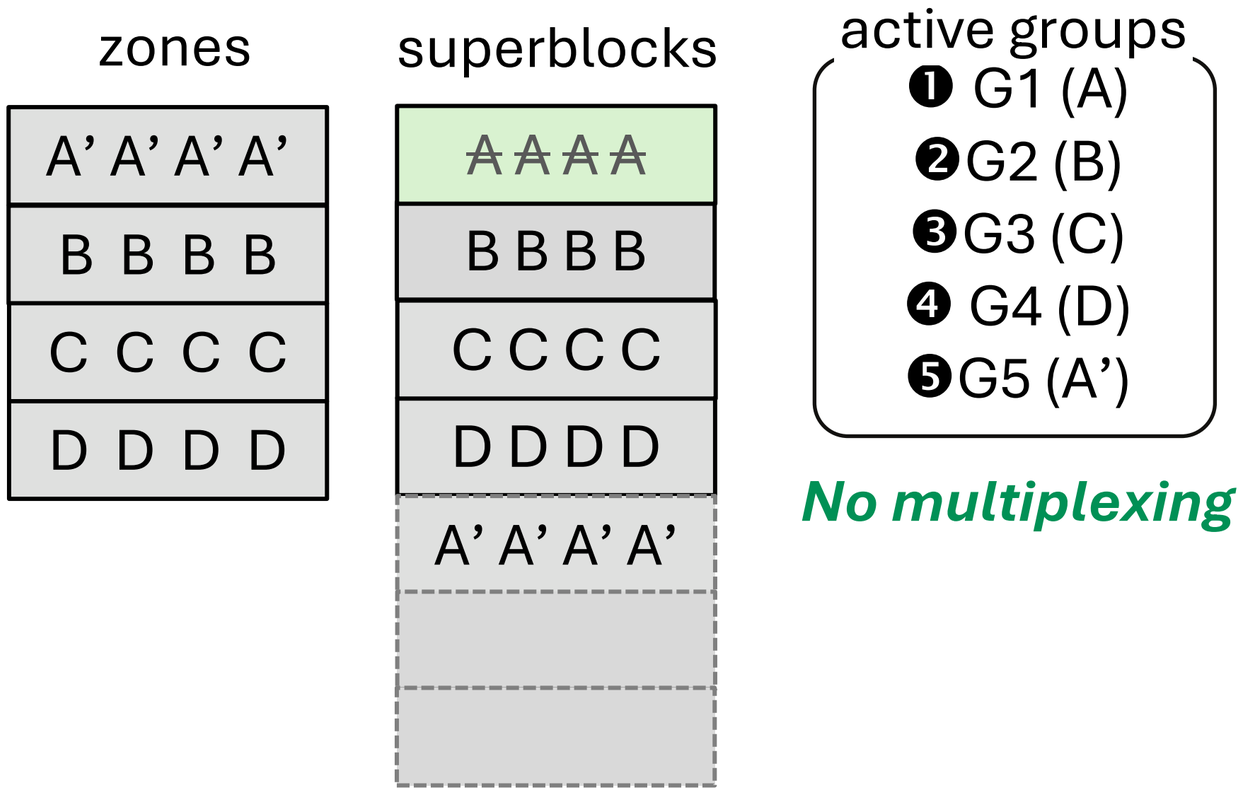
ZNS guarantees SSD WAF = 1

- No internal GC, no SSD WA, no SSD internal OP space
- Total WAF = DB WAF ~~x SSD WAF~~ → ***so simple!***



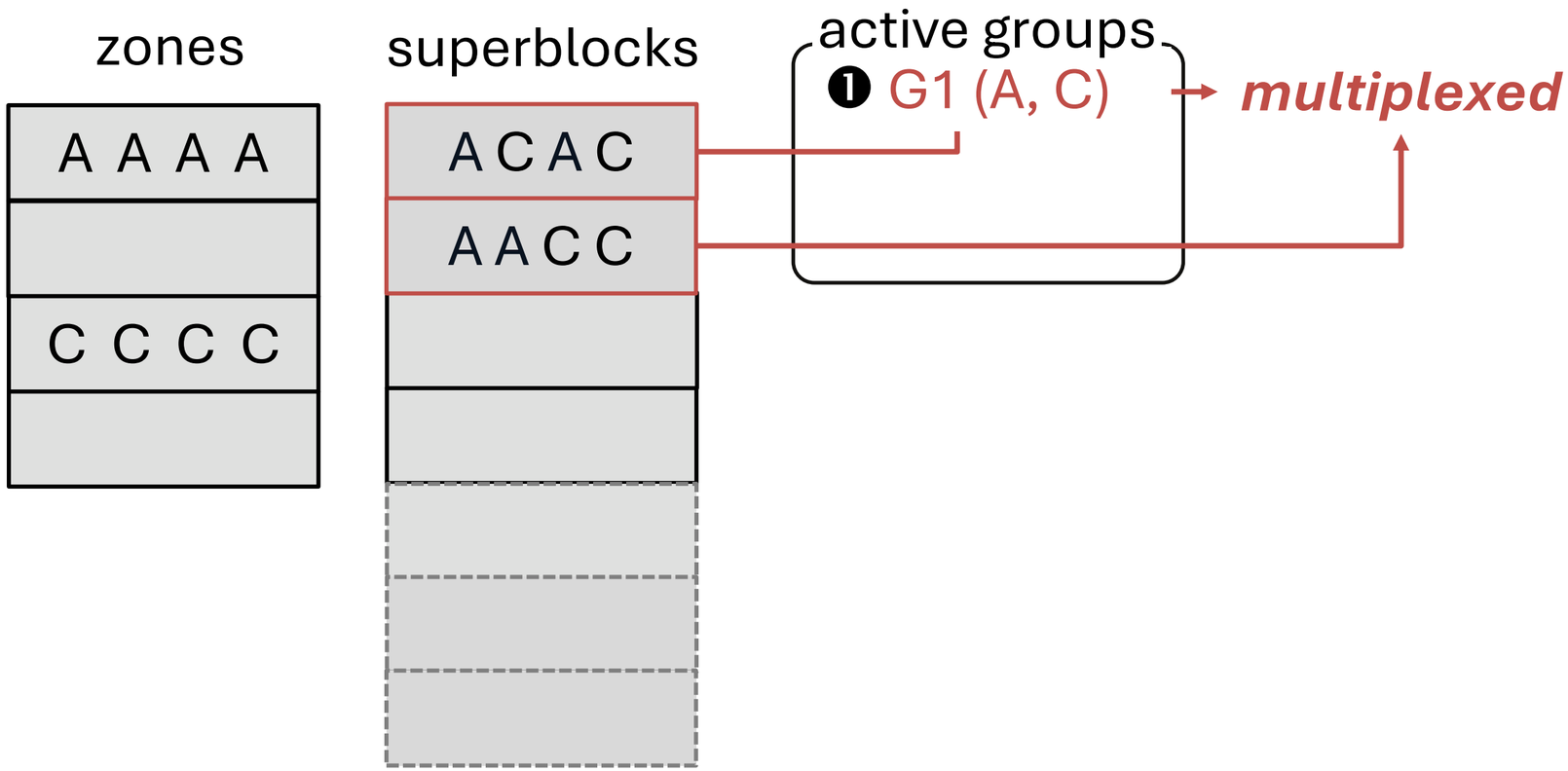
Standard SSDs (1) Multiplexing problem

- Standard SSDs can achieve SSD WAF =1 the same way as ZNS, as long as there is **one open zone** at a time



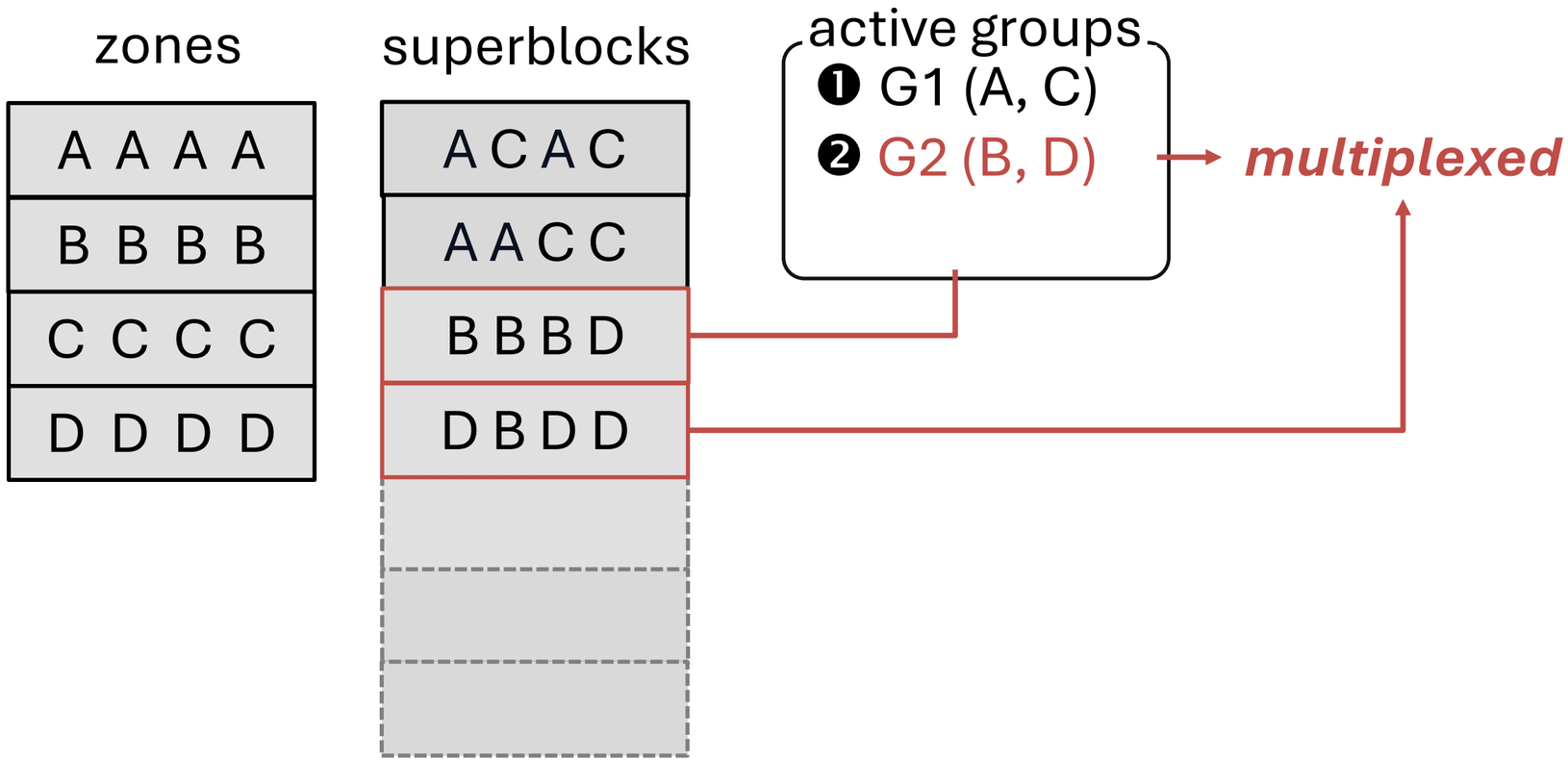
Standard SSDs (1) Multiplexing problem

- ***Multiplexing*** occurs across multiple superblocks when there are **multiple** active zones



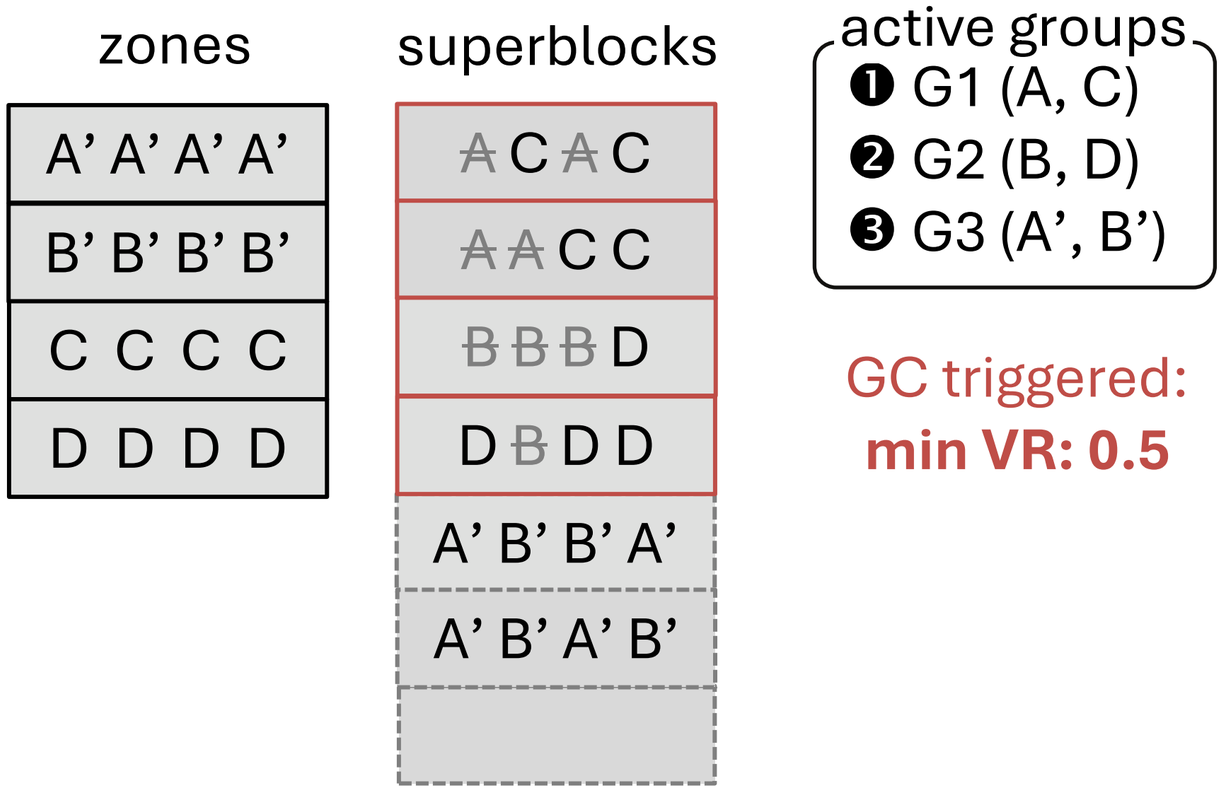
Standard SSDs (1) Multiplexing problem

- ***Multiplexing*** occurs across multiple superblocks when there are **multiple** active zones



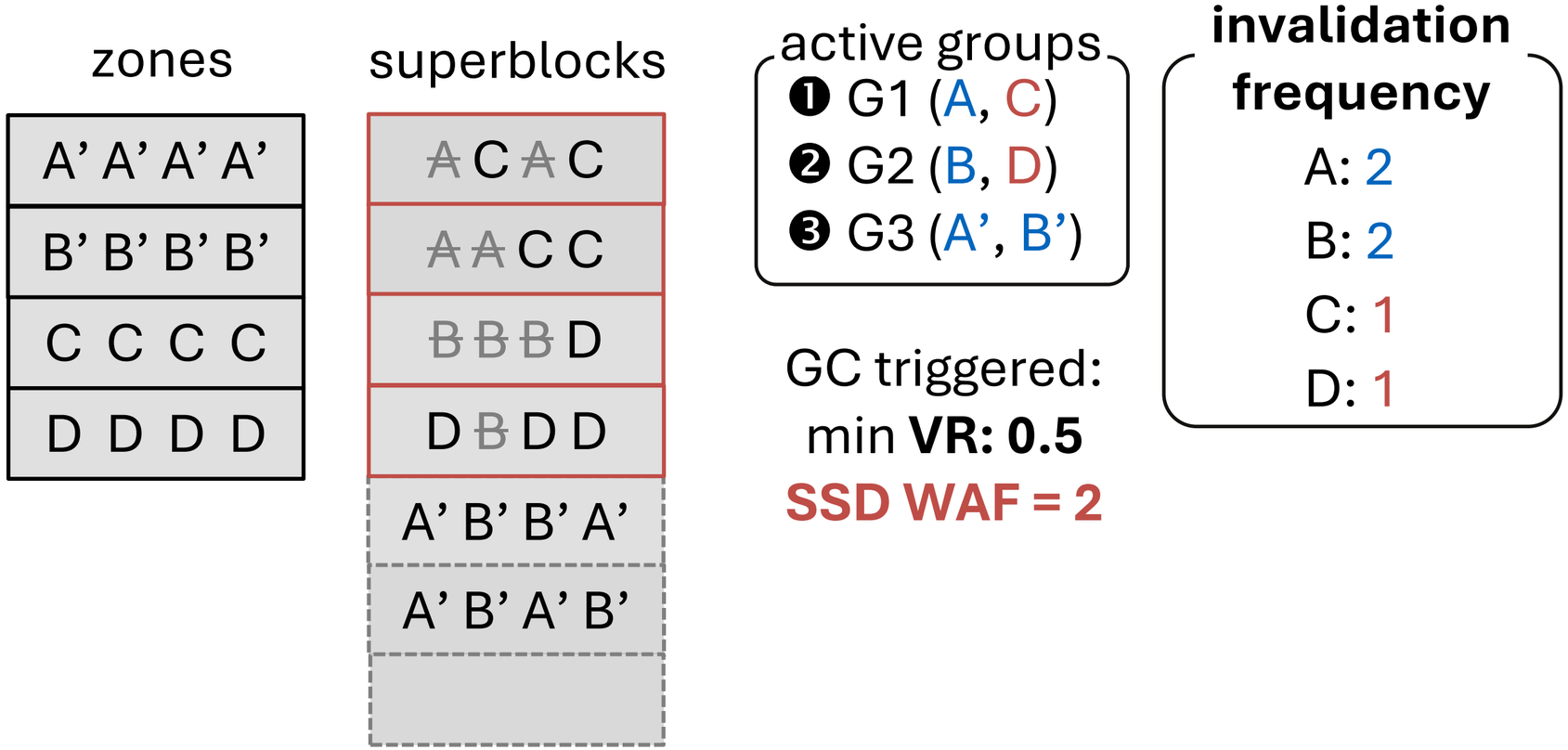
Standard SSDs (1) Multiplexing problem

- ***Multiplexing*** occurs across multiple superblocks when there are **multiple** active zones



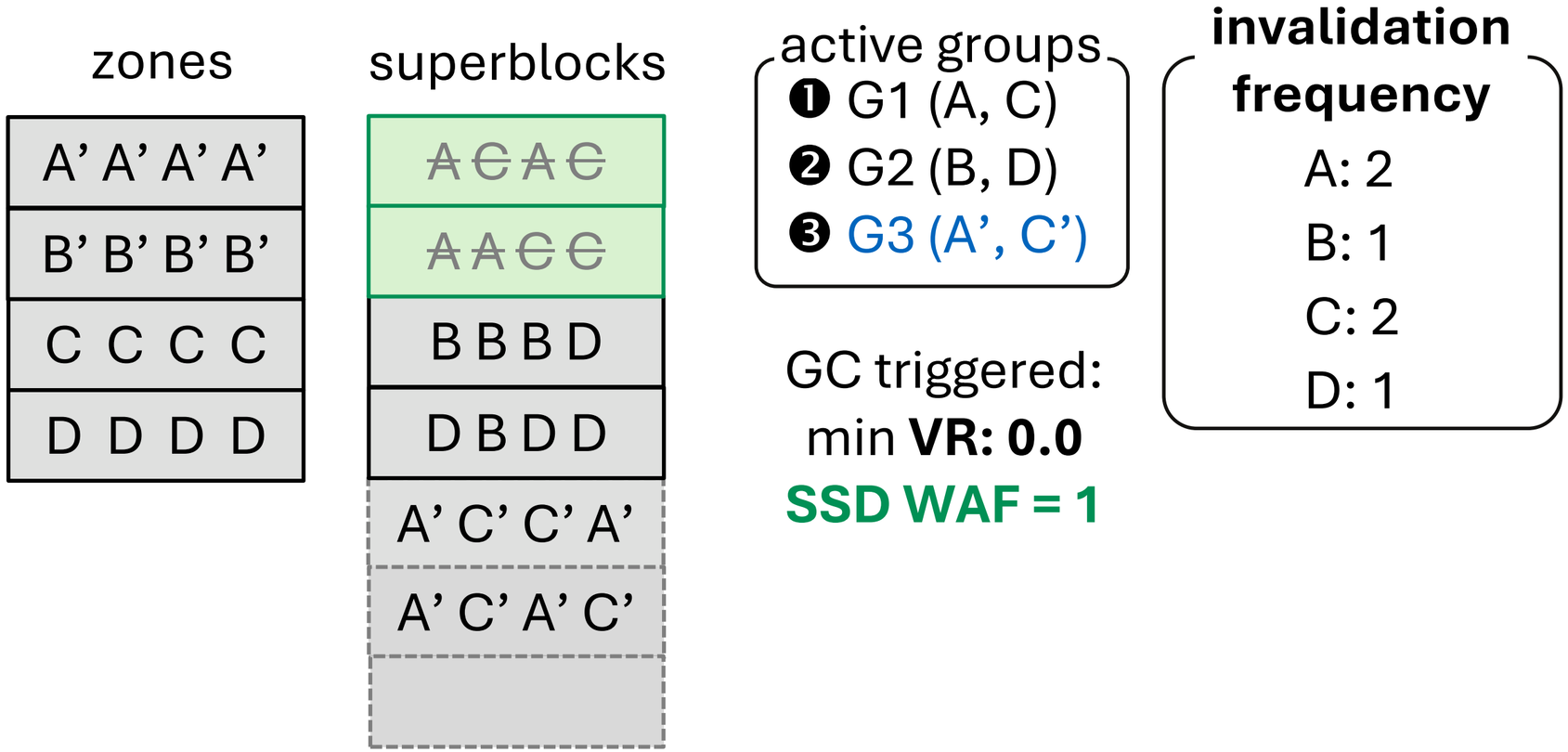
Standard SSDs (2) Frequency imbalance

- **WA** occurs as the **frequency imbalance** occurs across the zones inside the same active group



Standard SSDs (2) Frequency imbalance

- If there is **no frequency imbalance**, WAF remains 1



Standard SSD optimization: NoWA pattern

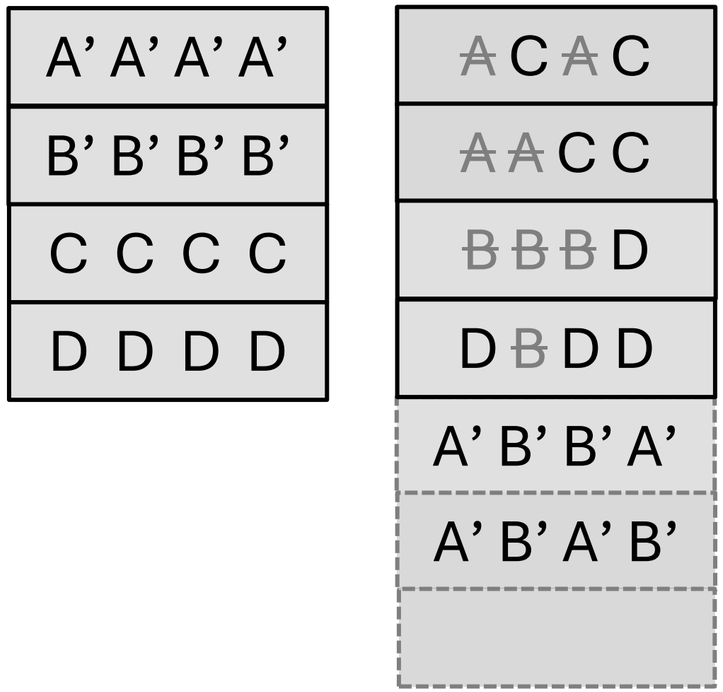
- **NoWA (No Write Amplification)** write pattern

- Guarantees **SSD WAF =1** even with full capacity utilization, as long as you maintain this write pattern

Standard SSD optimization: NoWA pattern

Principles of **NoWA** write pattern

- (1) zone size **=** SSD GC granularity



active groups

❶ G1 (A, C)

❷ G2 (B, D)

❸ G3 (A', B')

invalidation frequency

A: 2

B: 2

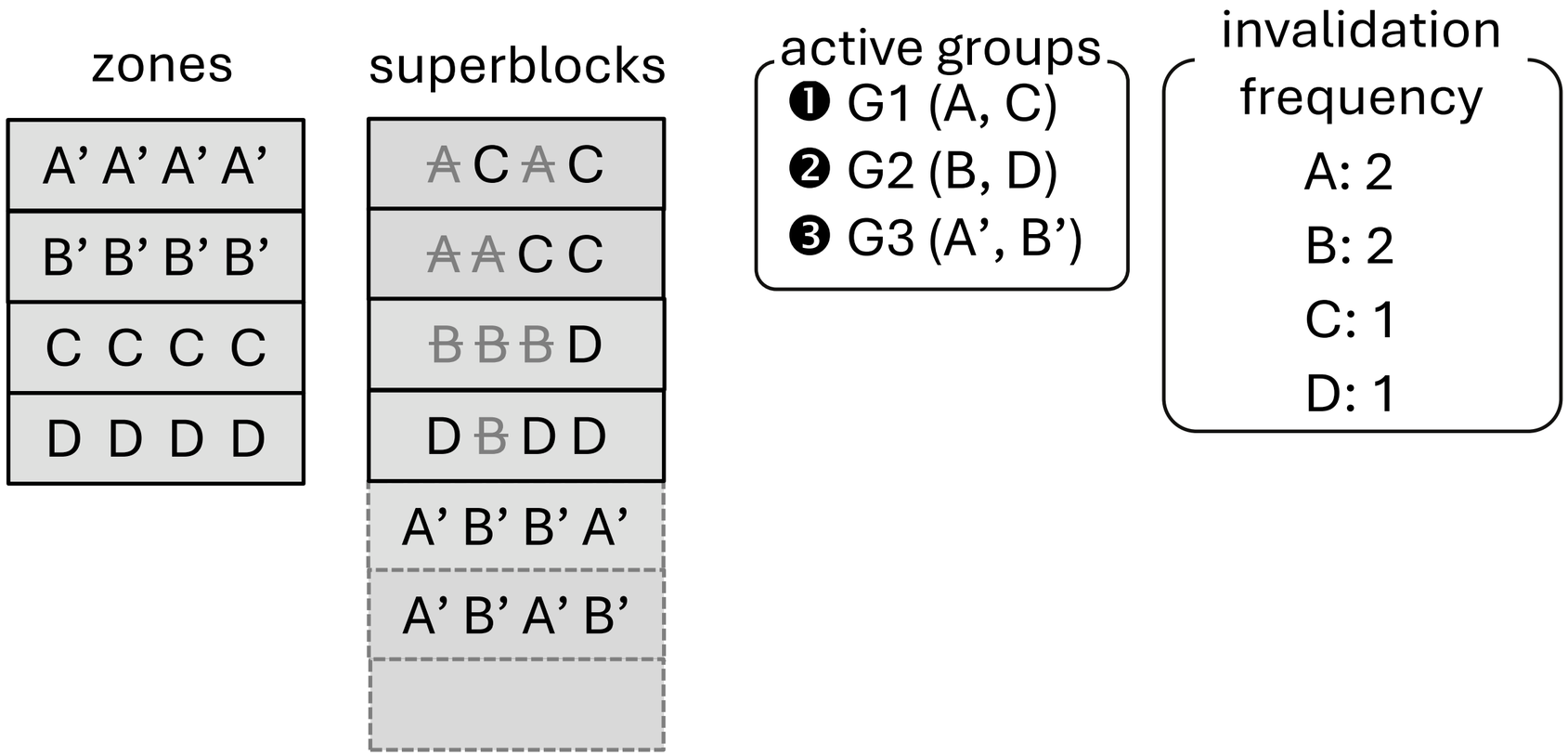
C: 1

D: 1

Standard SSD optimization: NoWA pattern

Principles of **NoWA** write pattern

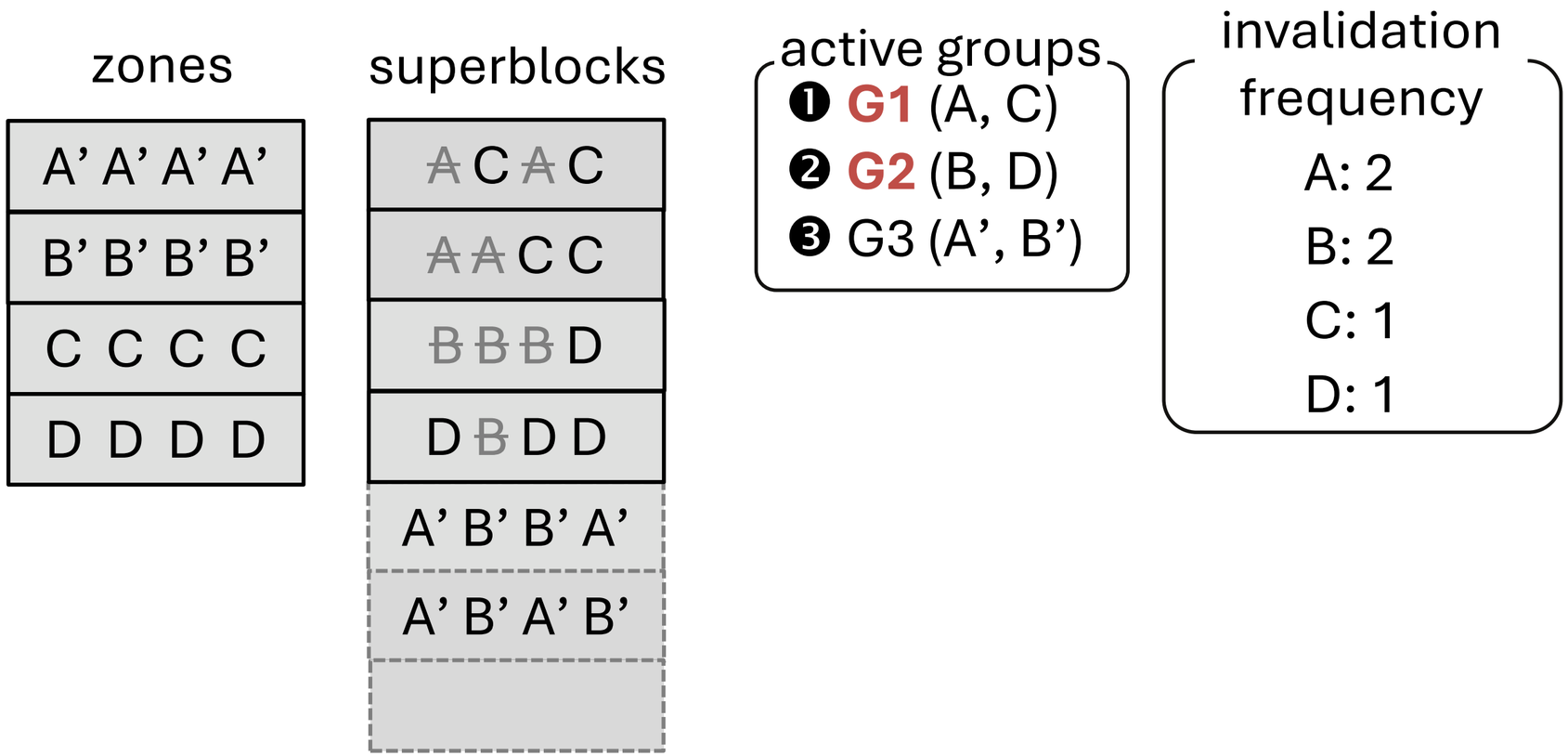
- (2) Do not open a new zone until all the active zones are full



Standard SSD optimization: NoWA pattern

Principles of **NoWA** write pattern

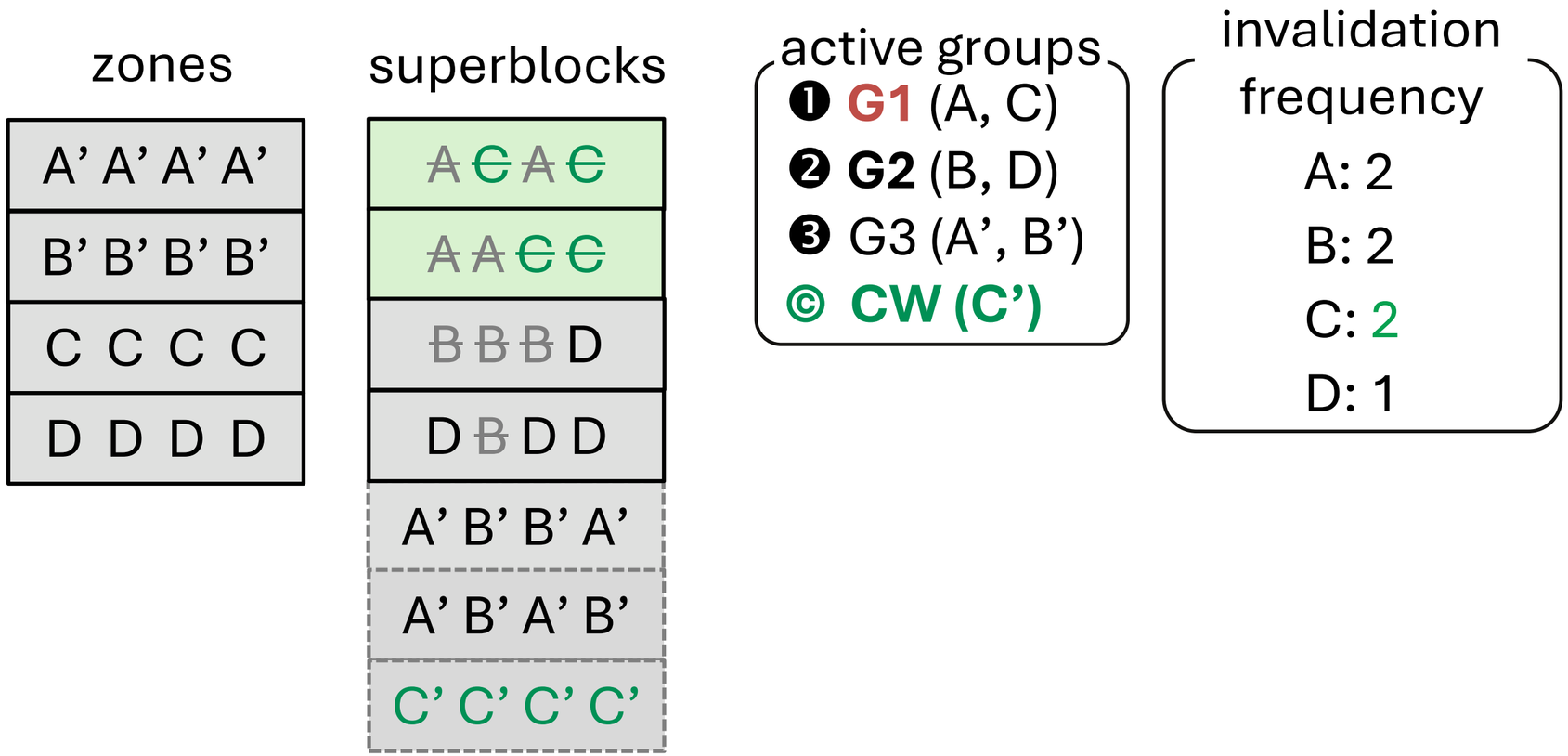
- (3) Detect *frequency imbalance* among active groups (**G1, G2**)



Standard SSD optimization: NoWA pattern

Principles of **NoWA** write pattern

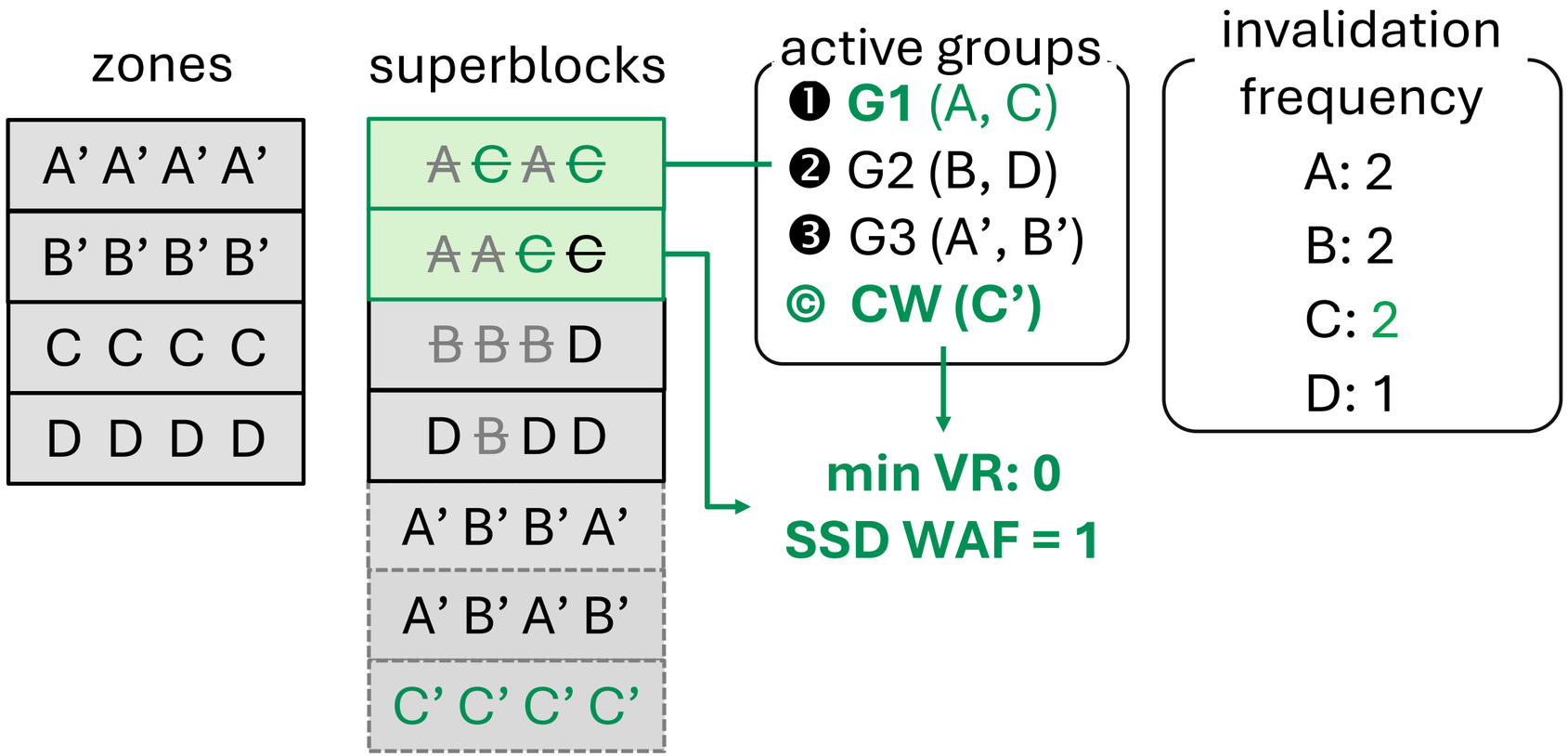
- (3) If detected, then issue **compensation write** to fix the frequency imbalance



Standard SSD optimization: NoWA pattern

Principles of **NoWA** write pattern

- (3) If detected, then issue compensation write



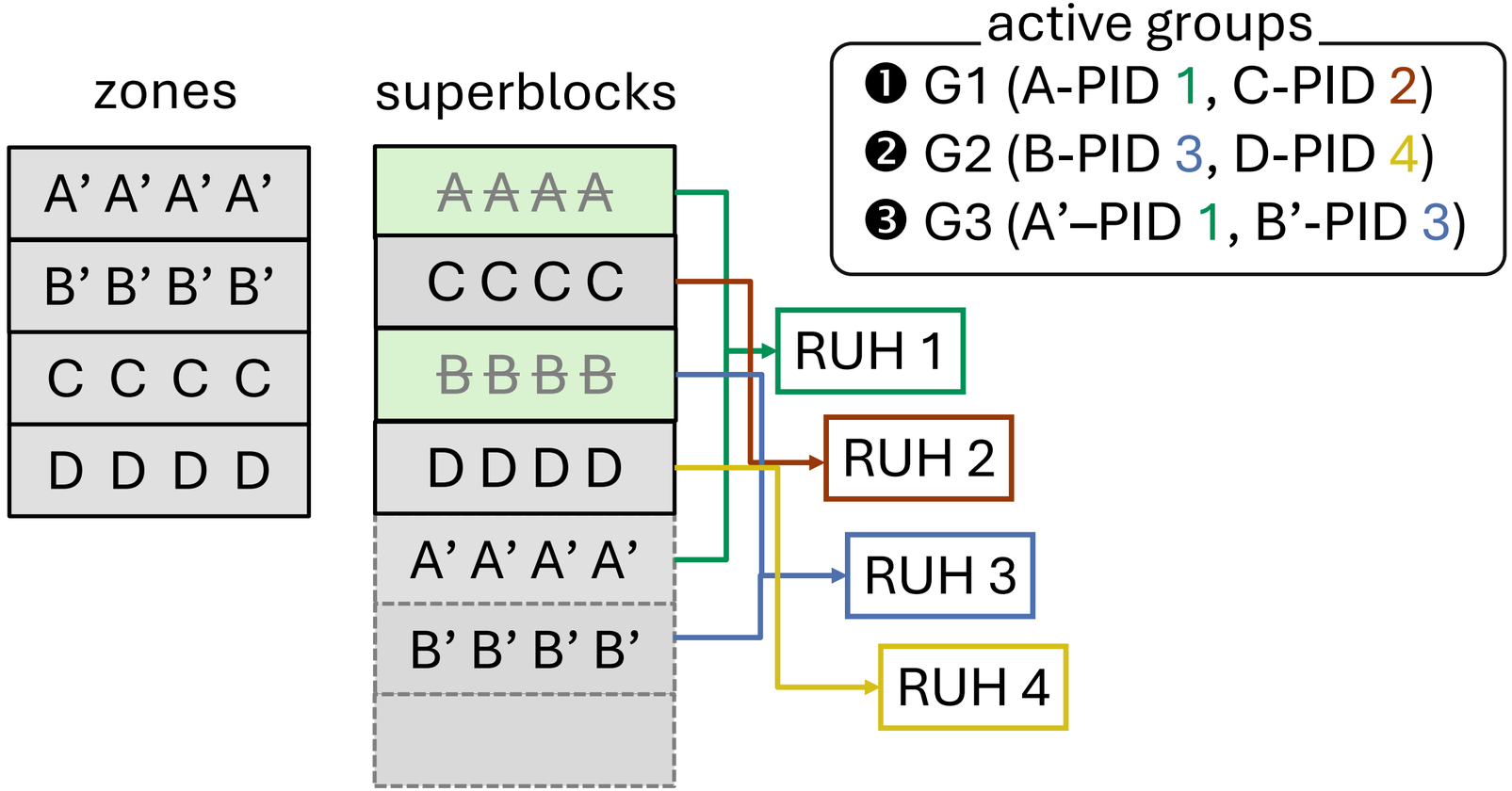
NoWA Pattern (3) integrating NoWA to DB GC

DB GC can select victim zones considering both **DB WA** and **compensation write**

- E.g., frequency imbalance-aware victim selection

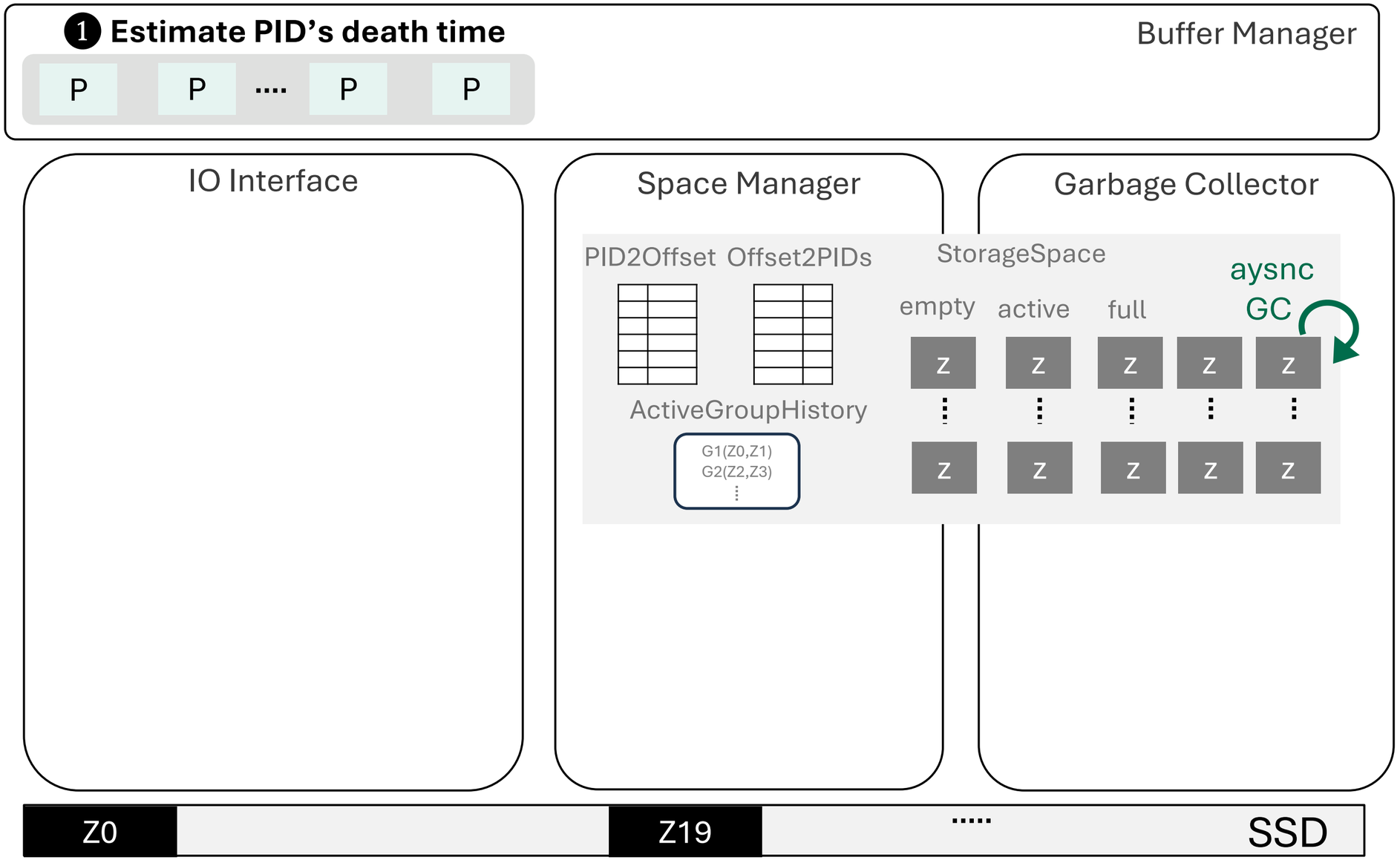
FDP SSD optimization: use placement hints

- Multiplexing is avoidable by using FDP placement hints
- E.g., **A-PID 1** means zone A with **Placement ID 1**

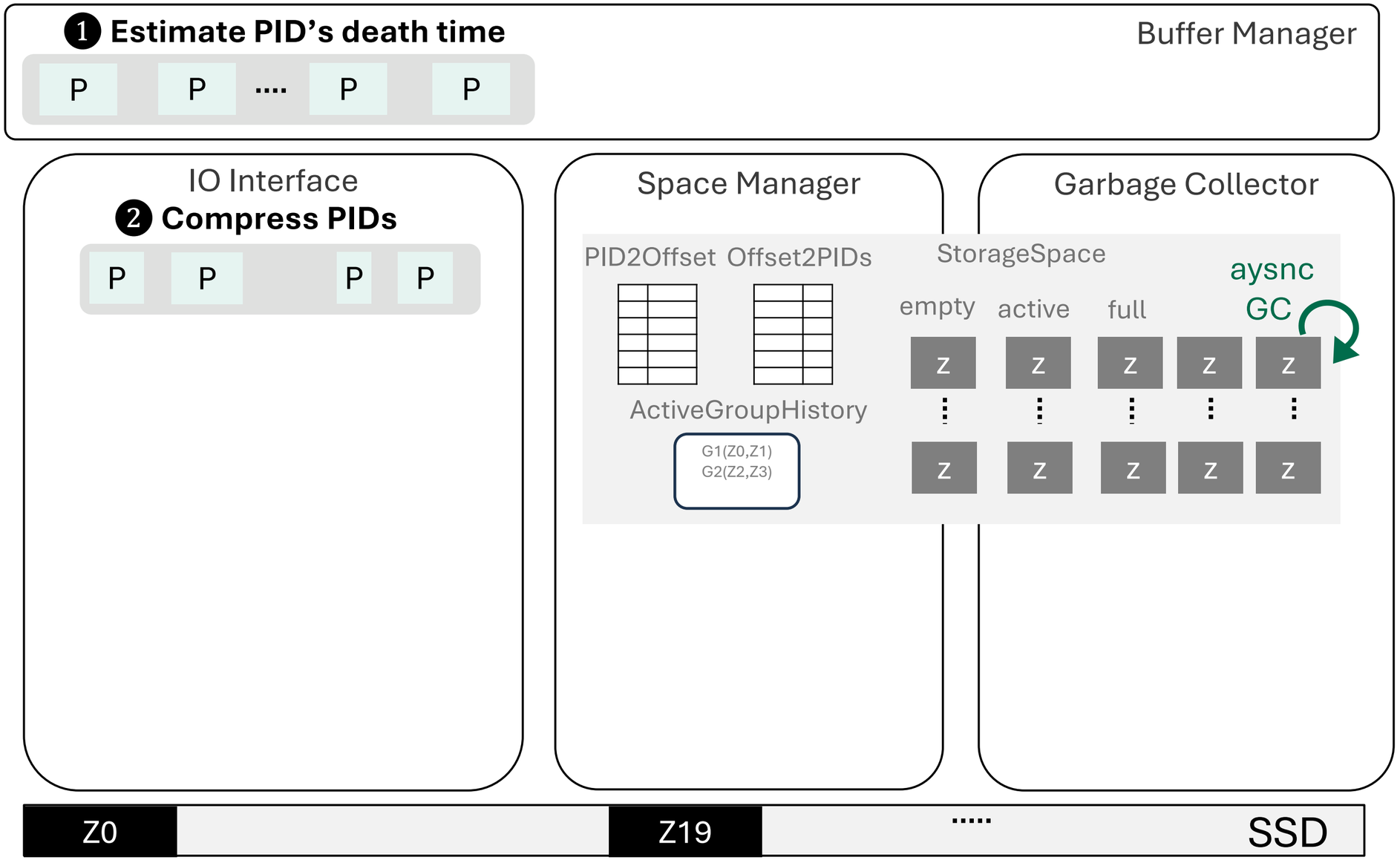


Putting everything all together

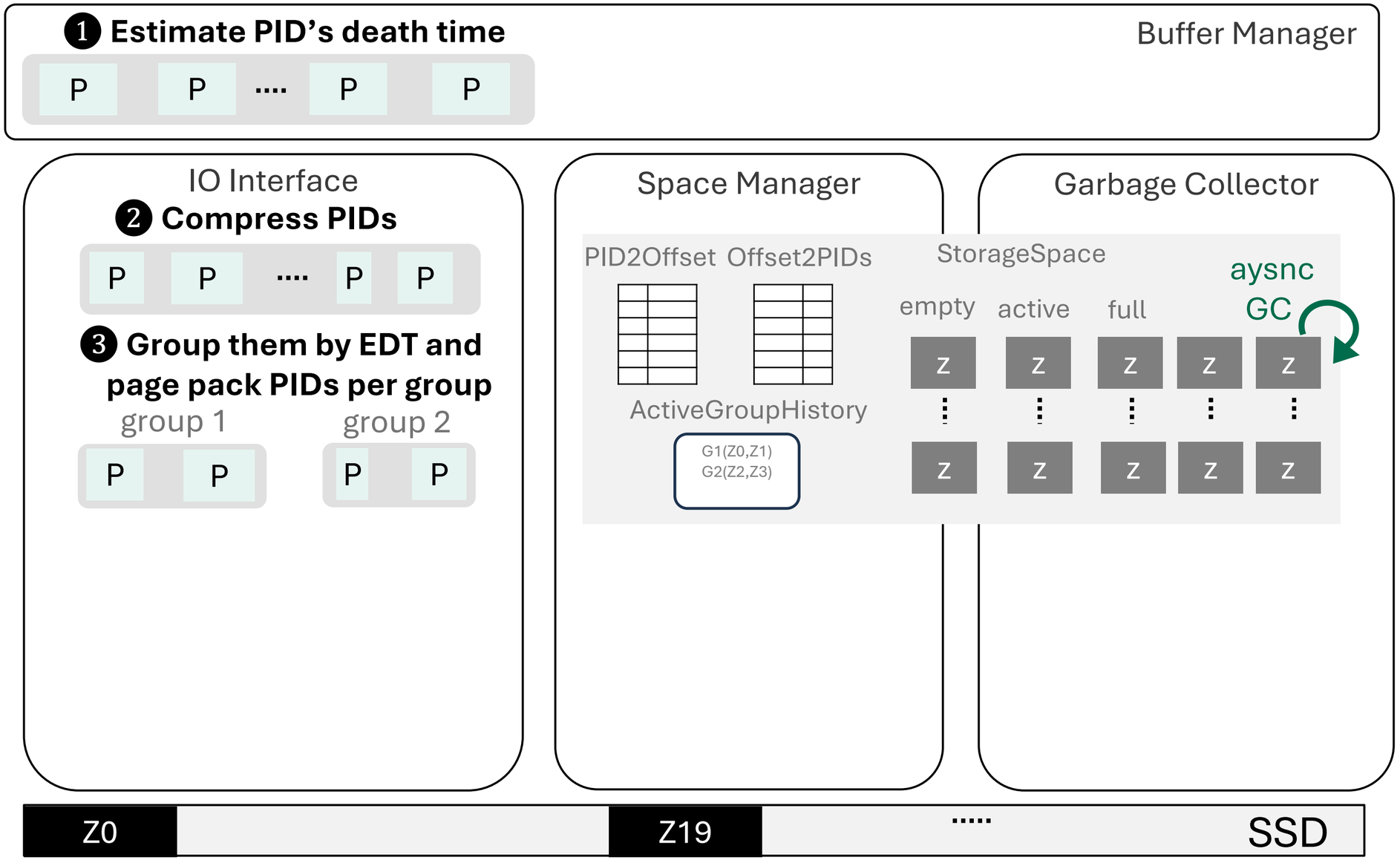
Overview of Write Path



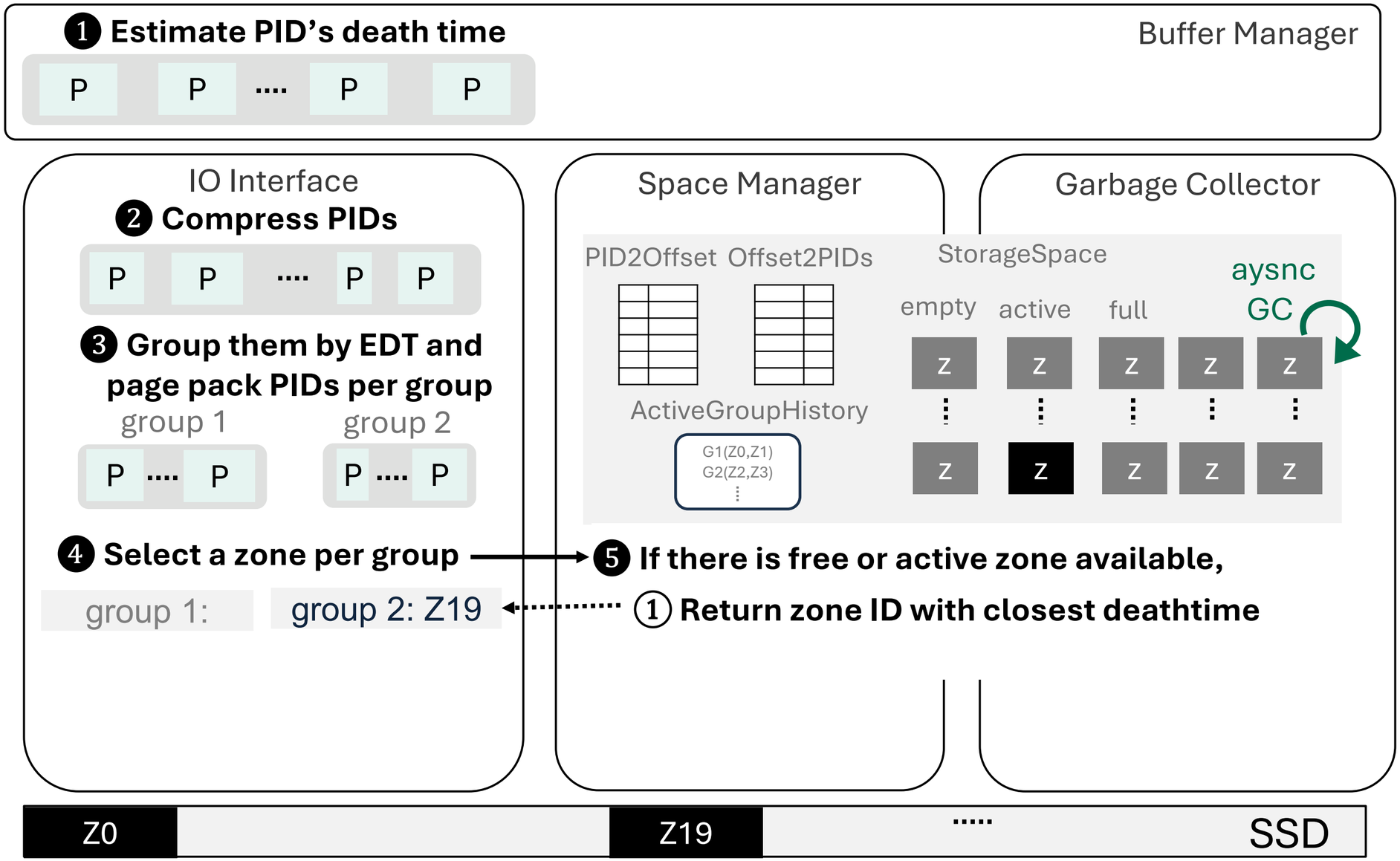
Overview of Write Path



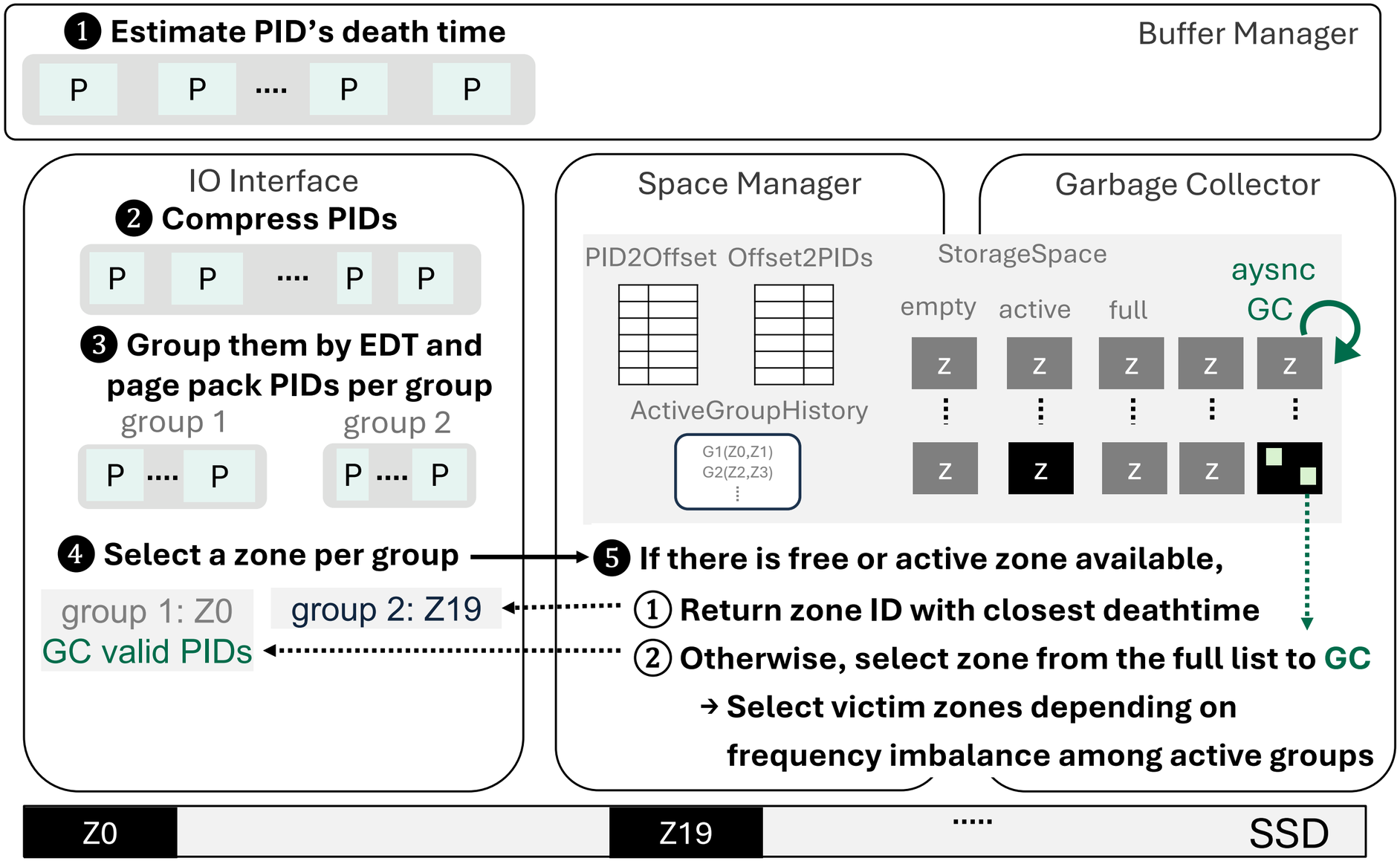
Overview of Write Path



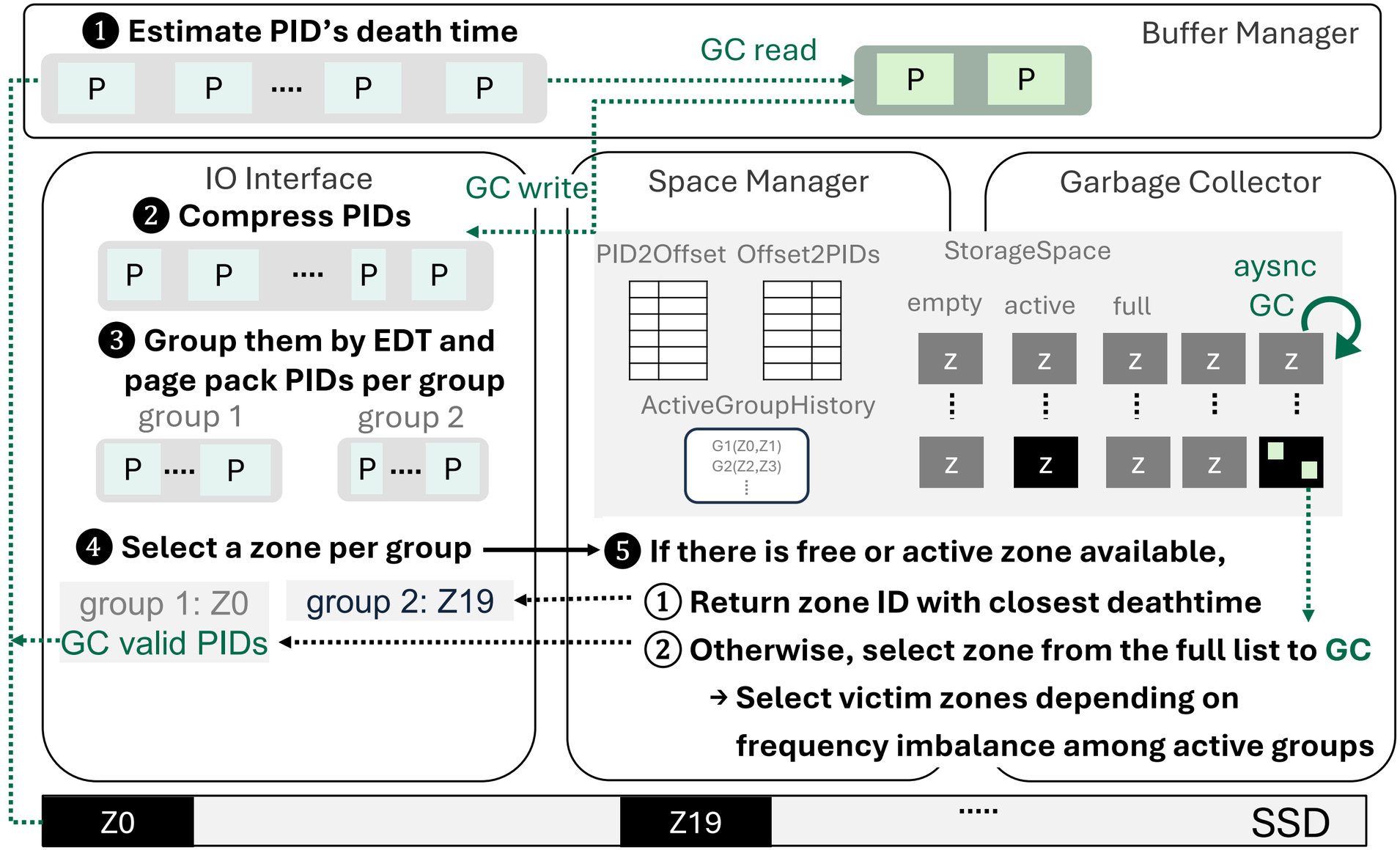
Overview of Write Path



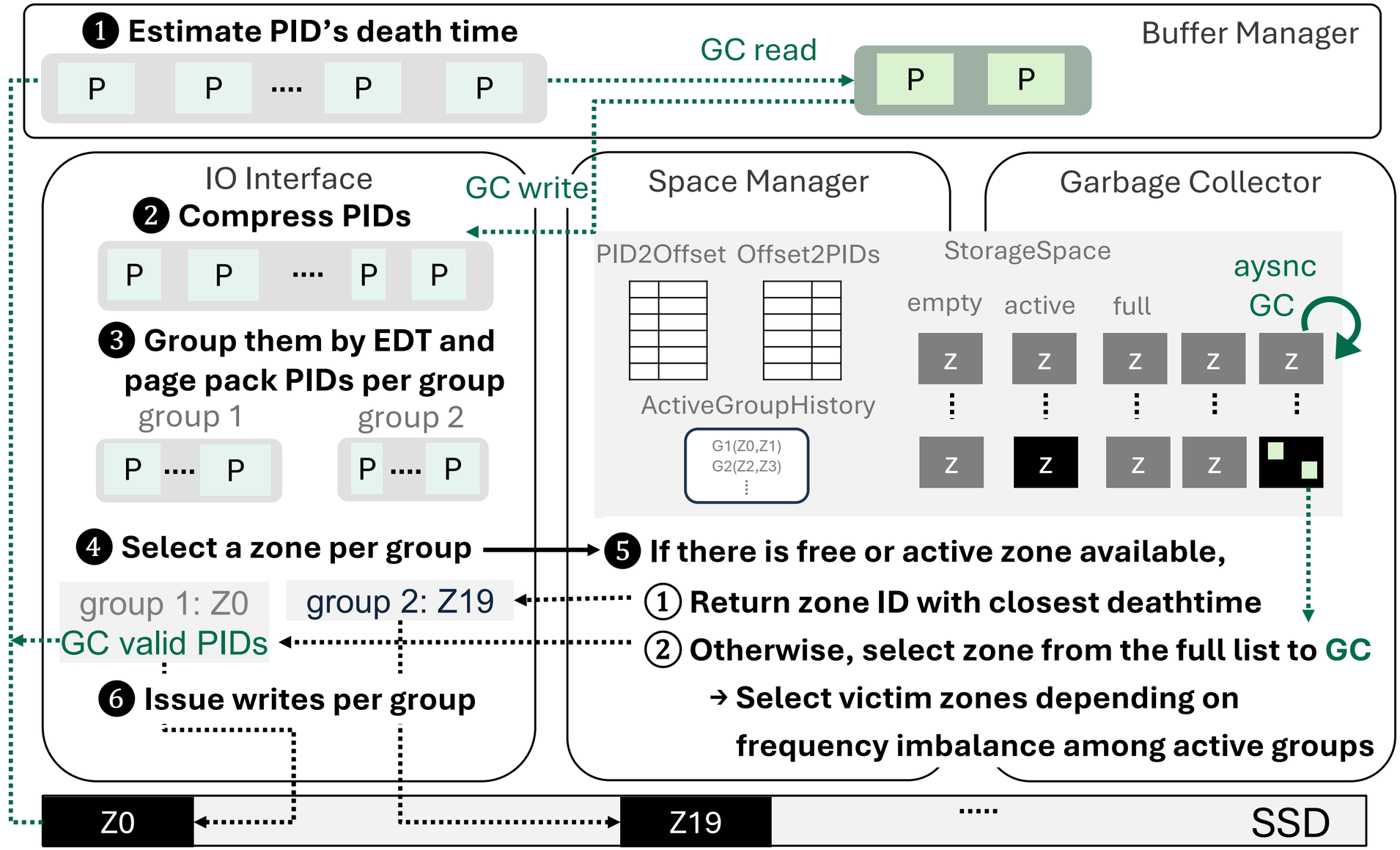
Overview of Write Path



Overview of Write Path (6)

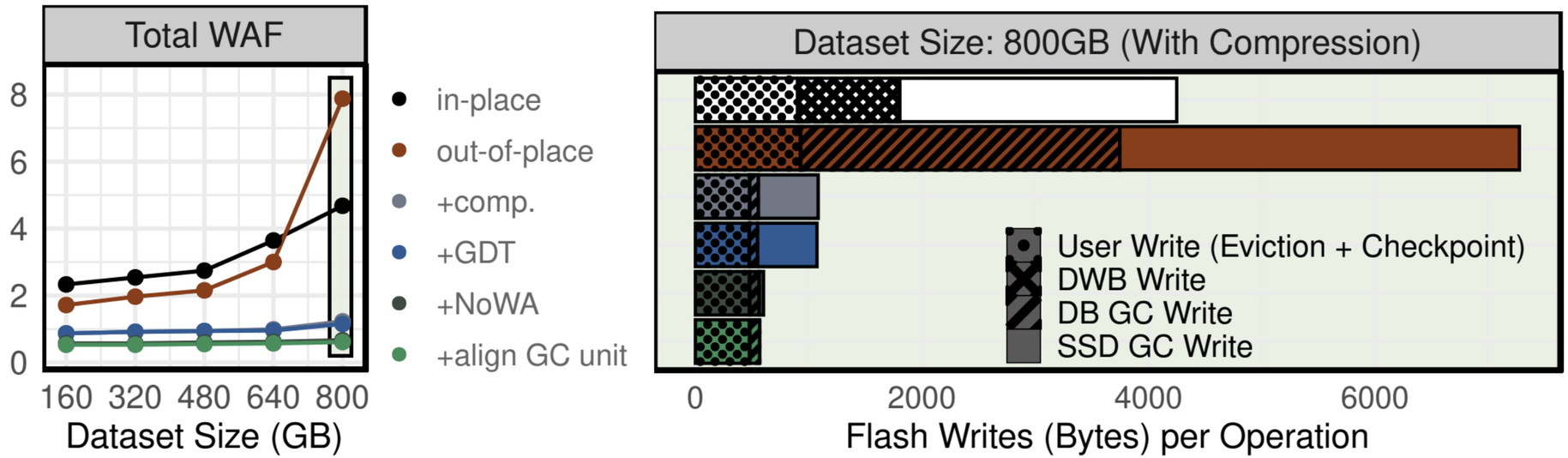


Overview of Write Path



Evaluation Results

Main Performance Result



(a) Total WAF (b) Write drilldown with compression

Total WAF on different dataset sizes and write drilldown with 800 GB dataset (YCSB-A zipf θ = 0.8, Samsung PM9A3).

Conclusion

Designing system writes for SSDs

switch from in-place to *out-of-place write*

↓

\+ compressing 4KiB pages

↓

\+ page packing compressed pages

↓

\+ deathtime-based placement & GC

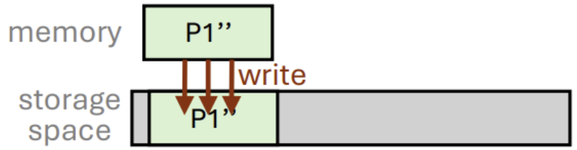
↓

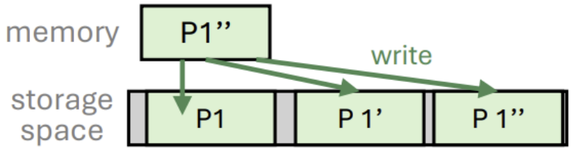
\+ aligning DB SSD GC unit size

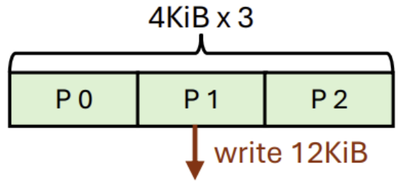
↓

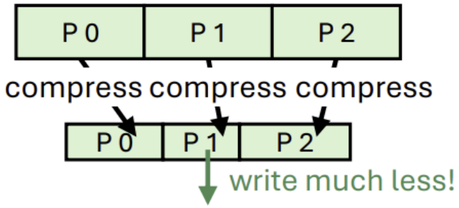
SSD WAF = 1 !

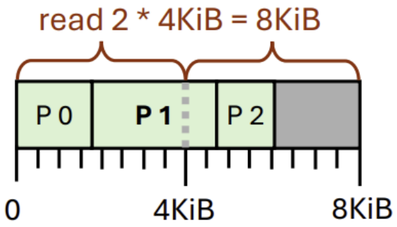
which SSD? → standard SSD | ZNS SSD | FDP SSD

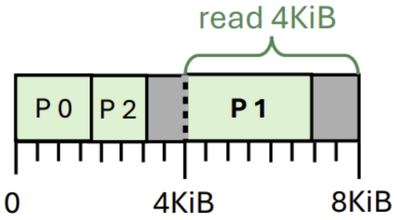


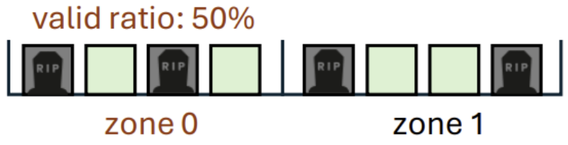


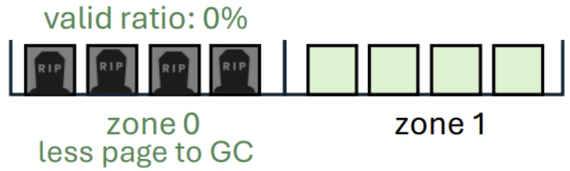


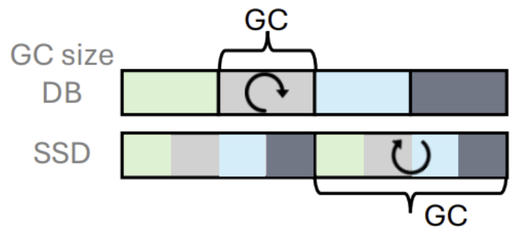


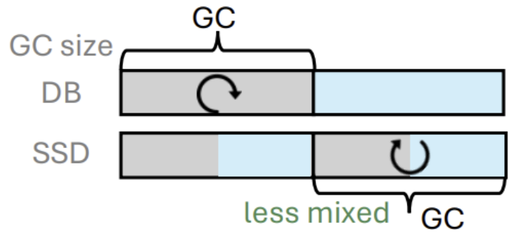












→ + NoWA pattern + zone cmds + use placement hints

Designing system writes for SSDs

switch from in-place to *out-of-place write*

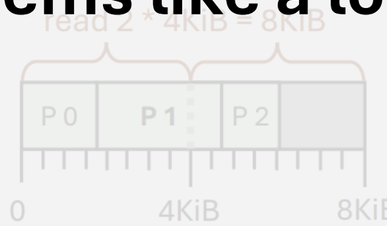
↓

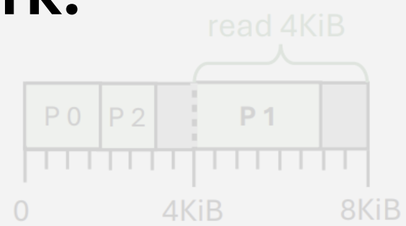
\+ compressing 4KiB pages

↓

\+ page packing compressed pages

It seems like a lot of work.





↓

\+ deathtime-based placement & GC

↓

\+ aligning DB SSD GC unit size

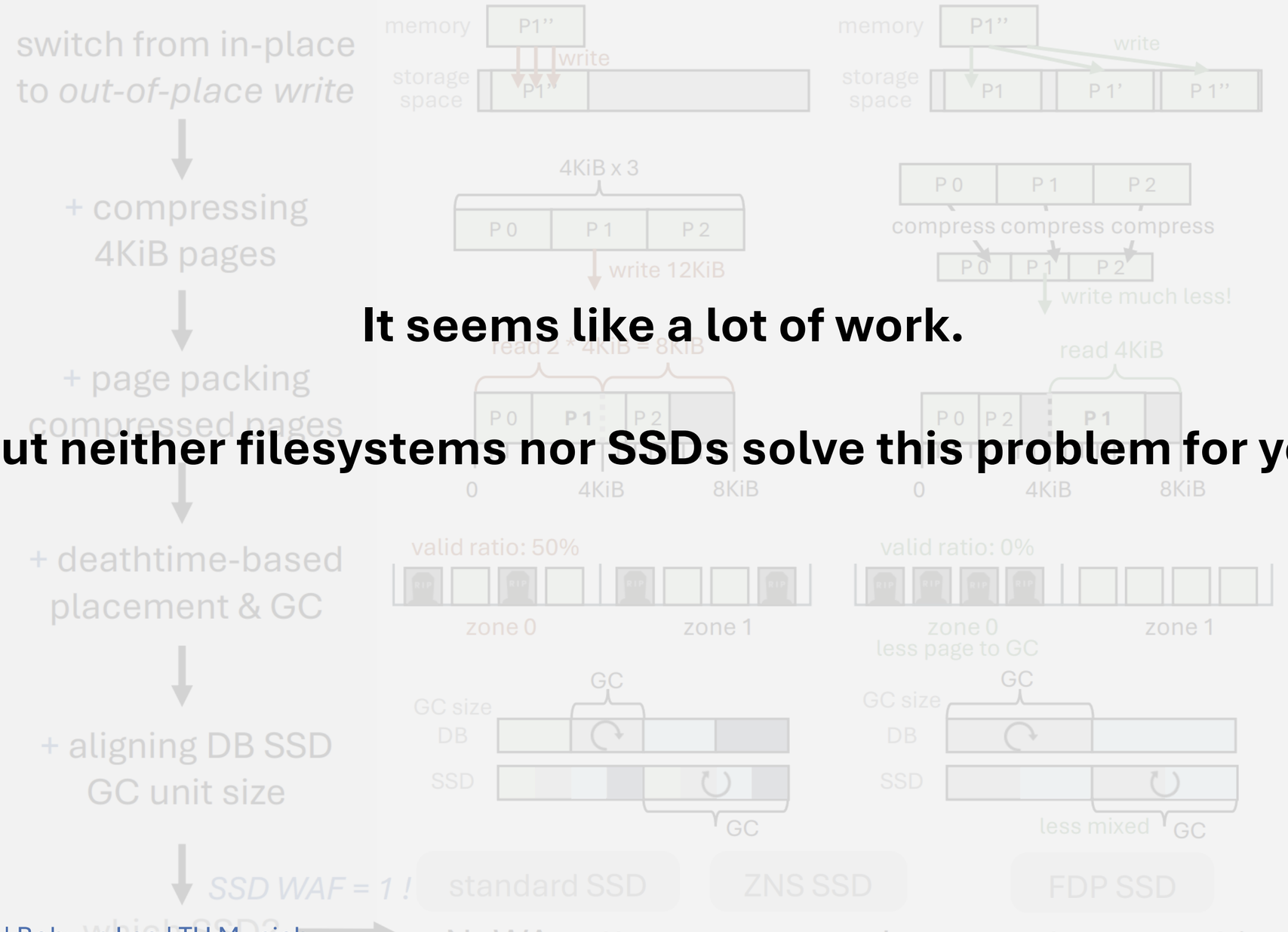
↓

SSD WAF = 1 !

standard SSD | ZNS SSD | FDP SSD

which SSDs → + NoWA pattern + zone cmds + use placement hints

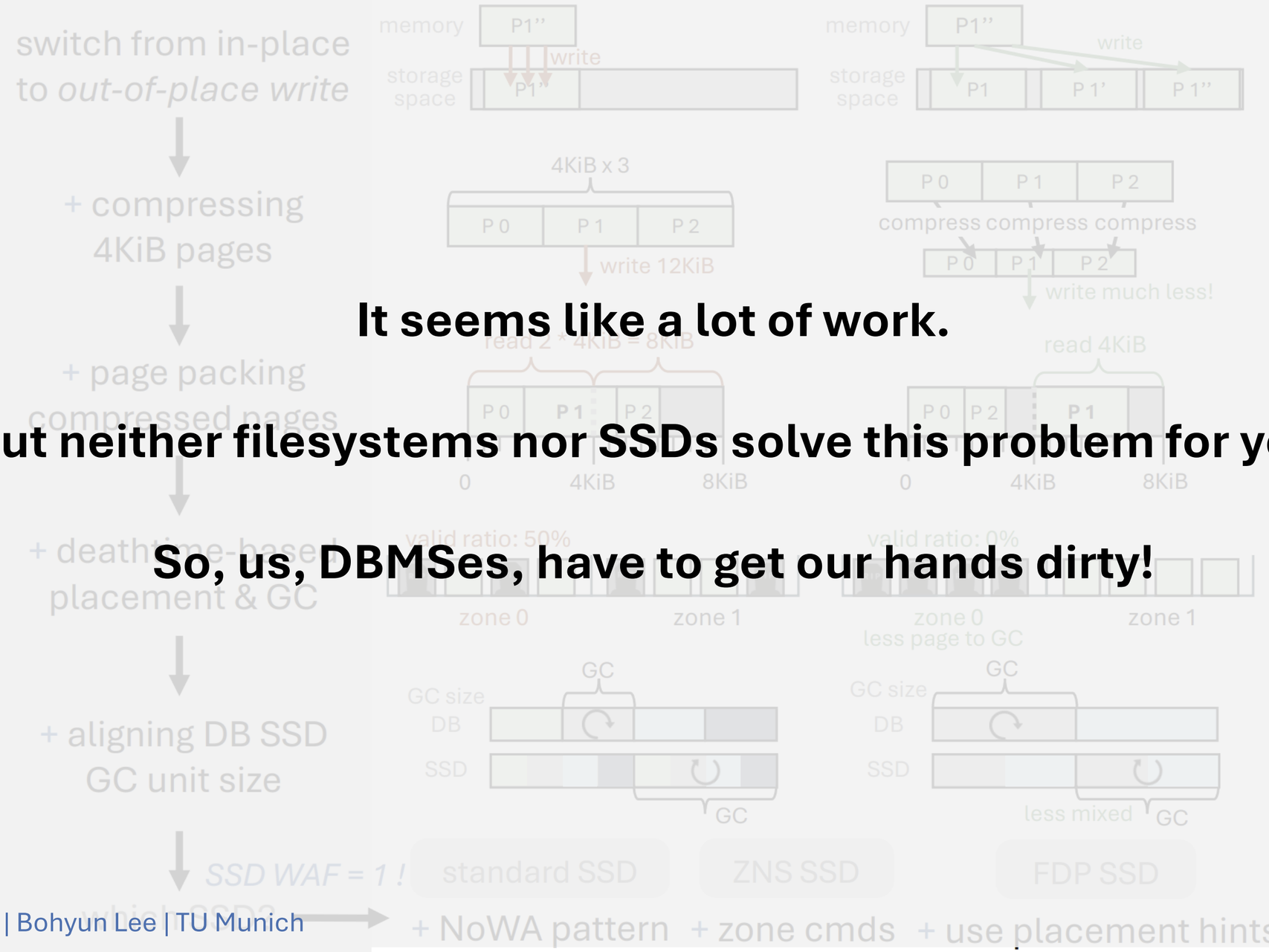
Designing system writes for SSDs



It seems like a lot of work.

But neither filesystems nor SSDs solve this problem for you.

Designing system writes for SSDs



It seems like a lot of work.

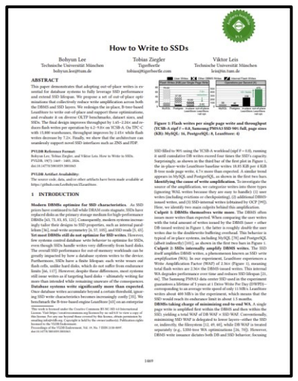
But neither filesystems nor SSDs solve this problem for you.

So, us, DBMSes, have to get our hands dirty!

Questions?

extended version

VLDB version



Code available at

zleanstorey

Varying Benchmark : TPC-C

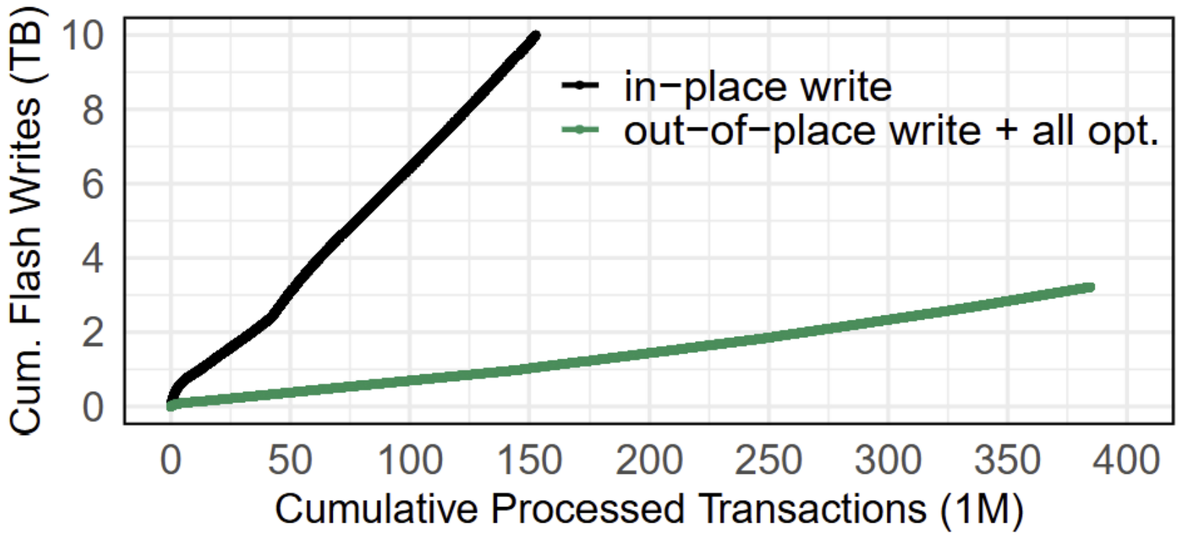


Figure 16: Comparing cumulative flash writes per tranasction: TPC-C (15,000 warehouses (=1,600GB), 32 clients, FDP SSD A)

Varying Skew Factor

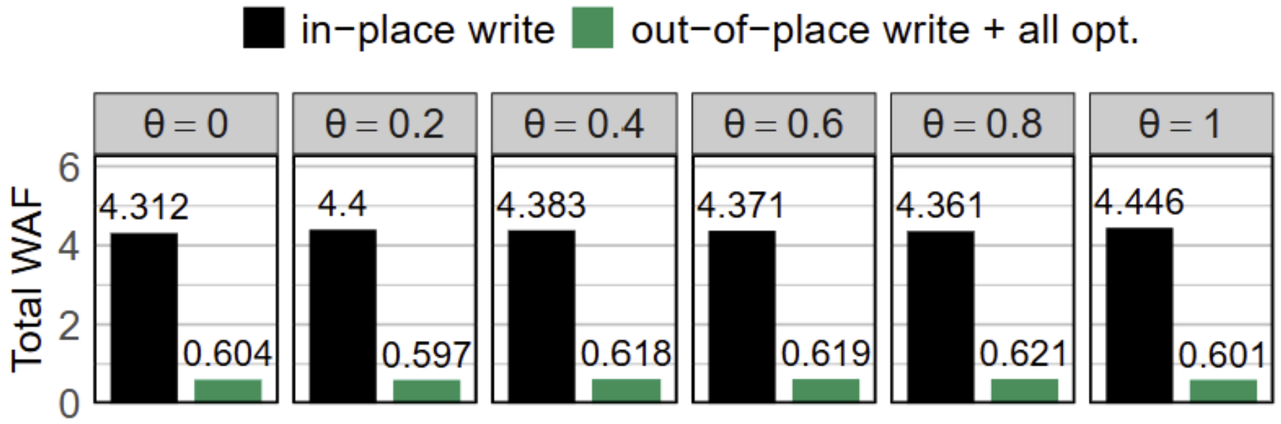


Figure 14: Total WAF while varying skewness. (YCSB-A, DB size: 800GB, Micron 7450 PRO)

Varying SSDs

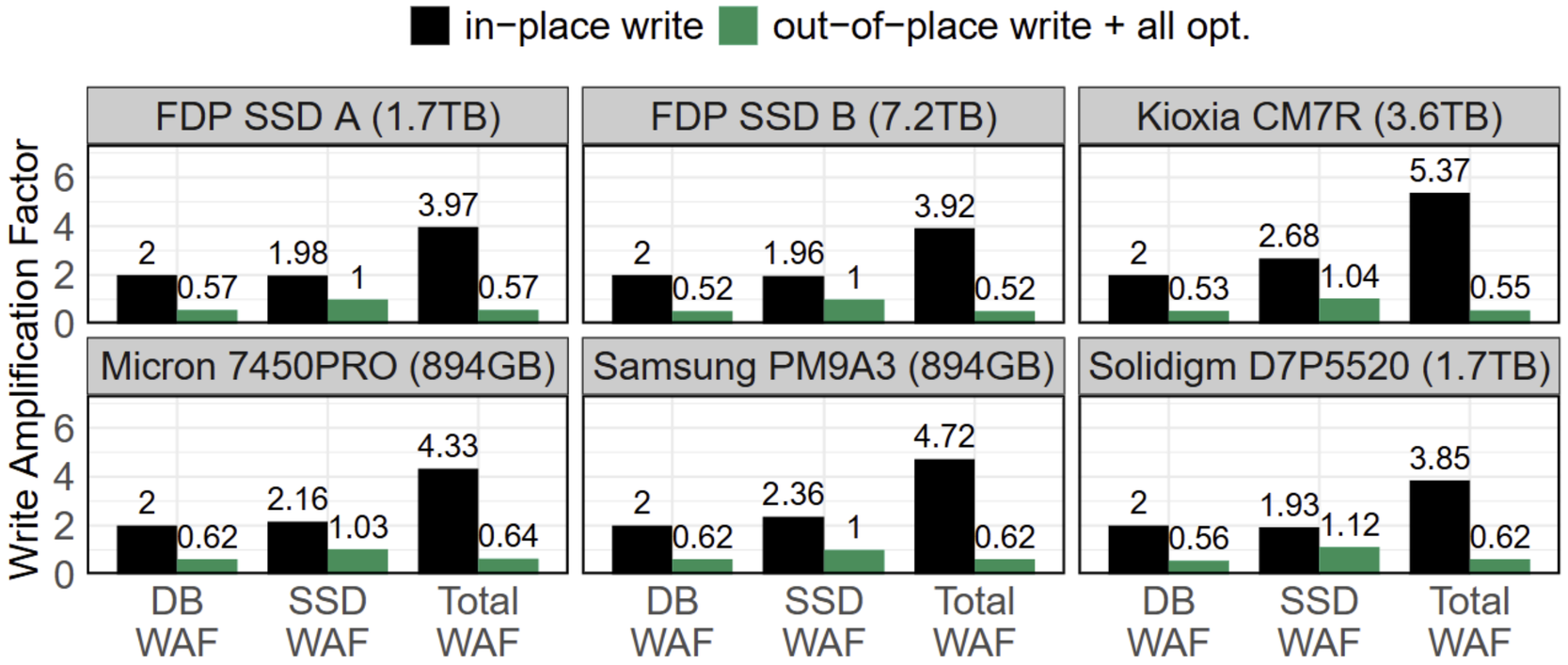


Figure 12: WAF comparison across different SSD models (YCSB-A zipfian θ = 0.8, SSD space 90% full)

Varying Dataset Size

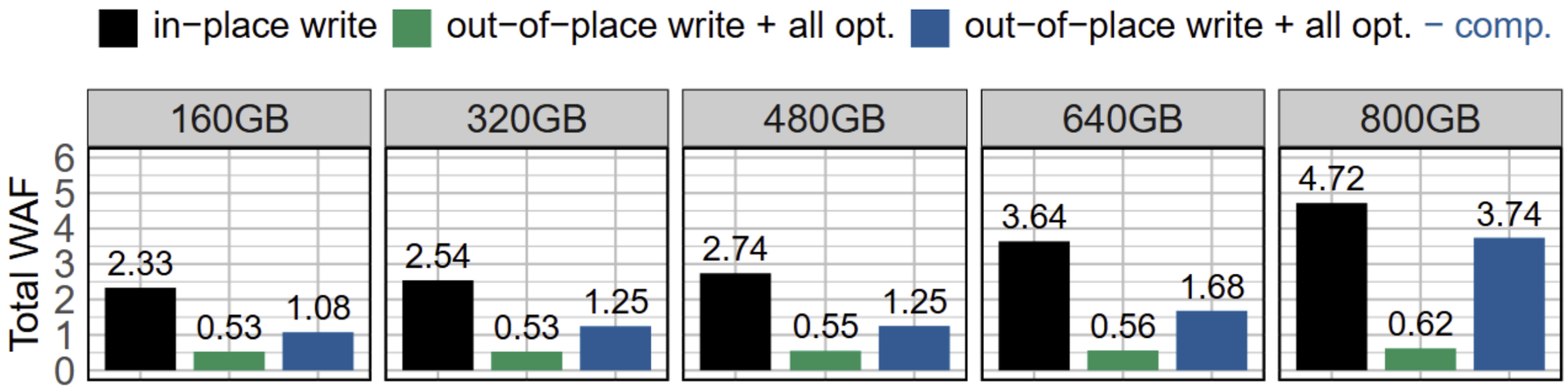


Figure 13: Total WAF while varying dataset size (YCSB-A zipfian θ = 0.8, Samsung PM9A3)